



RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE



Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

UNIVERSITÉ DES SCIENCES ET DE LA TECHNOLOGIE HOUARI BOUMEDIENE

Faculté et d'Informatique

Département SIQ

MÉMOIRE DE MASTER

Spécialité

RÉSEAU ET SYSTÈMES DISTRIBUÉS

Thème

Conception et déploiement d'une infrastructure
Cloud privée hyperconvergée basée sur
des technologies open source

Proposé et Dirigé par :

Mr. Neffah Mohammed

Prof. MERAZKA Fatiha

Réalisé et présenté par :

Mr. Bouzara Zakaria

Mr. Ouacherine Ilyes

Devant le jury composé de :

M

M

Soutenu le : ../../2026

Binôme N° 18/2026

Remerciements

Votre texte ici...

Table des matières

Table des matières	4
Table des figures	7
Liste des tableaux	8
Liste des abréviations	9
Introduction générale	11
I Présentation du Contexte	2
I.1 Introduction	3
I.2 Présentation de Sonatrach	3
I.3 Infrastructure Existante et Contexte Technologique	4
I.3.1 Infrastructure actuelle de Sonatrach	4
I.3.2 Les besoins en infrastructure légère pour les zones de test	4
I.4 Problématique	5
I.5 Objectifs du Projet	6
I.5.1 Objectif général	6
I.5.2 Objectifs spécifiques	6
I.6 Périmètre et Limitations du Projet	6
I.7 Conclusion	7
II État de l'Art et Concepts Théoriques	8
II.1 Introduction	9
II.2 L'Hyperconvergence (HCI)	9
II.2.1 Définition	9
II.2.2 Avantages de l'hyperconvergence	9
II.2.3 HCI face à l'Infrastructure Traditionnelle	10
II.3 Comparaison : Solutions Propriétaires et Open Source	10
II.3.1 Solutions propriétaires	10
II.3.2 Solutions open source	10
II.4 Analyse des Solutions Open Source HCI	11
II.4.1 OpenStack	11
II.4.2 Harvester HCI	11
II.4.3 Proxmox VE	12
II.4.4 Tableau comparatif	12
II.5 Présentation des Technologies Retenues	13
II.5.1 Proxmox VE : L'hyperviseur	13
II.5.2 Ceph : Le stockage distribué	17
II.5.3 Open vSwitch (OVS) : La virtualisation réseau	20
II.6 L'Équilibrage Dynamique de Charge dans les Clusters Virtualisés	23

II.6.1	Problématique du Déséquilibre de Charge	23
II.6.2	Mécanismes d'Équilibrage dans les Plateformes Existantes	23
II.6.3	L'Algorithme CSLB : Base Théorique du CLBS	23
II.7	Outils Complémentaires d'Infrastructure	24
II.7.1	HAProxy : Équilibrage de Charge Applicatif	24
II.7.2	Stack de Supervision : InfluxDB, Prometheus et Grafana	24
II.7.3	Ansible : Automatisation d'Infrastructure	25
II.7.4	PXE et TFTP : Provisionnement Réseau	25
II.7.5	OPNsense : Pare-feu et Sécurité Périphérique	26
II.7.6	NTP : Synchronisation Temporelle	26
II.7.7	GitLab CE : Intégration et Déploiement Continu	26
II.8	Conclusion	26
III	Conception de la Solution Proposée	27
III.1	Introduction	28
III.2	Vue Globale de l'Architecture	28
III.3	Organisation de la Solution	29
III.4	Partie I : Calcul	30
III.4.1	Infrastructure Physique	31
III.4.2	Architecture du Cluster Proxmox	31
III.4.2.1	Organisation du Cluster	31
III.4.2.2	Mécanisme de Quorum et Corosync	32
III.4.2.3	Haute Disponibilité	32
III.4.2.4	Matrice des Scénarios de Défaillance	32
III.4.3	Mécanisme d'Équilibrage Dynamique : CLBS	33
III.4.3.1	Contexte et Besoin	33
III.4.3.2	Base Algorithmique : CSLB	33
III.4.3.3	Modifications Apportées au CSLB	34
III.4.3.4	Algorithme de Décision (Pseudocode)	35
III.4.4	Application de Démonstration : Sona-Web	37
III.4.4.1	Architecture Applicative	37
III.4.4.2	Scénario de Validation HA	38
III.4.5	Architecture du Monitoring	38
III.4.6	Automatisation	38
III.4.6.1	Ansible : Périmètre et Playbooks	40
III.4.6.2	Pipeline PXE/TFTP	40
III.5	Partie II : Stockage	42
III.5.1	Architecture du Cluster Ceph	42
III.5.1.1	Composants du Cluster	42
III.5.1.2	Dimensionnement du Pool	43
III.5.1.3	Configuration CRUSH et Domaines de Défaillance	43
III.6	Partie III : Réseau	44
III.6.1	Contexte et Contraintes Réseau	44
III.6.2	Topologie Réseau	44
III.6.2.1	Deux Niveaux de Segmentation	44
III.6.2.2	Bridges Open vSwitch par Nœud	46
III.6.3	Plan d'Adressage	46
III.6.4	VM OPNsense : Routeur et Pare-feu Centralisé	47
III.6.4.1	Interfaces et Rôles	47
III.6.5	Architecture du Pare-feu	47

III.6.5.1 Pare-feu OPNsense : Filtrage Inter-réseaux	47
III.6.5.2 Pare-feu Proxmox : Filtrage au Niveau Hyperviseur	48
III.7 Services Complémentaires d'Infrastructure	49
III.7.1 Système de Notification : Grafana Alertmanager	49
III.7.2 Centralisation des Journaux : Loki, Alloy et Grafana	50
III.7.3 Synchronisation Temporelle : Serveur NTP	51
III.7.4 Gestion des Identités : Proxmox et Perspectives	52
III.7.5 Gestion du Code Source et CI/CD : GitLab	52
III.7.6 Automatisation Opérationnelle : Rundeck	53
III.8 Conclusion	53
IV Implémentation de la Solution Proposée	54
IV.1 Introduction	55
IV.2 Partie I : Couche Calcul	55
IV.2.1 Préparation de l'Infrastructure Physique	55
IV.2.1.1 Installation de Proxmox VE sur les nœuds	55
IV.2.1.2 Configuration de la connectivité Internet via le laptop gateway	56
IV.2.2 Formation du cluster Proxmox et haute disponibilité	58
IV.2.2.1 Création et jonction au cluster	58
IV.2.2.2 Configuration de la migration à chaud	59
IV.2.3 Déploiement de la stack de monitoring	61
IV.2.3.1 Déploiement d'InfluxDB	61
IV.2.3.2 Déploiement de Prometheus	61
IV.2.3.3 Déploiement de Grafana	63
IV.2.4 Implémentation du mécanisme CLBS	64
IV.2.4.1 Structure du service	65
IV.2.4.2 Logique de décision	65
IV.2.4.3 Déploiement comme service systemd	65
IV.2.4.4 Validation du mécanisme	66
IV.2.5 Déploiement de l'application Sona-Web	67
IV.2.5.1 Création des machines virtuelles	67
IV.2.5.2 Résultats de l'application Sona-Web	67
IV.2.6 Automatisation avec Ansible	68
IV.2.6.1 Environnement et structure du projet	68
IV.2.6.2 Inventaire	69
IV.2.6.3 Playbook de provisionnement	70
IV.2.6.4 Playbook de bootstrap	70
IV.2.6.5 Playbook de synchronisation NTP	71
IV.2.6.6 Provisionnement en lot de VMs	71
IV.3 Partie II : Couche Stockage	72
IV.3.1 Déploiement du cluster Ceph	72
IV.3.1.1 Initialisation de Ceph	72
IV.3.1.2 Déploiement des MON et MGR	72
IV.3.1.3 Déploiement des OSDs	73
IV.3.1.4 Création du pool de stockage	74
IV.3.2 Validation de la tolérance aux pannes	76
IV.4 Partie III : Couche Réseau	76
IV.4.1 Configuration Open vSwitch et tunnels VxLAN	76
IV.4.1.1 Installation d'Open vSwitch	76
IV.4.1.2 Configuration complète des interfaces (exemple sur PVE-01)	77

IV.4.2	Déploiement de la VM OPNsense	81
IV.4.2.1	Création et interfaces réseau	81
IV.4.2.2	NAT sortant	82
IV.4.3	Configuration du service DHCP	83
IV.4.4	Configuration du Pare-feu	83
IV.4.4.1	Pare-feu OPNsense : Filtrage Inter-réseaux	83
IV.4.4.2	Pare-feu Proxmox : Filtrage au Niveau Hyperviseur	84
IV.4.5	Validation de la haute disponibilité réseau et applicative	87
IV.5	Partie IV : Services Complémentaires d’Infrastructure	87
IV.5.1	Système de Notification : Alertmanager	88
IV.5.1.1	Configuration des règles d’alerte	88
IV.5.1.2	Configuration du contact point Gmail	88
IV.5.1.3	Validation	88
IV.5.2	Centralisation des Journaux : Loki et Alloy	89
IV.5.2.1	Déploiement de Loki	89
IV.5.2.2	Déploiement de l’agent Grafana Alloy	89
IV.5.3	Synchronisation Temporelle : NTP/Chrony	90
IV.5.3.1	Configuration du serveur Chrony sur PVE-01	90
IV.5.3.2	Configuration des clients NTP	91
IV.5.3.3	Déploiement via Ansible	91
IV.5.3.4	Validation de la synchronisation	91
IV.5.4	Gestion du Code Source et CI/CD : GitLab	92
IV.5.4.1	Déploiement du LXC Turnkey GitLab CE	92
IV.5.4.2	Création des dépôts	92
IV.5.4.3	Configuration des pipelines CI/CD	93
IV.5.4.4	Validation	93
IV.5.5	Automatisation Opérationnelle : Rundeck	93
IV.5.5.1	Déploiement du LXC Rundeck	93
IV.5.5.2	Intégration du dépôt ansible-infra via GitLab Runner	94
IV.5.5.3	Définition des jobs	94
IV.5.5.4	Validation	95
IV.6	Conclusion	95
Conclusion générale		96
Bibliographie		98
A Dashboards Grafana de Supervision		101
B Configuration Complète des Interfaces Réseau		107
C Résultat d’Exécution Rundeck		120

Table des figures

II.1	Pile de virtualisation KVM + QEMU dans Proxmox VE	14
III.1	Architecture Globale de l'Infrastructure HCI	29
III.2	Organisation de la Solution en Trois Parties et leurs Interactions	30
III.3	Pseudocode de l'algorithme CLBS	35
III.4	Logique de Décision du Mécanisme CLBS	36
III.5	Architecture Applicative Sona-Web	37
III.6	Pipeline d'Automatisation : PXE/TFTP et Ansible	39
III.7	Topologie Réseau et Plan VxLAN	45
III.8	Architecture du Pare-feu : OPNsense et Proxmox Firewall	48
III.9	Diagramme de déploiement : répartition des services sur les trois nœuds physiques	49
III.10	Pipeline de notification Grafana Alertmanager	50
III.11	Architecture de centralisation des journaux : Alloy, Loki et Grafana	51
III.12	Hierarchie NTP : synchronisation temporelle de l'infrastructure	52
III.13	Pipeline CI/CD GitLab pour les projets <code>clbs</code> et <code>ansible-infra</code>	53
IV.1	Interface web Proxmox au premier login sur PVE-01.	56
IV.2	Activation du routage IP sur le laptop	56
IV.3	Configuration NAT avec iptables	57
IV.4	Persistance des règles iptables	57
IV.5	Configuration réseau de base d'un nœud Proxmox	57
IV.6	Ajout des routes statiques vers OPNsense	57
IV.7	Création du cluster Proxmox	58
IV.8	Jonction de PVE-02 et PVE-03 au cluster	58
IV.9	Vérification de l'état du cluster	58
IV.10	Vue du cluster Proxmox montrant les trois nœuds en ligne.	59
IV.11	Avant la migration live d'une machine virtuelle.	60
IV.12	Après la migration live d'une machine virtuelle.	60
IV.13	Explorateur InfluxDB montrant les métriques reçues depuis Proxmox.	61
IV.14	Activation du module Prometheus sur les nœuds Ceph	62
IV.15	Extrait du fichier <code>prometheus.yml</code>	62
IV.16	Interface Prometheus montrant les cibles Ceph et CLBS avec l'état UP.	63
IV.17	Dashboard Grafana Infrastructure pour les nœuds Proxmox.	64
IV.18	Dashboard Grafana Ceph affichant l'état du cluster, des pools et la latence des OSDs.	64
IV.19	Gestion du service CLBS avec <code>systemd</code>	65
IV.20	Journaux <code>systemd</code> du service CLBS lors d'un cycle de détection et de migration.	66
IV.21	Génération d'une charge CPU de test	66
IV.22	Dashboard CLBS montrant une migration déclenchée automatiquement.	67
IV.23	Interface principale de l'application Sona-Web.	68

IV.24	Structure du projet Ansible	68
IV.25	Fichier ansible.cfg	69
IV.26	Extrait de l'inventaire hosts.yml	69
IV.27	Exemples d'utilisation du playbook provision_vm.yml	70
IV.28	Formule d'attribution automatique de l'adresse IP	70
IV.29	Configuration cloud-init d'une VM Proxmox	70
IV.30	Exemple d'utilisation du playbook bootstrap.yml	71
IV.31	Exemples d'utilisation du playbook ntp_sync.yml	71
IV.32	Exemple de liste de VMs pour provision_vm_batch.yml	71
IV.33	Initialisation du cluster Ceph	72
IV.34	Création d'un monitor Ceph	73
IV.35	Interface Proxmox affichant les monitors et managers Ceph actifs sur les trois nœuds.	73
IV.36	Activation du module Prometheus pour Ceph	73
IV.37	Création des OSDs Ceph sur chaque nœud	74
IV.38	État du cluster Ceph montrant les douze OSDs actifs sur les trois nœuds.	74
IV.39	Affichage de l'arbre CRUSH	74
IV.40	Création du pool Ceph	75
IV.41	État global du cluster Ceph	75
IV.42	État du cluster Ceph montrant le pool RBD.	75
IV.43	Installation d'Open vSwitch	76
IV.44	Création du bridge OVS principal	76
IV.45	Vue d'ensemble de la configuration réseau de PVE-01 : quatre parties logiques	78
IV.46	Interfaces de base et bridge OVS de management	78
IV.47	VxLAN Corosync (clé 20) et Ceph interne (clé 30) – interfaces internes PVE-01	79
IV.48	Ceph public (clé 40), migration (clé 50) et sauvegarde (clé 90) – interfaces internes PVE-01	80
IV.49	VxLAN applicatifs (clés 60, 70, 80) et bridges Linux – PVE-01	80
IV.50	Configuration des interfaces de la VM OPNsense.	82
IV.51	Capture du trafic DHCP avec tcpdump	83
IV.52	Paramètres SMTP du contact point Gmail	88
IV.53	Règles d'alerte configurées dans Grafana Alertin.	88
IV.54	Installation de Loki	89
IV.55	Configuration de l'agent Alloy pour la collecte des journaux	89
IV.56	Journaux de l'infrastructure visualisés dans Grafana via Loki.	90
IV.57	Configuration du serveur Chrony sur PVE-01	90
IV.58	Configuration Chrony côté clients (nœuds et VMs)	91
IV.59	Sortie de chronyc -n clients sur PVE-01.	92
IV.60	Page Projects de l'instance GitLab avec les dépôts de l'infrastructure.	93
IV.61	Installation de Rundeck	93
IV.62	Interface Rundeck affichant les jobs de provisionnement Ansible.	95
A.1	Dashboard Grafana Infrastructure – vue complète des métriques CPU, mé- moire et réseau des trois nœuds Proxmox.	102
A.2	Dashboard Grafana Ceph – vue complète de l'état du cluster, des pools et de la latence des OSDs.	103
A.3	Dashboard Grafana Loki – journaux système centralisés des nœuds Prox- mox et des VMs.	105
B.1	Fichier /etc/network/interfaces complet – PVE-01 (192.168.10.100)	107

B.2	Fichier <code>/etc/network/interfaces</code> complet – PVE-02 (192.168.10.213)	111
B.3	Fichier <code>/etc/network/interfaces</code> complet – PVE-03 (192.168.10.84)	115
C.1	Résultat d'exécution d'un job Rundeck avec le journal Ansible complet. .	120

Liste des tableaux

II.1	Comparaison entre l'infrastructure traditionnelle et l'infrastructure HCI . . .	10
II.2	Comparaison des solutions HCI open source : OpenStack, Harvester et Proxmox VE	12
II.3	Comparaison entre une machine virtuelle KVM et un conteneur LXC	15
II.4	Récapitulatif des composants de la pile technique Proxmox VE	17
III.1	Dimensionnement physique des serveurs Dell PowerEdge R720	31
III.2	Adresses IP de management des nœuds Proxmox	32
III.3	Matrice des scénarios de défaillance et comportements attendus	33
III.4	Comparaison des protocoles envisagés dans le pipeline PXE	41
III.5	Répartition du stockage Ceph par usage	43
III.6	Paramètres CRUSH appliqués automatiquement par Proxmox	44
III.7	Bridges OVS et tunnels VxLAN par nœud Proxmox	46
III.8	Adressage IP statique des nœuds Proxmox par réseau logique	47
IV.1	Tableaux de bord Grafana configurés.	63
IV.2	Interfaces de la VM OPNsense.	82
IV.3	Plages DHCP configurées sur OPNsense.	83
IV.4	Règles de filtrage inter-réseaux configurées sur OPNsense.	84
IV.5	Alias définis au niveau datacenter dans le pare-feu Proxmox.	84
IV.6	IPSets configurés au niveau datacenter.	85
IV.7	Groupes de sécurité Proxmox et leurs règles de ports par service.	85
IV.8	Flux autorisés au niveau nœud via les groupes de sécurité (PVE-01, PVE-02, PVE-03).	86
IV.9	Règles de pare-feu par VM applicative.	87

Liste des abréviations

API	: Application Programming Interface
BIOS	: Basic Input/Output System
BSD	: Berkeley Software Distribution
CD	: Continuous Deployment
CI	: Continuous Integration
CLI	: Command Line Interface
CLBS	: Central Load Balancing Scheduler
CPU	: Central Processing Unit
CRUSH	: Controlled Replication Under Scalable Hashing
CSLB	: Central Scheduler Load Balancing
DHCP	: Dynamic Host Configuration Protocol
DMZ	: DeMilitarized Zone
DNS	: Domain Name System
DRS	: Distributed Resource Scheduler
EC	: Erasure Coding
FTP	: File Transfer Protocol
GRE	: Generic Routing Encapsulation
GUI	: Graphical User Interface
HA	: High Availability
HCI	: Hyper Converged Infrastructure
HPC	: High Performance Computing
HTTP	: HyperText Transfer Protocol
HTTPS	: HyperText Transfer Protocol Secure
I/O	: Input / Output
ICMP	: Internet Control Message Protocol
IP	: Internet Protocol
ISO	: International Organization for Standardization (image disque)
KPI	: Key Performance Indicator
KVM	: Kernel-based Virtual Machine
LACP	: Link Aggregation Control Protocol
LAN	: Local Area Network
LDAP	: Lightweight Directory Access Protocol
LXC	: Linux Containers
MDS	: MetaData Server
MGR	: Manager
MON	: Monitor
NAT	: Network Address Translation
NIC	: Network Interface Card
NTP	: Network Time Protocol
OS	: Operating System

OSD	: Object Storage Daemon
OVMF	: Open Virtual Machine Firmware
OVS	: Open Virtual Switch
PBS	: Proxmox Backup Server
PC	: Personal Computer
PG	: Placement Group
PVE	: Proxmox Virtual Environment
PXE	: Preboot eXecution Environment
QEMU	: Quick EMUlator
QoS	: Quality of Service
RADOS	: Reliable Autonomic Distributed Object Store
RAM	: Random Access Memory
RBD	: RADOS Block Device
REST	: Representational State Transfer
RSD	: Réseau Et Systèmes Distribués
RTO	: Recovery Time Objective
SAN	: Storage Area Network
SAS	: Serial Attached SCSI
SDN	: Software Defined Networking
SFTP	: SSH File Transfer Protocol
SIEM	: Security Information and Event Management
SMTP	: Simple Mail Transfer Protocol
SPOF	: Single Point Of Failure
SSH	: Secure Shell
TCO	: Total Cost of Ownership
TCP	: Transmission Control Protocol
TFTP	: Trivial File Transfer Protocol
TLS	: Transport Layer Security
UDP	: User Datagram Protocol
UEFI	: Unified Extensible Firmware Interface
USB	: Universal Serial Bus
vCPU	: Virtual Central Processing Unit
VE	: Virtual Environment
VLAN	: Virtual Local Area Network
VM	: Virtual Machine
VNI	: VXLAN Network Identifier
VPN	: Virtual Private Network
VxLAN	: Virtual eXtensible LAN
WAL	: Write-Ahead Log
WAN	: Wide Area Network

Introduction générale

L'infrastructure informatique des grandes entreprises repose aujourd'hui sur trois piliers : le calcul, le stockage et le réseau. Pendant longtemps, chacun de ces piliers était géré par des équipements spécialisés distincts, ce qui entraînait une complexité administrative et des coûts élevés. L'hyperconvergence (HCI) est une architecture qui regroupe ces trois couches dans des serveurs standard, pilotés par une couche logicielle centralisée. Cette approche permet de simplifier l'administration, de réduire les coûts matériels et d'assurer la haute disponibilité de manière native. Dans un contexte de transformation numérique des entreprises industrielles, l'hyperconvergence est devenue une réponse concrète aux besoins d'infrastructures flexibles, évolutives et économiques.

Sonatrach, la compagnie nationale algérienne des hydrocarbures, dispose au sein de sa Direction Centrale Recherche & Développement d'une infrastructure hyperconvergée reposant sur des solutions propriétaires, Nutanix et VMware vSAN, pour ses environnements de production. Si ces solutions offrent des garanties solides en termes de fiabilité et de support, elles présentent des contraintes structurelles importantes : des licences coûteuses dont le prix augmente chaque année, une dépendance forte vis-à-vis des éditeurs qui limite toute liberté d'adaptation, et un manque de souveraineté technique. Pour les environnements de test, de développement et de pré-production, qui nécessitent des clusters de trois à quatre nœuds seulement, déployer ces mêmes solutions reste disproportionné par rapport aux besoins réels. La question qui se pose est donc la suivante : comment concevoir et déployer une infrastructure hyperconvergée open source, capable de répondre aux exigences de fiabilité et de performance de Sonatrach, tout en constituant une alternative crédible et souveraine aux solutions propriétaires ?

Plusieurs solutions open source existent pour répondre à ce besoin. OpenStack est la plateforme cloud la plus complète et la plus standardisée, mais sa complexité de déploiement et sa consommation élevée de ressources la rendent inadaptée aux petits clusters. Harvester, développé par SUSE sur Kubernetes et KubeVirt, propose une interface moderne mais reste orienté vers les charges de travail conteneurisées et consomme des ressources importantes avant même d'allouer des capacités aux machines virtuelles classiques. Proxmox VE, en revanche, est un hyperviseur open source léger, qui déploie un cluster de trois nœuds en quelques heures et intègre nativement Ceph pour le stockage distribué. Pour la couche réseau, Open vSwitch avec des tunnels VLAN permet de créer une segmentation réseau complète sans nécessiter de modifications sur le switch physique. Ces technologies constituent aujourd'hui une pile technique mature et bien documentée, mais aucune d'elles n'intègre nativement un mécanisme d'équilibrage dynamique de charge des ressources du Clusters.

Dans le cadre de ce projet, nous avons conçu et déployé une infrastructure Cloud privée hyperconvergée entièrement open source sur trois serveurs Dell PowerEdge R720 fournis par Sonatrach. Nous avons installé Proxmox VE en mode bare metal sur chaque nœud, puis formé un cluster haute disponibilité géré par Corosync, avec basculement automatique des

machines virtuelles en cas de panne. Nous avons déployé un cluster Ceph à douze OSDs avec un facteur de réplication de trois, garantissant la persistance des données même en cas de défaillance d'un nœud entier. Pour la couche réseau, nous avons conçu une architecture à neuf segments logiques, matérialisés par des tunnels VXLAN gérés par Open vSwitch, couvrant les flux d'infrastructure (Corosync, réplication Ceph, migration à chaud) et les zones applicatives (Dev, Prod, DMZ — “zone démilitarisée”). Nous avons également développé le **CLBS** (“Central Load Balancing Scheduler”), un service d'équilibrage dynamique de charge écrit en Go, qui surveille en continu la charge CPU et RAM des nœuds, calcule une bande de charge modérée et déclenche automatiquement des migrations à chaud pour redistribuer les machines virtuelles. Ce mécanisme est une adaptation de l'algorithme CSLB (“Central Scheduler Load Balancing”), étendu pour prendre en compte la RAM, avec des protections anti-oscillation et un circuit breaker. Pour compléter la plateforme, nous avons déployé une stack de supervision complète (InfluxDB, Prometheus, Grafana, Loki), des pipelines CI/CD via GitLab, une automatisation d'infrastructure par Ansible et Rundeck, une synchronisation temporelle par Chrony, et un pare-feu OPNsense assurant le routage et le filtrage inter-réseaux.

Ce mémoire est organisé en quatre chapitres :

- **Chapitre I** : Contexte du projet, infrastructure Sonatrach, problématique et objectifs.
- **Chapitre II** : État de l'art, comparaison des solutions et justification des choix technologiques.
- **Chapitre III** : Conception de l'architecture, dimensionnement physique, plan d'adressage et logique CLBS.
- **Chapitre IV** : Déploiement, configuration via PXE et Ansible, et validation de la plateforme.

Chapitre I

Présentation du Contexte

I.1 Introduction

Ce chapitre présente le contexte dans lequel s’inscrit notre projet de fin d’études. Nous y aborderons les fondements de l’hyperconvergence, les enjeux liés à la virtualisation et à la gestion unifiée des ressources informatiques, ainsi que les besoins spécifiques de l’entreprise Sonatrach. Ce contexte permettra de mieux saisir les choix technologiques et les objectifs fixés pour le déploiement d’une infrastructure Cloud privée hyperconvergée basée sur des solutions open source.

I.2 Présentation de Sonatrach

Sonatrach est une entreprise publique algérienne créée le 31 décembre 1963, peu après l’indépendance de l’Algérie. Elle a été fondée dans le but de gérer, développer et exploiter les ressources hydrocarbures du pays, qui représentent une richesse stratégique pour l’économie nationale. À ses débuts, Sonatrach se consacrait essentiellement au transport et à la commercialisation du pétrole brut extrait en Algérie.

Avec le temps, Sonatrach a évolué pour devenir la plus grande entreprise en Afrique dans le domaine des hydrocarbures. Elle intervient aujourd’hui dans l’ensemble de la chaîne de valeur pétrolière et gazière : exploration, production, transport, transformation et commercialisation. L’entreprise est également active dans la pétrochimie et les énergies renouvelables.

Dans les années 1970, suite à la nationalisation des hydrocarbures, Sonatrach a renforcé son rôle stratégique en prenant le contrôle total des ressources naturelles du pays. Cela a permis à l’Algérie de mieux contrôler ses revenus énergétiques et d’investir davantage dans son développement économique et social.

Sonatrach entretient de nombreuses collaborations internationales avec des compagnies pétrolières et gazières étrangères. Elle est impliquée dans de grands projets de partenariat à travers des joint-ventures et accords technologiques, notamment avec des entreprises européennes, américaines, asiatiques et africaines. Ces coopérations ont permis un transfert de savoir-faire et de technologie vers l’Algérie.

Sonatrach est également engagée dans la stratégie énergétique nationale, notamment dans la sécurité d’approvisionnement, la réduction progressive des émissions carbonées et le développement de nouveaux secteurs tels que le gaz naturel liquéfié (GNL), le gaz de schiste et les énergies renouvelables.

Aujourd’hui, Sonatrach opère dans plusieurs pays et représente l’un des principaux postes des exportations algériennes [1].

Le projet est réalisé au sein du Data Center de la Direction Centrale Recherche et Développement (DC RD) de Sonatrach. Cette direction coordonne les travaux de recherche appliquée en exploration et production et supervise l’optimisation des processus associés. Elle dispose de clusters de calcul et de solutions de stockage haute capacité dédiés à la simulation et au traitement sismique. C’est dans ce cadre que s’inscrit notre projet, avec pour objectif de renforcer les capacités de calcul et d’améliorer l’efficacité du traitement sismique grâce à une architecture HPC (“High Performance Computing”) performante.

I.3 Infrastructure Existante et Contexte Technologique

Cette section décrit l’environnement technologique dans lequel s’inscrit le projet. Elle présente d’abord l’infrastructure hyperconvergée actuellement déployée au sein de Sonatrach, puis les besoins spécifiques qui justifient la recherche d’une alternative open source pour les zones de test.

I.3.1 Infrastructure actuelle de Sonatrach

Sonatrach dispose actuellement d’une infrastructure hyperconvergée reposant sur deux solutions propriétaires complémentaires : Nutanix et VMware vSAN. Nutanix, déployé à l’échelle nationale sur plusieurs wilayas, équipe les infrastructures de grande envergure et les environnements de production critiques, tandis que VMware vSAN est privilégié pour les déploiements de plus petite taille. Ces deux solutions sont bien établies dans le domaine, avec une gestion centralisée, une haute disponibilité et une scalabilité éprouvées.

Cependant, ces solutions propriétaires présentent des contraintes structurelles communes qui deviennent de plus en plus préoccupantes dans une perspective d’évolution à long terme :

- **Coûts de licences croissants** : les frais de licence de ces plateformes propriétaires, Nutanix comme VMware, suivent une trajectoire haussière, représentant une charge financière significative et récurrente pour l’entreprise, indépendamment du niveau d’utilisation réel des plateformes.
- **Dépendance technologique forte** : le recours exclusif à des éditeurs propriétaires expose Sonatrach à un risque de “vendor lock-in”, limitant sa capacité à négocier les conditions contractuelles et à adopter librement de nouvelles technologies.
- **Manque de souveraineté technique** : les évolutions des plateformes, les mises à jour et les décisions d’architecture sont entièrement soumises à l’orientation technique des éditeurs, sans possibilité d’adaptation aux besoins spécifiques de l’entreprise.
- **Inadaptation aux petits déploiements** : pour les environnements de test, de développement et de pré-production, le déploiement d’une solution propriétaire de ce type sur de petits clusters de 3 à 4 nœuds reste disproportionné par rapport aux besoins réels, tant en complexité qu’en coût.

Ce projet répond donc à un besoin concret de Sonatrach : explorer et valider la viabilité d’une infrastructure hyperconvergée open source, d’abord en environnement de test, et potentiellement en production à terme.

I.3.2 Les besoins en infrastructure légère pour les zones de test

Au-delà des besoins immédiats en environnements de test et de pré-production, Sonatrach a besoin d’une infrastructure qui réponde à des exigences opérationnelles précises, tout en s’affranchissant des contraintes des solutions propriétaires. Ces besoins couvrent plusieurs

dimensions :

- **Fiabilité et haute disponibilité** : l'infrastructure doit garantir la continuité de service en cas de défaillance d'un ou plusieurs nœuds, avec basculement automatique des charges de travail sans intervention humaine.
- **Performance et scalabilité** : la solution doit pouvoir monter en charge progressivement, en ajoutant des nœuds au cluster sans interruption de service, pour accompagner la croissance des besoins de l'entreprise.
- **Gestion unifiée des ressources** : calcul, stockage et réseau doivent être administrés depuis une interface centralisée unique, réduisant la complexité opérationnelle et les besoins en compétences spécialisées multiples.
- **Équilibrage dynamique de charge** : la plateforme doit être capable de redistribuer automatiquement les charges de travail entre les nœuds en fonction de l'utilisation des ressources, optimisant ainsi l'efficacité globale du cluster.
- **Souveraineté et indépendance technologique** : en s'appuyant exclusivement sur des technologies open source, l'entreprise reprend le contrôle total de son infrastructure, avec la liberté d'auditer, de modifier et d'adapter la solution à ses besoins propres.
- **Réduction du coût total de possession (TCO)** : l'absence de licences propriétaires représente une économie substantielle sur le long terme, réallouable vers des investissements matériels ou humains.

I.4 Problématique

La problématique centrale de ce projet est la suivante :

Comment concevoir et déployer une infrastructure Cloud privée hyperconvergée, entièrement basée sur des technologies open source, capable de répondre aux exigences de fiabilité, de performance et de scalabilité d'une grande entreprise industrielle, tout en constituant une alternative crédible et souveraine aux solutions propriétaires de type Nutanix ?

Cette problématique soulève plusieurs questions techniques structurantes :

- Comment concevoir un système de stockage distribué, redondant et performant garantissant l'intégrité des données en cas de défaillance d'un nœud ?
- Comment assurer le basculement et l'équilibrage automatique des charges de travail dans un cluster à plusieurs nœuds, sans perte de données ni interruption prolongée de service ?
- Comment segmenter et isoler strictement les flux réseau au sein d'une infrastructure hyperconvergée, en séparant les trafics de gestion, de réplication, de migration et de production ?
- Dans quelle mesure une solution open source peut-elle répondre aux exigences d'une infrastructure de niveau production dans le contexte d'une grande entreprise ?

I.5 Objectifs du Projet

Les objectifs du projet se déclinent en un objectif général, qui fixe la finalité globale de la démarche, et en plusieurs objectifs spécifiques précisant les réalisations techniques attendues.

I.5.1 Objectif général

Concevoir et déployer une infrastructure Cloud privée hyperconvergée, entièrement open source, sur trois serveurs physiques Dell PowerEdge R720 mis à disposition par Sonatrach, offrant des garanties de fiabilité, de performance et d'évolutivité comparables aux solutions propriétaires de type Nutanix, et constituant une alternative stratégique à moyen et long terme.

I.5.2 Objectifs spécifiques

- **Déployer un cluster de trois nœuds physiques à haute disponibilité** sur les serveurs Dell PowerEdge R720 fournis par Sonatrach, avec gestion centralisée, administration simplifiée et support de la migration à chaud.
- **Configurer Proxmox VE sur chaque serveur physique** en exploitant pleinement les ressources matérielles disponibles (256 Go de RAM, 5 disques SAS ("Serial Attached SCSI") par nœud).
- **Concevoir et mettre en place un système de stockage distribué Ceph** en s'appuyant sur les disques SAS 600 Go de chaque nœud pour le stockage distribué, et les disques 300 Go pour le système d'exploitation.
- **Implémenter une segmentation réseau avancée** via Open vSwitch, en configurant une architecture multi-VLAN séparant les flux de gestion, de réplication, de migration et de production sur l'infrastructure physique réelle.
- **Développer un mécanisme d'équilibrage dynamique de la charge**, capable de redistribuer automatiquement les machines virtuelles selon les métriques de performance en temps réel.
- **Valider la solution** à travers un scénario de haute disponibilité démontrant la tolérance aux pannes sans perte de données ni interruption de service.
- **Fournir une base d'infrastructure évolutive**, accompagnée d'une documentation technique et d'une conception architecturale réutilisable à plus grande échelle au sein de Sonatrach.

I.6 Périmètre et Limitations du Projet

Ce projet vise le déploiement d'un cluster hyperconvergé open source destiné aux environnements de test, de développement et de pré-production. L'infrastructure est déployée sur un cluster physique de trois serveurs Dell PowerEdge R720 mis à disposition par Sonatrach, offrant les ressources matérielles nécessaires à l'exécution de l'hyperviseur Proxmox VE directement sur le matériel physique, du stockage distribué Ceph et des machines virtuelles de

démonstration.

Le passage à une architecture physique distribuée lève la contrainte majeure du point de défaillance unique (SPOF) matériel. Néanmoins, le périmètre de ce projet se limite volontairement à un environnement de laboratoire structuré, ce qui implique les contraintes suivantes :

- **Capacité de stockage** : La taille du pool de stockage distribué est techniquement contrainte par les disques physiques alloués au stockage (3 disques de 600 Go par serveur). Avec une réplication de facteur 3, le cluster offre environ 1,8 To d'espace utilisable. Cela rend cette infrastructure parfaitement adaptée aux charges de développement et de validation, mais insuffisante pour des charges de données à très grande échelle.
- **Limite de configuration du commutateur** : la segmentation réseau avancée repose sur une configuration précise des commutateurs physiques (affectation des ports aux VLANs, mode trunk). Cette configuration est du ressort exclusif de l'équipe réseau de Sonatrach et ne peut être modifiée dans le cadre de ce projet.
- **Absence de multilocation** : Proxmox VE, dans la configuration mise en œuvre, est orienté Cloud privé. La gestion multilocataire stricte (isolation forte entre différents clients ou départements totalement hermétiques), propre aux environnements Cloud publics, n'entre pas dans le périmètre de ce projet.
- **Maintenance à long terme** : La gestion opérationnelle continue de l'infrastructure (cycle de mises à jour, configuration d'un serveur de sauvegarde externe sécurisé, gestion des pannes matérielles physiques) n'est pas couverte par la phase de déploiement de ce projet et constitue une perspective d'évolution naturelle pour l'équipe d'exploitation.

I.7 Conclusion

Dans ce chapitre, nous avons présenté le contexte général du projet, les besoins identifiés au sein de Sonatrach, et la problématique qui motive notre démarche. Nous avons défini les objectifs, le périmètre et les limites du projet. Ce cadre nous permettra d'aborder, dans le chapitre suivant, l'état de l'art des solutions disponibles et les fondements théoriques de l'hyperconvergence, de la virtualisation et des technologies retenues.

Chapitre II

État de l'Art et Concepts Théoriques

II.1 Introduction

Dans un contexte de transformation numérique accélérée, les organisations scientifiques et industrielles cherchent des solutions d'infrastructure à la fois performantes, flexibles et économiques. Ce chapitre présente les concepts théoriques fondamentaux sur lesquels repose notre projet : l'hyperconvergence, la virtualisation, le stockage distribué et la virtualisation du réseau. Il dresse également un état de l'art comparatif des solutions disponibles, afin de justifier les choix technologiques retenus.

II.2 L'Hyperconvergence (HCI)

Cette section définit le concept d'hyperconvergence, en présente les composantes fondamentales et les principaux avantages, puis le confronte à l'architecture traditionnelle à trois niveaux.

II.2.1 Définition

L'hyperconvergence (HCI, Hyper-Converged Infrastructure) est une architecture informatique qui regroupe, dans un même système, les ressources de calcul, de stockage et de réseau. Ces ressources sont pilotées par une couche logicielle centralisée, supprimant ainsi la nécessité de gérer des équipements spécialisés distincts pour chaque fonction [2].

L'infrastructure hyperconvergée repose sur trois composantes fondamentales :

- **Le calcul** : des processeurs virtuels (vCPU) alloués dynamiquement aux machines virtuelles.
- **Le stockage** : un système de stockage distribué logiciel, agrégé à partir des disques locaux de chaque nœud.
- **Le réseau** : un switch virtuel logiciel permettant la segmentation et l'isolation du trafic réseau.

II.2.2 Avantages de l'hyperconvergence

L'approche HCI présente plusieurs avantages par rapport aux architectures traditionnelles à trois niveaux (compute, storage, network séparés) :

- **Simplicité d'administration** : une interface unique pour gérer l'ensemble des ressources.
- **Réduction des coûts matériels** : pas besoin de baies de stockage ou de switches réseau dédiés.
- **Évolutivité horizontale** : l'ajout d'un nœud augmente simultanément la capacité de calcul, de stockage et de réseau.
- **Haute disponibilité native** : la réplication des données entre les nœuds assure la continuité de service en cas de panne.

II.2.3 HCI face à l'Infrastructure Traditionnelle

Le tableau II.1 compare l'architecture hyperconvergée à l'architecture traditionnelle à trois niveaux, selon cinq critères : le mode de gestion, le coût initial, l'évolutivité, la complexité et la haute disponibilité.

TABLEAU II.1 – Comparaison entre l'infrastructure traditionnelle et l'infrastructure HCI

Critère	Infrastructure traditionnelle	Infrastructure HCI
Gestion	Silos séparés (calcul, stockage, réseau)	Interface unifiée
Coût initial	Élevé (équipements spécialisés)	Réduit (matériel standard)
Évolutivité	Verticale (mise à niveau du matériel)	Horizontale (ajout de nœuds)
Complexité	Haute	Faible
Haute disponibilité	Optionnelle et complexe	Native

II.3 Comparaison : Solutions Propriétaires et Open Source

Cette section met en regard les solutions propriétaires et les solutions open source, en soulignant les avantages et les limites de chacune dans le contexte du projet.

II.3.1 Solutions propriétaires

Les solutions propriétaires telles que **VMware vSAN** ou **Nutanix** offrent un support professionnel, des fonctionnalités avancées et une interface utilisateur soignée. Cependant, elles présentent plusieurs limites importantes dans notre contexte :

- **Coûts élevés** : les licences et la maintenance annuelle représentent un investissement significatif, difficilement justifiable pour des environnements de test.
- **Dépendance fournisseur (Vendor Lock-in)** : la migration vers une autre solution est souvent complexe et coûteuse.
- **Faible flexibilité** : les environnements propriétaires offrent peu de liberté pour personnaliser ou adapter l'infrastructure.

II.3.2 Solutions open source

Les solutions open source présentent plusieurs avantages stratégiques :

- **Indépendance technologique** : aucune dépendance vis-à-vis d'un éditeur.
- **Absence de coûts de licence** : seuls les coûts d'infrastructure matérielle et de main-d'œuvre sont à prévoir.

- **Transparence et auditabilité** : le code source est accessible et auditable.
- **Communauté active** : documentation abondante et mises à jour régulières.

Dans un contexte de zones de test et de petits déploiements, l'approche open source est clairement la plus adaptée.

II.4 Analyse des Solutions Open Source HCI

Le critère principal de sélection est la légèreté et la capacité à fonctionner de manière stable sur un cluster de 3 nœuds.

II.4.1 OpenStack

OpenStack est une plateforme cloud open source très complète et standardisée, utilisée par de nombreuses grandes entreprises et fournisseurs cloud. Elle propose une large gamme de services : calcul (Nova), stockage objet (Swift), stockage en bloc (Cinder), réseau (Neutron), identité (Keystone), etc.

Cependant, OpenStack présente des inconvénients majeurs pour notre cas d'usage :

- **Complexité de déploiement** : l'installation et la configuration d'OpenStack impliquent l'orchestration de nombreux services interdépendants (Nova, Neutron, Cinder, Keystone, etc.), ce qui rend le déploiement particulièrement long et difficile à maintenir.
- **Consommation de ressources élevée** : les services de base d'OpenStack consomment à eux seuls plusieurs Go de RAM, avant même de démarrer des VMs de production.
- **Inadapté aux petits clusters** : OpenStack est conçu pour des déploiements de grande envergure. Sur 3 nœuds, ses performances et sa stabilité sont compromises.

II.4.2 Harvester HCI

Harvester est une solution HCI open source développée par SUSE, basée sur Kubernetes et KubeVirt. Elle propose une interface moderne et une intégration native avec Rancher pour la gestion de clusters Kubernetes [4].

Ses limites dans notre contexte :

- **Consommation en RAM et CPU élevée** : Harvester nécessite des ressources importantes pour faire fonctionner la couche Kubernetes sous-jacente, avant même d'allouer des ressources aux VMs.
- **Maturité relative pour les VMs classiques** : Harvester est davantage orienté vers les charges de travail conteneurisées. La gestion de VMs Linux/Windows classiques est moins optimisée que sur une solution dédiée [3].

II.4.3 Proxmox VE

Proxmox Virtual Environment (VE) est une plateforme de virtualisation open source basée sur KVM (pour les VMs) et LXC (pour les conteneurs). Elle propose une interface web centralisée, un support natif de Ceph pour le stockage distribué, et une gestion intégrée du clustering et de la haute disponibilité [5].

Ses atouts pour notre cas d'usage sont nombreux :

- **Légèreté** : moins de 1 Go de RAM consommé à vide sur le nœud, laissant l'essentiel des ressources aux VMs.
- **Déploiement rapide** : un cluster de 3 nœuds est opérationnel en quelques heures.
- **Interface web intuitive** : la console Proxmox (accessible sur le port 8006) centralise la gestion des VMs, conteneurs, stockage, réseau et haute disponibilité.
- **Intégration native de Ceph** : Proxmox intègre nativement les outils de gestion Ceph, simplifiant considérablement le déploiement du stockage distribué.
- **Communauté active** : documentation officielle complète et forums très actifs.

II.4.4 Tableau comparatif

Le tableau II.2 synthétise la comparaison des trois solutions open source étudiées (OpenStack, Harvester et Proxmox VE) au regard des critères déterminants pour le contexte du projet : légèreté, facilité de déploiement, adaptation à un cluster de trois nœuds, intégration du stockage distribué, gestion des machines virtuelles classiques, maturité et coût de licence.

TABLEAU II.2 – Comparaison des solutions HCI open source : OpenStack, Harvester et Proxmox VE

Critère	OpenStack	Harvester	Proxmox VE
Légèreté	Faible	Moyenne	Élevée
Facilité de déploiement	Difficile	Moyenne	Facile
Adapté au déploiement à 3 nœuds	Non	Partiel	Oui
Intégration stockage distribué	Via Cinder/Ceph	Via Longhorn	Ceph natif
Gestion des VMs classiques	Bonne	Moyenne	Excellente
Maturité	Très mature	En cours	Mature
Coût de licence	Gratuit	Gratuit	Gratuit (Enterprise payant)

Proxmox VE est la solution la mieux adaptée au contexte de ce projet : légère, rapide à déployer sur un cluster de 3 nœuds, et capable de s'étendre à de plus grands clusters sans changement d'architecture. Elle sera retenue comme hyperviseur principal. Le choix de la solution de stockage distribué et de la couche réseau est abordé dans les sections suivantes.

II.5 Présentation des Technologies Retenues

Cette section présente en détail les trois technologies open source retenues pour l'infrastructure : l'hyperviseur Proxmox VE, le système de stockage distribué Ceph et le commutateur logiciel Open vSwitch.

II.5.1 Proxmox VE : L'hyperviseur

Présentation générale

Proxmox Virtual Environment (VE) est une plateforme de virtualisation open source basée sur un noyau Linux Debian modifié. Elle supporte deux types de virtualisation : **KVM** (“Kernel-based Virtual Machine”) pour les machines virtuelles complètes, compatibles avec tout système d'exploitation (Linux, Windows, BSD), et **LXC** (“Linux Containers”) pour les conteneurs légers partageant le noyau de l'hôte. L'ensemble est administré via une interface web centralisée accessible sur le port 8006, qui unifie la gestion des VMs, conteneurs, stockage, réseau et haute disponibilité depuis un point d'entrée unique.

La pile de virtualisation : KVM, QEMU et libvirt

La virtualisation des machines dans Proxmox repose sur une pile logicielle à trois niveaux qui collaborent étroitement.

KVM : Kernel-based Virtual Machine

KVM est un module du noyau Linux (chargé via `kvm.ko` et `kvm_intel.ko` ou `kvm_amd.ko` selon le processeur) qui transforme le noyau Linux lui-même en hyperviseur de type 1 [21]. KVM expose au système des périphériques virtuels (`/dev/kvm`) et exploite les extensions matérielles de virtualisation du processeur **Intel VT-x** ou **AMD-V** pour exécuter le code des VMs directement sur le CPU physique sans émulation. Cela signifie que les instructions non privilégiées de la VM s'exécutent à la vitesse native du processeur, tandis que les instructions privilégiées (accès mémoire, I/O) sont interceptées et gérées par le noyau hôte. Le résultat est une performance CPU très proche du bare metal, typiquement entre 95% et 99% des performances natives mesurées en benchmarks.

QEMU : Quick EMUlator

QEMU est l'émulateur de machine qui accompagne KVM [22]. Il opère en espace utilisateur et assure plusieurs fonctions essentielles :

- Il **émule les périphériques** de la VM : carte réseau virtuelle (VirtIO, e1000, RTL8139), contrôleur de disque (VirtIO SCSI, IDE, SATA), carte graphique, USB (“Universal Serial Bus”), BIOS (“Basic Input/Output System”) ou UEFI (“Unified Extensible Firmware Interface”) (via OVMF (“Open Virtual Machine Firmware”). Chaque VM en cours d'exécution correspond à un processus `qemu-system-x86_64` sur l'hôte.
- Il gère le **CPU virtuel (vCPU)** : le nombre de cœurs, le modèle de CPU exposé à

la VM (host-passthrough, kvm64, Haswell, Broadwell, etc.), et les flags CPU visibles par le système invité.

- Il assure la **gestion mémoire** : allocation de RAM, support du “Memory Ballooning” (ajustement dynamique de la RAM d’une VM en cours d’exécution), et des “Huge Pages” pour réduire la pression sur le TLB.
- Il fournit les **interfaces de migration** utilisées pour la live migration : QEMU implémente le protocole de transfert de l’état complet d’une VM (mémoire, état CPU, état des périphériques) vers un autre hôte sans interruption de service.

Lorsque KVM est disponible, QEMU délègue l’exécution du code de la VM à KVM et n’émule que les périphériques. C’est cette combinaison KVM+QEMU qui constitue l’hyperviseur réel.

libvirt : couche d’abstraction de la virtualisation Linux

libvirt est une bibliothèque open source et une API unifiée de gestion de la virtualisation sous Linux [23]. Elle offre une interface commune au-dessus de plusieurs hyperviseurs (KVM, QEMU, LXC, Xen) et est utilisée par de nombreux outils de la communauté : `virt-manager`, OpenStack Nova, ou encore oVirt. libvirt simplifie la création, la gestion du cycle de vie et la supervision des VMs en s’affranchissant des spécificités de chaque hyperviseur.

Proxmox VE fait un choix architectural différent : il n’utilise pas libvirt mais sa propre couche de gestion écrite en Perl (`PVE::QemuServer`), qui pilote QEMU directement. Cela lui confère un contrôle plus fin sur les paramètres QEMU et une intégration plus étroite avec l’API REST (“Representational State Transfer”) et les mécanismes de cluster (Corosync, HA, Ceph).

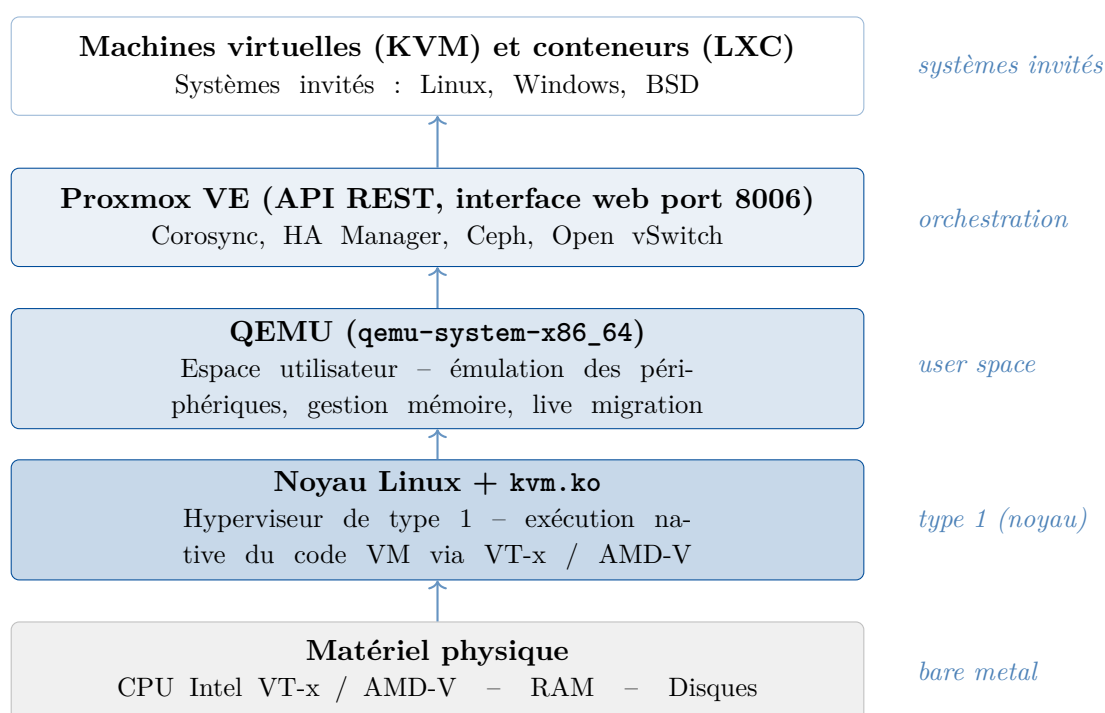


FIGURE II.1 – Pile de virtualisation KVM + QEMU dans Proxmox VE

La figure II.1 illustre les cinq niveaux de la pile de virtualisation Proxmox. KVM, intégré au noyau Linux, exploite les extensions matérielles VT-x et AMD-V pour exécuter le code des VMs directement sur le CPU physique. QEMU opère en espace utilisateur et émule les périphériques. Proxmox VE orchestre l'ensemble via son API REST et unifie la gestion du cluster, du stockage Ceph et du réseau Open vSwitch.

Les conteneurs Linux : LXC

LXC (“Linux Containers”) est une technologie de virtualisation légère reposant sur deux mécanismes natifs du noyau Linux : les **namespaces** et les **cgroups**. Contrairement aux machines virtuelles KVM qui émulent un matériel complet avec un noyau invité indépendant, les conteneurs LXC partagent le noyau de l'hôte. Cette approche réduit significativement la consommation de ressources : un conteneur démarre en quelques secondes et son surcoût en mémoire est de l'ordre de quelques dizaines de mégaoctets.

Les **namespaces** assurent l'isolation de chaque conteneur en lui attribuant des espaces de noms séparés pour les processus (PID), les interfaces réseau, les points de montage et les utilisateurs. Les **cgroups** (“control groups”) permettent de limiter et de comptabiliser la consommation de ressources (CPU, mémoire, I/O disque) de chaque conteneur, garantissant qu'aucun conteneur ne peut monopoliser les ressources de l'hôte.

Proxmox VE intègre LXC nativement et propose des modèles de distributions populaires (Debian, Ubuntu, Alpine). LXC est particulièrement adapté pour les services qui ne nécessitent pas de noyau spécifique : agents de supervision, serveurs NTP, services PXE, systèmes d'intégration continue et autres services Linux standards. Cette approche minimise la consommation de ressources par rapport à des machines virtuelles complètes pour ces cas d'usage [10].

Le tableau II.3 récapitule les principales différences entre une machine virtuelle KVM et un conteneur LXC, en termes d'isolation, de temps de démarrage, de surcoût mémoire, de systèmes supportés et de cas d'usage [10].

TABEAU II.3 – Comparaison entre une machine virtuelle KVM et un conteneur LXC

Critère	VM KVM	Conteneur LXC
Isolation	Matériel + noyau dédié	Noyau partagé (namespaces)
Temps de démarrage	20 à 60 secondes	1 à 5 secondes
Surcoût RAM	~512 MB	~50 MB
Systèmes supportés	Linux, Windows, BSD	Linux uniquement
Cas d'usage	Applications nécessitant un noyau spécifique	Services Linux standards

Gestion du cluster : Corosync et le quorum

La gestion du cluster Proxmox repose sur **Corosync**, un framework de communication de cluster qui assure le consensus entre les nœuds via un mécanisme de quorum [13]. Corosync envoie des messages “heartbeat” à intervalles réguliers entre les nœuds, sur un réseau dédié, pour détecter les défaillances. Un cluster Proxmox nécessite un minimum de **3 nœuds** pour

garantir un quorum stable.

Le quorum définit le nombre minimum de nœuds qui doivent être en accord pour que le cluster soit considéré comme opérationnel. Avec 3 nœuds, la majorité est de 2 : si un nœud tombe, les 2 restants ont toujours quorum et le cluster continue de fonctionner. Si 2 nœuds tombent simultanément, le nœud isolé n'a plus quorum et se met en retrait pour éviter le “split-brain”, une situation où deux parties du cluster croient chacune être la seule à fonctionner et modifient les données de manière concurrente.

Haute disponibilité (HA Manager)

Proxmox intègre un gestionnaire de haute disponibilité natif composé de deux services [24] :

- **pve-ha-crm** (“Cluster Resource Manager”) : le cerveau centralisé du HA. Il surveille l'état de toutes les ressources protégées (VMs, conteneurs) à l'échelle du cluster et prend les décisions de basculement.
- **pve-ha-lrm** (“Local Resource Manager”) : l'agent local sur chaque nœud. Il exécute les ordres du CRM (démarrer, arrêter, migrer une VM) et remonte l'état des ressources locales.

Lorsqu'un nœud devient inaccessible (perte de heartbeat Corosync), le CRM déclenche le **fencing** (isolation forcée du nœud défaillant via le watchdog matériel ou logiciel), puis ordonne le redémarrage des VMs protégées sur les nœuds survivants. Le délai typique de basculement est de 60 à 120 secondes. Les VMs enregistrées dans le HA Manager avec leurs disques sur un stockage partagé (Ceph RBD) peuvent être redémarrées sur n'importe quel nœud du cluster sans déplacement de données.

Live Migration

La migration à chaud (“live migration”) permet de déplacer une VM en cours d'exécution d'un nœud physique vers un autre sans interruption de service perceptible par l'utilisateur. Proxmox implémente ce mécanisme via QEMU en trois phases :

1. **Pré-copie mémoire** : QEMU copie itérativement les pages mémoire de la VM vers le nœud cible pendant que la VM continue de fonctionner. Les pages modifiées pendant la copie (“dirty pages”) sont recopiées lors de passes successives.
2. **Basculement** : lorsque le volume de dirty pages devient suffisamment faible (quelques dizaines de millisecondes de transfert), la VM est brièvement suspendue, les dernières pages sont transférées, et la VM reprend sur le nœud cible.
3. **Nettoyage** : les ressources de la VM sont libérées sur le nœud source.

Cette fonctionnalité est le mécanisme central d'un service d'équilibrage dynamique de charge, qui déclenche automatiquement des migrations à chaud pour redistribuer la charge CPU entre les nœuds. Le trafic de migration doit transiter sur un segment réseau dédié, isolé des autres flux pour garantir la bande passante nécessaire sans impacter les charges de travail de production.

Interface de gestion et API REST

Proxmox expose une **API REST complète** sur le port 8006 (HTTPS), utilisée aussi bien par l'interface web que par des outils externes. Chaque opération réalisable depuis l'interface graphique est accessible via l'API : création de VMs, gestion du stockage, déclenchement de migrations, consultation des métriques en temps réel. Cette API permet notamment d'interroger l'état des nœuds, de lister les VMs en cours d'exécution, et de déclencher des migrations programmatiquement via des appels HTTP authentifiés par ticket de session.

Récapitulatif de la pile technique Proxmox

Le tableau II.4 récapitule les principaux composants de la pile technique Proxmox VE présentés ci-dessus, en rappelant le rôle de chacun au sein de l'infrastructure.

TABLEAU II.4 – Récapitulatif des composants de la pile technique Proxmox VE

Composant	Rôle
KVM (<code>kvm.ko</code>)	Hyperviseur noyau, exécution native du code VM via VT-x / AMD-V
QEMU (<code>qemu-system-x86_64</code>)	Émulation des périphériques, gestion mémoire, migration à chaud
LXC	Conteneurs Linux légers partageant le noyau hôte
Corosync	“heartbeat” du cluster, consensus, détection de panne
pve-ha-crm / lrm	Gestionnaire de haute disponibilité, isolation (“fencing”), basculement automatique
API REST (port 8006)	Interface de pilotage programmatique et d'automatisation

II.5.2 Ceph : Le stockage distribué

Ceph est une solution libre de stockage distribuée (Software-Defined Storage) qui s'impose aujourd'hui comme une référence dans le domaine des infrastructures open source [16]. Conçu pour répondre aux besoins de fiabilité, d'extensibilité et de performance, Ceph constitue le choix idéal pour assurer la couche de stockage d'une infrastructure hyperconvergée telle que celle que nous proposons. Sa capacité à fonctionner sur du matériel standard (serveurs x86) et son architecture auto-réparatrice en font un composant central de notre projet.

Architecture de Ceph

L'architecture de Ceph repose sur un composant fondamental appelé RADOS (Reliable Autonomic Distributed Object Store). RADOS constitue le cœur du système, chargé de la gestion, de la réplication et de la répartition des données sur l'ensemble du cluster. Autour de ce cœur gravitent plusieurs types de démons qui interagissent pour former un cluster cohérent et performant :

- **Ceph Monitor (MON)** : Les Moniteurs sont les arbitres d'état du cluster. Ils maintiennent une carte maîtresse (“cluster map”) décrivant la topologie complète du cluster

et assurent le consensus entre les nœuds via un mécanisme de quorum. En cas de désaccord, la majorité des Monitors décide de l'état authoritative du cluster, évitant ainsi les situations de "split-brain".

- **Ceph Manager (MGR)** : Le Manager agit comme un chef d'orchestre. Il collecte toutes les métriques et informations d'état du cluster en temps réel (utilisation du stockage, performances, statistiques). Il expose ces données via une interface web de monitoring et les met à disposition des systèmes de gestion externes, facilitant ainsi l'administration et la supervision.
- **Ceph OSD (Object Storage Daemon)** : L'OSD est le démon qui gère les périphériques de stockage physiques ou logiques (disques, partitions, volumes logiques). Chaque OSD est responsable de la lecture et de l'écriture des données sur son disque. Les OSDs communiquent entre eux pour assurer la réplication, le rééquilibrage et la réparation des données en cas de panne. Dans une architecture hyperconvergée, les OSDs résident sur les mêmes nœuds que l'hyperviseur (Proxmox), chaque nœud contribuant ainsi au stockage global.
- **Ceph Metadata Server (MDS)** : Ce démon est spécifique au système de fichiers distribué CephFS. Il gère les métadonnées des fichiers (noms, permissions, arborescences). Pour un usage principalement en mode bloc (disques de machines virtuelles), ce composant n'est pas requis.

La gestion intelligente des données : CRUSH et Placement Groups

L'intelligence de Ceph réside dans sa manière unique de gérer l'emplacement des données. Contrairement aux systèmes de stockage traditionnels qui utilisent une table de correspondance centralisée, Ceph utilise un algorithme appelé CRUSH (Controlled Replication Under Scalable Hashing) [17]. Cet algorithme permet à un client de calculer mathématiquement l'emplacement exact d'un objet (ou de sa réplique) en se basant sur la carte CRUSH, sans avoir à interroger un serveur central. Cette approche élimine un point de défaillance unique et améliore considérablement la scalabilité.

La carte CRUSH est une structure hiérarchique qui définit l'organisation physique du cluster. L'administrateur peut y définir des "domaines de défaillance" (nœuds, racks, salles, data centers). Ceph utilise ensuite ces informations pour placer les répliques des données sur des nœuds distincts, garantissant ainsi que la perte d'un équipement n'entraîne pas de perte de données.

Pour optimiser la gestion des milliers d'objets, Ceph les regroupe dans des unités logiques appelées Placement Groups (PGs). Les PGs constituent une abstraction intermédiaire entre l'objet et l'OSD, permettant un équilibrage efficace des données et une meilleure performance du cluster.

Les modes de protection des données

Pour assurer la fiabilité et la disponibilité des données, Ceph propose deux mécanismes principaux, adaptés à différents besoins :

- **Réplication** : C'est le mode le plus simple et le plus performant. Chaque objet est dupliqué en N copies (généralement 3) et chaque copie est placée sur des OSDs différents,

dans des domaines de défaillance distincts. En cas de panne d'un OSD ou d'un nœud, les données restent disponibles sur les autres répliques. Ce mode offre une reconstruction rapide et est idéal pour les charges de travail exigeant de hautes performances en écriture, comme les disques de machines virtuelles. Dans le cadre d'un cluster à 3 nœuds, une réplication de facteur 3 garantit une haute disponibilité en cas de panne d'un nœud.

- **Erasur Coding (Codage d'effacement)** : Ce mode est plus efficace en termes d'espace de stockage. Les données sont découpées en K fragments de données, puis encodées avec M fragments de parité, pour un total de K+M fragments répartis sur différents OSDs. Ce mode permet de tolérer la perte de M fragments tout en reconstruisant les données initiales. Il est souvent utilisé pour les données froides ou l'archivage, car il nécessite moins d'espace que la réplication (ex : 1,5x l'espace brut contre 3x pour la réplication), mais il est plus coûteux en calcul lors des opérations de reconstruction.

Intégration de Ceph dans une architecture hyperconvergée

L'intégration de Ceph avec Proxmox VE, l'hyperviseur retenu, est native et profondément intégrée. Proxmox offre une interface unifiée qui permet de créer et de gérer un cluster Ceph directement depuis son interface web, simplifiant considérablement les opérations de déploiement et de maintenance.

Dans une architecture hyperconvergée, chaque nœud joue un double rôle :

- **Rôle de calcul** : Il exécute les machines virtuelles via KVM.
- **Rôle de stockage** : Il contribue au stockage global via ses disques alloués au pool Ceph.

Cette convergence des rôles est l'essence même de l'hyperconvergence. Les volumes de stockage des VMs sont fournis via Ceph RBD (RADOS Block Device), qui expose des périphériques de stockage en bloc sur le réseau. Ces volumes sont distribués et répliqués sur l'ensemble du cluster, assurant ainsi la persistance des données même en cas de panne d'un nœud.

Les avantages apportés par Ceph dans ce type d'infrastructure sont multiples :

1. **Évolutivité horizontale** : L'ajout d'un nouveau nœud apporte simultanément de la puissance de calcul et de la capacité de stockage. Ceph rééquilibre automatiquement les données, permettant une croissance linéaire et sans interruption.
2. **Haute disponibilité** : L'absence de point de défaillance unique (SPOF) et la réplication des données sur plusieurs nœuds garantissent que le système de stockage reste opérationnel même en cas de panne d'un ou plusieurs serveurs.
3. **Auto-réparation** : En cas de panne d'un disque ou d'un nœud, le cluster détecte l'incident et commence immédiatement à recréer les répliques manquantes sur les autres nœuds, assurant ainsi la pérennité des données sans intervention manuelle.
4. **Coût maîtrisé** : Ceph est open source et fonctionne sur du matériel standard (x86), ce qui le rend bien moins coûteux que les solutions propriétaires comme les SAN ("Storage Area Network") ou les appliances HCI propriétaires.

5. **Simplicité de gestion** : L'unification du stockage via Proxmox et les outils modernes comme `cephadm` (permettant un déploiement conteneurisé et simplifié) réduisent considérablement la complexité administrative.

II.5.3 Open vSwitch (OVS) : La virtualisation réseau

Présentation générale

Open vSwitch (OVS) est un commutateur logiciel multicouche open source, distribué sous licence Apache 2.0, conçu pour fournir une plateforme de commutation de qualité production dans les environnements de virtualisation [6]. Il fonctionne selon deux composants principaux : `ovs-vswitchd`, le démon qui implémente le switch avec son module noyau Linux pour un transfert haute performance, et `ovsdb-server`, une base de données légère de configuration persistante. Parmi ses fonctionnalités natives figurent le modèle VLAN 802.1Q, le bonding avec ou sans LACP (“Link Aggregation Control Protocol”), la supervision via NetFlow et sFlow, la configuration QoS, ainsi que le support natif des protocoles de tunneling GRE (“Generic Routing Encapsulation”), VXLAN et Geneve [7].

Dans un cluster HCI, OVS joue le rôle de commutateur central de chaque nœud, assurant la segmentation de tout le trafic en flux distincts et isolés depuis un bridge virtuel unique par serveur.

Pourquoi segmenter le réseau dans un cluster HCI ?

La nécessité d'une segmentation réseau stricte découle de la coexistence, sur la même infrastructure physique, de trafics aux profils radicalement différents et incompatibles :

- La **réplication Ceph** génère un trafic continu à très haute bande passante entre les disques de stockage distribués, pouvant saturer un réseau partagé.
- Les **heartbeats Corosync** sont extrêmement sensibles à la latence : tout délai ou perte de paquet peut déclencher un basculement non souhaité du cluster.
- Les **migrations à chaud** (live migration) produisent des pics de transfert ponctuels mais intenses, correspondant au déplacement complet de la mémoire vive d'une VM entre deux nœuds.
- Les **charges de travail applicatives** génèrent un trafic variable et imprévisible, qui ne doit pas interférer avec les flux d'infrastructure.

Sans isolation stricte, ces flux en compétition se dégradent mutuellement et menacent la stabilité globale du cluster. Cette problématique est identique à celle que résout Nutanix avec la séparation entre le trafic du contrôleur interne et le trafic des VMs utilisateur. Cette logique se traduit en segments réseau dédiés gérés par OVS.

Les recommandations officielles de la documentation Proxmox et les guides Ceph préconisent explicitement la séparation du réseau interne Ceph (réplication entre nœuds de stockage) du réseau public Ceph (accès des VMs au stockage), ainsi que l'isolation du trafic de synchronisation du cluster sur une interface dédiée.

Les neuf types de trafic à isoler

Ces contraintes conduisent à identifier neuf types de trafic aux profils distincts, chacun justifiant un segment réseau dédié. Le plan d'adressage précis et l'attribution des identifiants de tunnels sont définis dans le chapitre suivant ; ce qui importe ici est la nature du besoin qui motive chaque isolation :

- **Administration et gestion** : flux d'accès à la plateforme de virtualisation (interface web, SSH, API). Volume faible mais disponibilité permanente requise. Ce segment doit être strictement isolé des réseaux applicatifs pour prévenir tout accès non autorisé à la console d'administration depuis une VM compromise.
- **Synchronisation du cluster** : heartbeats et mécanismes de consensus entre les nœuds. Ce trafic est extrêmement sensible à la latence et aux pertes de paquets : tout délai supérieur aux seuils Corosync peut déclencher un basculement non souhaité et déstabiliser l'ensemble du cluster.
- **Réplication interne du stockage distribué** : communications directes entre les processus de stockage pour la réplication, le rééquilibrage et la reconstruction des données après panne. Ce flux est continu et à très haute bande passante ; le mélanger avec d'autres trafics dégrade directement les performances de stockage et risque de provoquer des délais de réplication.
- **Accès des machines virtuelles au stockage** : flux entre les VMs et le pool de stockage distribué (montage des volumes en mode bloc). Séparé du réseau de réplication interne pour éviter que les opérations de rééquilibrage ou de resynchronisation ne pénalisent les performances applicatives des VMs en production.
- **Migrations à chaud** : transferts de l'état mémoire complet d'une VM entre deux nœuds physiques. Ces flux sont ponctuels mais intenses, pouvant temporairement saturer la bande passante disponible ; leur isolation garantit que les migrations n'impactent ni le trafic applicatif ni la réplication du stockage.
- **Machines virtuelles de production** : trafic applicatif des services exposés aux utilisateurs finaux. Ce segment est le seul accessible depuis les points d'entrée contrôlés (répartiteur de charge) et doit être isolé des environnements de développement.
- **Machines virtuelles de test et de développement** : trafic des environnements de développement et de validation. Strictement isolé de la production pour éviter toute interférence entre les charges de travail de test et les services en production.
- **Zone exposée** : services destinés à être accessibles depuis l'extérieur du cluster. Cette zone est protégée par le pare-feu périmétrique et accessible uniquement via le répartiteur de charge comme point d'entrée unique et contrôlé.
- **Sauvegarde** : transferts de sauvegarde et d'instantanés de l'infrastructure. Ces opérations peuvent générer un trafic soutenu et doivent rester hors bande par rapport aux réseaux applicatifs afin de ne pas dégrader les performances en production.

Contrainte d'infrastructure rencontrée et adaptation : adoption de VXLAN

L'architecture initiale reposait sur une segmentation par **VLANs 802.1Q**, nécessitant que le switch physique soit configuré en mode "trunk" pour propager les tags VLAN entre

les trois nœuds. Cette approche est standard et parfaitement supportée par OVS.

Cependant, le déploiement sur l'infrastructure de Sonatrach a révélé une contrainte opérationnelle déterminante : les trois serveurs Dell R720 sont connectés à un switch administré dont la configuration est gérée exclusivement par l'équipe réseau de l'entreprise. Ce switch dispose déjà de VLANs définis (VLAN 10 pour le management, VLAN 20 pour un autre usage interne), et l'équipe projet ne dispose d'aucun droit d'administration pour modifier sa configuration, créer de nouveaux VLANs, ou reconfigurer les ports en mode trunk. Les serveurs sont simplement connectés en mode access sur le VLAN de gestion.

Face à cette contrainte, l'approche retenue est le remplacement des VLANs 802.1Q par des tunnels **VXLAN** ("Virtual eXtensible LAN", RFC 7348) [25].

VXLAN : principe de fonctionnement

VXLAN est un protocole d'encapsulation réseau qui encapsule des trames Ethernet de niveau 2 dans des paquets UDP/IP de niveau 4, permettant de créer des réseaux virtuels superposés ("overlay networks") par-dessus un réseau physique existant, sans nécessiter aucune modification de ce dernier.

Le principe est le suivant : lorsqu'une VM sur le nœud A souhaite communiquer avec une VM sur le nœud B à travers un réseau logique donné, OVS encapsule la trame Ethernet originale dans un paquet UDP à destination de l'adresse IP physique du nœud B. Ce paquet transite sur le réseau physique sous-jacent comme un paquet IP ordinaire. À la réception, OVS sur le nœud B désencapsule le paquet et restitue la trame originale à la VM de destination. Le switch physique ne voit que du trafic UDP ordinaire et n'a aucune connaissance des réseaux virtuels qui transitent au-dessus de lui.

Chaque réseau logique est identifié par un **VNI** ("VXLAN Network Identifier"), un entier sur 24 bits, l'équivalent fonctionnel du tag VLAN. Chacun de ces neuf types de trafic se traduit en un tunnel VXLAN distinct, identifié par un VNI unique, créé et géré nativement par OVS sur chacun des nœuds. L'attribution précise des VNIs et des plages d'adressage est détaillée dans le chapitre suivant.

Impact sur l'architecture et bilan

L'adoption de VXLAN ne remet pas en cause l'architecture réseau à neuf segments définie initialement. Les neuf segments logiques (management, Corosync, Ceph interne, Ceph public, migration à chaud, VMs de production, test/dev, DMZ, backup) sont intégralement conservés, chacun matérialisé par un identifiant VNI dédié au lieu d'un tag VLAN. OVS assure la création, la gestion et l'isolation de ces tunnels de manière transparente, sans aucune intervention sur l'infrastructure réseau physique de l'entreprise.

Cette approche constitue en réalité une évolution positive sur plusieurs points. Premièrement, elle renforce l'indépendance de l'infrastructure HCI vis-à-vis du réseau physique sous-jacent : le cluster peut fonctionner sur n'importe quel réseau IP sans prérequis de configuration. Deuxièmement, VXLAN est le protocole d'isolation réseau standard des infrastructures cloud modernes (AWS, Azure, GCP utilisent tous des superpositions de type VXLAN ou similaire) [25], ce qui renforce la représentativité technologique du projet. Troi-

sièment, la capacité de 16 millions d'identifiants VNI, contre 4 094 VLANs 802.1Q, offre une scalabilité largement suffisante pour toute extension future du cluster.

La seule contrainte introduite est un surcoût d'encapsulation de 50 octets par paquet UDP, négligeable au regard des débits disponibles sur des liaisons Gigabit ou 10 Gigabit Ethernet. Les performances de réplication Ceph, des migrations à chaud et des heartbeats Corosync ne sont pas affectées de manière mesurable par cette encapsulation supplémentaire.

II.6 L'Équilibrage Dynamique de Charge dans les Clusters Virtualisés

Cette section expose la problématique du déséquilibre de charge dans un cluster de virtualisation, passe en revue les mécanismes d'équilibrage proposés par les plateformes existantes, puis présente l'algorithme CSLB sur lequel s'appuie le mécanisme CLBS développé dans ce projet.

II.6.1 Problématique du Déséquilibre de Charge

Dans un cluster de virtualisation, les VMs sont placées sur les nœuds physiques lors de leur création. Sans mécanisme d'équilibrage actif, ce placement reste statique pendant toute la durée de vie des VMs. Au fil du temps, les charges de travail évoluent et la charge se répartit inégalement : certains nœuds peuvent être saturés à plus de 90 % de CPU tandis que d'autres restent utilisés à moins de 20 %. Ce déséquilibre dégrade les performances des VMs sur les nœuds chargés et gaspille les ressources disponibles sur les nœuds sous-chargés.

II.6.2 Mécanismes d'Équilibrage dans les Plateformes Existantes

VMware vSphere propose le **DRS** ("Distributed Resource Scheduler") [8], mécanisme d'équilibrage automatique qui analyse en continu les métriques de charge de chaque hôte et déclenche des migrations à chaud (vMotion) pour redistribuer les VMs. Le DRS calcule un score de déséquilibre pour le cluster et ne déclenche une migration que si le gain attendu dépasse un seuil configurable, afin d'éviter des migrations inutiles dans un cluster globalement sous-chargé.

D'autres plateformes proposent des mécanismes comparables. **Microsoft SCVMM** ("System Center Virtual Machine Manager") intègre la "Dynamic Optimization" [26], qui rééquilibre les VMs Hyper-V selon des intervalles configurables en minimisant les migrations inutiles. **oVirt** (plateforme open source de Red Hat) dispose d'un planificateur de cluster ("cluster scheduler") qui évalue la charge des hôtes et déclenche des migrations via libvirt [27]. **OpenStack Nova** implémente enfin un planificateur distribué capable de placer et de migrer les VMs selon des filtres et des pondérations paramétrables [28].

Proxmox VE ne dispose pas d'un tel mécanisme natif. Ce projet développe un service équivalent, nommé CLBS, s'appuyant sur l'API REST Proxmox et le stockage partagé Ceph.

II.6.3 L'Algorithme CSLB : Base Théorique du CLBS

Le CLBS est fondé sur l'algorithme CSLB proposé par Wilcox pour les environnements de virtualisation distribués [19]. L'algorithme repose sur un scheduler centralisé qui opère en

cinq phases successives [12] :

1. **Évaluation de la charge** : lecture des métriques CPU et RAM de chaque nœud sur une fenêtre temporelle glissante, pour s'affranchir des pics instantanés.
2. **Détermination de rentabilité** : calcul d'une bande modérée autour de la moyenne des charges du cluster. Les nœuds hors de cette bande sont candidats à une action d'équilibrage.
3. **Calcul du vecteur de transfert** : détermination de la quantité de ressources à migrer pour ramener le nœud surchargé dans la bande modérée.
4. **Sélection de la VM** : choix de la VM dont la charge correspond le mieux au vecteur de transfert calculé.
5. **Exécution de la migration** : déclenchement d'une live migration vers le nœud destination présentant la charge la plus faible.

La migration à chaud est le mécanisme d'exécution de toute décision d'équilibrage. Elle n'est applicable que si le stockage est partagé entre tous les nœuds (comme Ceph RBD), condition qui rend le disque de la VM accessible depuis le nœud de destination sans déplacement préalable des données [11].

II.7 Outils Complémentaires d'Infrastructure

Au-delà des trois technologies centrales que sont Proxmox VE, Ceph et Open vSwitch, l'infrastructure s'appuie sur un ensemble d'outils complémentaires. Cette section présente la répartition de charge applicative, la supervision, l'automatisation, le provisionnement réseau, le pare-feu périmétrique, la synchronisation temporelle et l'intégration continue.

II.7.1 HAProxy : Équilibrage de Charge Applicatif

HAProxy ("High Availability Proxy") est un proxy TCP/HTTP open source conçu pour la haute disponibilité et l'équilibrage de charge applicatif [15]. Il agit comme point d'entrée unique pour un ensemble de serveurs applicatifs, distribue les requêtes selon différents algorithmes (round-robin, moins de connexions, hachage par IP source) et surveille la disponibilité des backends via des contrôles de santé périodiques. En cas d'indisponibilité d'un backend, HAProxy redirige automatiquement le trafic vers les instances opérationnelles sans interruption pour l'utilisateur.

II.7.2 Stack de Supervision : InfluxDB, Prometheus et Grafana

La supervision d'une infrastructure hyperconvergente nécessite des outils capables de collecter, stocker et visualiser des métriques en temps réel sur l'ensemble des composants. L'objectif est de couvrir quatre dimensions clés : l'état de santé et la charge de chaque nœud du cluster, les journaux système et événements d'infrastructure, l'historique des migrations à chaud et leur impact sur la répartition de charge, et l'état du pool de stockage distribué (utilisation, latence, état de santé des disques).

Trois outils complémentaires constituent la pile de supervision d'une telle infrastructure.

InfluxDB [9] est une base de données orientée séries temporelles (“time series database”), optimisée pour le stockage de données horodatées à haute fréquence d’écriture. Elle reçoit les métriques de l’hyperviseur (CPU, RAM, réseau, stockage des nœuds et des VMs) via le mécanisme natif “Metric Server”, sans agent supplémentaire. Cet historique est utilisé à la fois pour la visualisation et pour alimenter les décisions de migration du service d’équilibrage.

Prometheus [29] est un système de collecte de métriques par interrogation HTTP, organisé autour d’un modèle de données par étiquettes. Il collecte les métriques détaillées du cluster de stockage distribué via le point d’accès exposé par son gestionnaire, ainsi que les métriques applicatives du service d’équilibrage.

Grafana [14] est un outil de visualisation connecté aux deux sources de données précédentes. Il permet de construire des tableaux de bord unifiés couvrant l’état des nœuds, l’état du stockage distribué et l’activité du service d’équilibrage, et d’y associer des règles d’alerte.

II.7.3 Ansible : Automatisation d’Infrastructure

Ansible [30] est un outil d’automatisation d’infrastructure (“Infrastructure as Code”) qui exécute des playbooks YAML via SSH, sans agent logiciel sur les machines cibles. Un playbook décrit l’état cible d’une machine (paquets installés, services configurés, fichiers déployés) et Ansible l’applique de manière **idempotente** : si la configuration est déjà conforme à l’état cible, aucune action n’est effectuée.

Chaque playbook est organisé en tâches séquentielles exécutées sur un inventaire de machines défini en YAML. La connexion SSH et les privilèges d’administration (**sudo**) sont les seuls prérequis sur les machines cibles. Ansible automatise typiquement l’intégration de nouveaux nœuds physiques dans le cluster, le provisionnement de machines virtuelles et de conteneurs, et la promotion d’environnements de développement vers la production.

II.7.4 PXE et TFTP : Provisionnement Réseau

PXE (“Preboot eXecution Environment”) est un mécanisme permettant à un serveur de démarrer depuis le réseau sans support de démarrage local. Il repose sur la coopération de trois protocoles :

- **DHCP** : attribution d’une adresse IP temporaire au serveur et indication de l’adresse du serveur de boot (options DHCP 66 et 67).
- **TFTP** (“Trivial File Transfer Protocol”, UDP/69) : téléchargement du bootloader et du noyau d’amorçage. TFTP est retenu pour cette phase initiale car c’est le seul protocole utilisable dans un firmware pre-OS.
- **HTTP** : téléchargement de l’image d’installation (plusieurs centaines de Mo). TFTP est inadapté aux fichiers volumineux en raison de l’absence de mécanisme de contrôle de flux.

Ce mécanisme est intégré au pipeline d’automatisation pour l’installation initiale de l’hyperviseur sur les serveurs physiques, sans aucune intervention manuelle sur les serveurs.

II.7.5 OPNsense : Pare-feu et Sécurité Péri-métrique

OPNsense [31] est une distribution de pare-feu et de routeur open source basée sur FreeBSD. Il assure le filtrage stateful des flux réseau, la gestion des règles de pare-feu par interface et par VLAN, ainsi que des fonctionnalités avancées telles que la détection d'intrusion (Suricata), le VPN ("Virtual Private Network") et le portail captif. Dans une infrastructure hyperconver-gée, OPNsense constitue le point de contrôle périmétrique entre les segments réseau internes et l'extérieur, protégeant notamment la zone exposée (DMZ).

II.7.6 NTP : Synchronisation Temporelle

La synchronisation temporelle est un prérequis critique pour les systèmes distribués. Dans un cluster, une dérive d'horloge entre les nœuds peut provoquer des incohérences dans les journaux d'événements, perturber les mécanismes de consensus (Corosync, Ceph) et invalider les timestamps des métriques de supervision. Le protocole NTP ("Network Time Protocol") [32] permet à chaque nœud de synchroniser son horloge avec un serveur de référence. Un serveur NTP interne au cluster garantit que tous les composants (hyperviseur, stockage distribué, service d'équilibrage, outils de supervision) partagent une base de temps cohérente.

II.7.7 GitLab CE : Intégration et Déploiement Continu

GitLab CE ("Community Edition") [20] est une plateforme DevOps open source intégrant un gestionnaire de dépôts Git, un système d'intégration continue (CI) et de déploiement continu (CD). Les pipelines CI/CD de GitLab permettent d'automatiser les tests, la construction et le déploiement des applications à chaque modification du code source. Dans le contexte d'une infrastructure hyperconver-gée, GitLab est utilisé pour versionner les configurations d'infrastructure (playbooks Ansible, fichiers de configuration), déclencher les déploiements automatisés et maintenir une traçabilité complète des modifications apportées à l'environnement.

II.8 Conclusion

Ce chapitre a posé les bases théoriques du projet. Le concept d'hyperconvergence a été défini et les principales solutions open source ont été comparées, aboutissant au choix de Proxmox VE comme hyperviseur, Ceph comme système de stockage distribué, et Open vSwitch comme commutateur logiciel. La pile de virtualisation (hyperviseur noyau, émulateur de machine, bibliothèque de gestion) a été décrite en détail, ainsi que les mécanismes de clustering, de haute disponibilité et de migration à chaud. La virtualisation réseau par tunnels superposés a été présentée comme réponse aux contraintes d'un réseau physique non administrable. Les fondements théoriques de l'équilibrage dynamique de charge ont été établis, en référence aux mécanismes existants dans d'autres plateformes. Enfin, les outils complémentaires d'infrastructure (répartition de charge applicative, supervision, automatisation, provisionnement réseau, pare-feu, synchronisation temporelle, intégration continue) et la justification du passage à un déploiement physique distribué ont été présentés. Le chapitre suivant présente la conception détaillée de l'architecture proposée.

Chapitre III

Conception de la Solution Proposée

III.1 Introduction

Ce chapitre présente la conception détaillée de l'infrastructure Cloud privée hyperconvergente proposée. Sur la base des technologies retenues dans le chapitre précédent (Proxmox VE, Ceph et Open vSwitch), nous décrivons ici les choix architecturaux guidant la construction de chaque couche de l'infrastructure. Nous présentons également la conception du mécanisme d'équilibrage dynamique de charge CLBS, ainsi que les outils d'automatisation retenus pour le déploiement et le provisionnement.

L'objectif de ce chapitre est de fournir une vision complète et cohérente de l'architecture avant toute phase d'implémentation, en justifiant chaque décision par des contraintes techniques, des recommandations officielles ou des exigences fonctionnelles identifiées précédemment.

III.2 Vue Globale de l'Architecture

L'architecture proposée repose sur un modèle à trois couches superposées et interdépendantes, chacune apportant une dimension essentielle à l'infrastructure hyperconvergente.

La première couche est la **couche physique**, constituée de trois serveurs Dell PowerEdge R720 mis à disposition par le Centre d'innovation de Sonatrach. Proxmox VE sera installé directement en mode bare metal sur chacun de ces serveurs, sans couche de virtualisation intermédiaire, ce qui garantira des performances maximales.

La deuxième couche est la **couche de virtualisation et de cluster**, composée des trois nœuds Proxmox (PVE-01, PVE-02, PVE-03) interconnectés via le réseau Sonatrach. Ces nœuds formeront un cluster haute disponibilité géré par Corosync. Chaque nœud exécutera simultanément les daemons Ceph pour le stockage distribué et une instance Open vSwitch avec des tunnels VxLAN pour la segmentation réseau. Cette convergence des rôles est l'expression directe du principe HCI.

La troisième couche est la **couche applicative et de services**, hébergeant la VM OPNsense pour le routage et le pare-feu, les VMs de démonstration Sona-Web, le mécanisme CLBS, la stack de monitoring, et les outils d'automatisation Ansible.

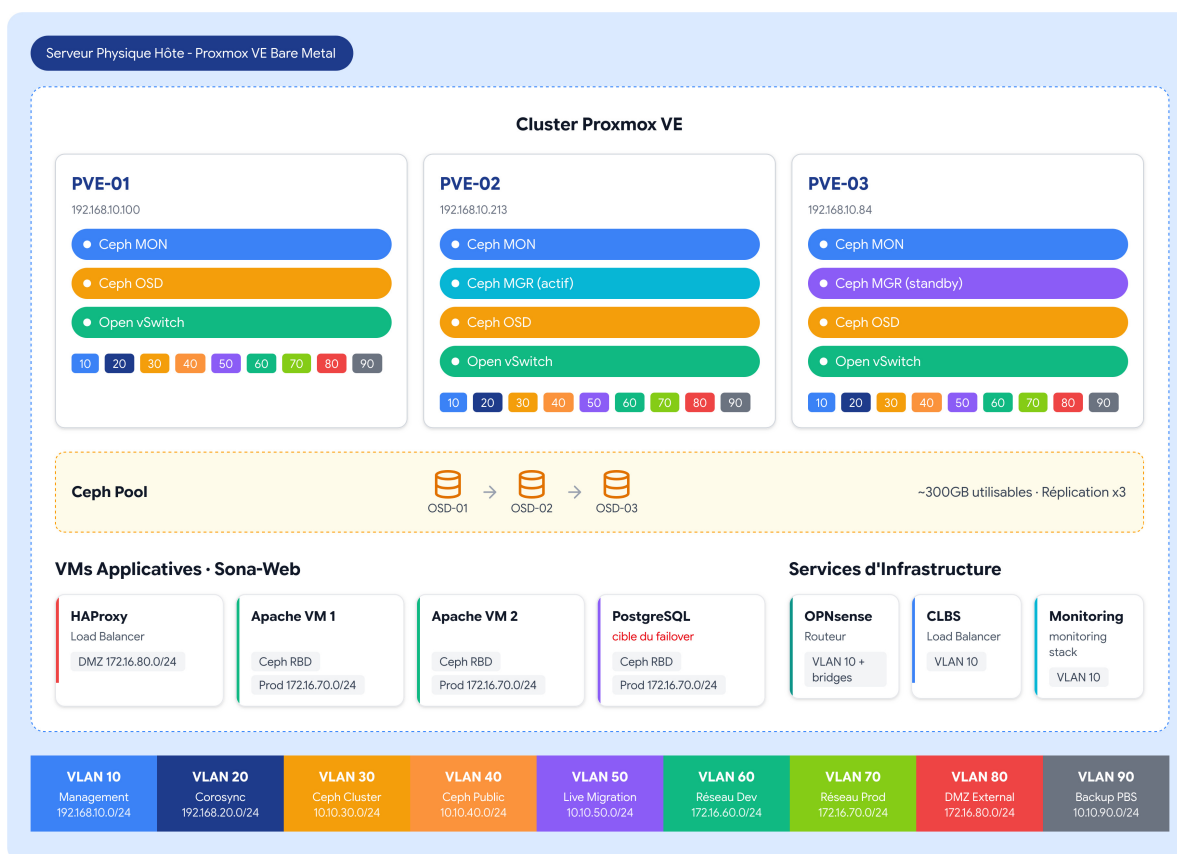


FIGURE III.1 – Architecture Globale de l'Infrastructure HCI

La figure III.1 représente les trois couches superposées de l'architecture HCI : la couche physique composée des serveurs Dell R720, la couche de virtualisation et de cluster portant Proxmox, Ceph et Open vSwitch, et la couche applicative hébergeant les services déployés. Elle met en évidence la convergence des rôles sur chaque noeud physique, principe fondateur de l'hyperconvergence.

III.3 Organisation de la Solution

La conception de l'infrastructure hyperconvergée a été structurée en trois parties complémentaires et interdépendantes : le **Calcul**, le **Stockage** et le **Réseau**. Cette partition reflète les trois piliers fondamentaux de tout système HCI, chacun apportant une dimension essentielle à l'ensemble. Les choix effectués dans chaque partie influencent directement les deux autres : le stockage distribué Ceph repose sur le réseau pour la réplication, le mécanisme CLBS dépend du stockage partagé pour les migrations à chaud, et la segmentation réseau conditionne la sécurité de l'ensemble. Le schéma ci-dessous illustre cette organisation ainsi que les interactions entre les trois parties.

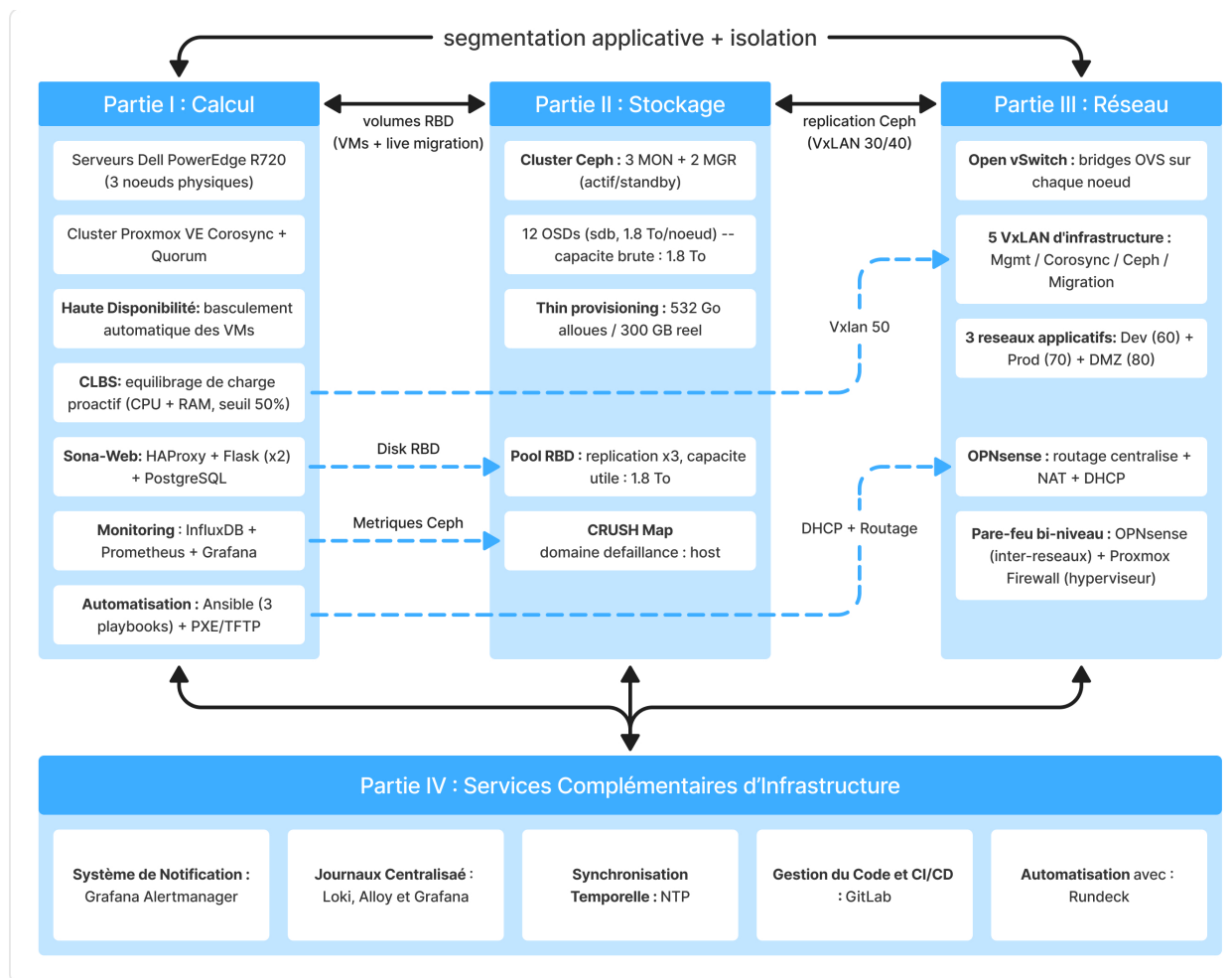


FIGURE III.2 – Organisation de la Solution en Trois Parties et leurs Interactions

La figure III.2 met en évidence les flux entre les parties : la partie Calcul s'appuie sur le Stockage pour les volumes RBD des VMs et pour les migrations à chaud réalisées par CLBS, le Stockage utilise le Réseau pour transporter les flux de réplication Ceph sur les VxLAN 30 et 40, et le Réseau fournit la segmentation applicative qui conditionne l'isolation et la sécurité de toutes les couches. Les relations détaillées au niveau composant (CLBS vers VxLAN 50, VMs Sona-Web vers pool RBD, Monitoring vers endpoint Ceph, Ansible vers OPNsense) illustrent le niveau d'interdépendance réel entre les trois parties.

III.4 Partie I : Calcul

Cette partie traite de la couche de calcul de l'infrastructure. Elle présente les serveurs physiques constituant les nœuds du cluster, l'organisation du cluster Proxmox et ses mécanismes de haute disponibilité via Corosync, le mécanisme d'équilibrage de charge CLBS développé spécifiquement pour ce projet, l'application de démonstration Sona-Web, la stack de monitoring déployée pour superviser l'ensemble de la plateforme, et enfin les outils d'automatisation Ansible et PXE permettant le déploiement et le provisionnement sans intervention manuelle.

III.4.1 Infrastructure Physique

L'infrastructure physique est constituée de trois serveurs Dell PowerEdge R720. Ces serveurs forment les nœuds du cluster Proxmox : PVE-01, PVE-02 et PVE-03. Proxmox VE sera installé directement en mode bare metal sur chacun d'eux, sans couche de virtualisation intermédiaire. Le Dell PowerEdge R720 est un serveur rack 2U de la gamme Entreprise de Dell, équipé de processeurs Intel Xeon E5-2600, adapté aux charges de virtualisation intensive et au stockage distribué.

Chaque serveur dispose de cinq disques SAS répartis selon leurs rôles respectifs, conformément aux recommandations Proxmox concernant la séparation des entrées/sorties système et de stockage :

- **Disque système (sda, 1 × 300 GB SAS) :** dédié au système d'exploitation Proxmox VE. Cette séparation garantit que les opérations de réplication Ceph ne perturbent pas les I/O système du nœud.
- **Disques OSD Ceph (1 × 300 GB SAS + 3 × 600 GB SAS) :** les quatre disques restants sont affectés comme OSDs Ceph. Chaque nœud contribue ainsi avec quatre OSDs au cluster de stockage distribué, soit douze OSDs au total dans le cluster.

Le tableau III.1 récapitule le dimensionnement matériel de chaque serveur Dell PowerEdge R720 ainsi que les ressources totales du cluster.

TABLEAU III.1 – Dimensionnement physique des serveurs Dell PowerEdge R720

Composant	Par nœud	Total cluster	Justification
Modèle serveur	Dell R720	3 serveurs	Fourni par Sonatrach
RAM	64 GB	192 GB	Proxmox + Ceph + VMs
Disque OS (sda)	1 × 300 GB SAS	3 × 300 GB	Isolation I/O hôte / Ceph
Disques OSD (sdb-sde)	1 × 300 GB + 3 × 600 GB SAS	12 OSDs (6,3 TB brut)	Stockage distribué Ceph
IP Management	voir tableau III.2		Réseau Sonatrach 192.168.10.0/24

III.4.2 Architecture du Cluster Proxmox

Le cluster Proxmox regroupera les trois nœuds physiques en une entité unique administrée de façon centralisée. Cette section en détaille l'organisation, le mécanisme de quorum assuré par Corosync, le fonctionnement de la haute disponibilité et les scénarios de défaillance pris en charge.

III.4.2.1 Organisation du Cluster

Le cluster Proxmox sera constitué de trois nœuds physiques identiques, interconnectés via le réseau Sonatrach. Le trafic Corosync sera transporté sur un réseau logique dédié (Vx-LAN ID 20) afin de garantir la stabilité du quorum indépendamment des autres flux. Cette

organisation à trois nœuds est le minimum requis pour garantir un quorum stable et éviter les situations de split-brain.

Le tableau III.2 indique les adresses IP de management attribuées à chacun des trois nœuds du cluster.

TABLEAU III.2 – Adresses IP de management des nœuds Proxmox

Nœud	Nom d'hôte	Adresse IP Management
Nœud 1	PVE-01	192.168.10.100
Nœud 2	PVE-02	192.168.10.213
Nœud 3	PVE-03	192.168.10.84

III.4.2.2 Mécanisme de Quorum et Corosync

Corosync est le framework de communication utilisé par Proxmox pour assurer le consensus entre les nœuds. Il envoie des messages de heartbeat à intervalles réguliers via le VxLAN 20. Le quorum dans un cluster de trois nœuds est atteint dès que deux nœuds sur trois sont opérationnels. La perte simultanée de deux nœuds entraîne le gel du cluster pour prévenir toute corruption des données.

III.4.2.3 Haute Disponibilité

Le mécanisme de haute disponibilité de Proxmox s'appuie sur Corosync pour la détection des pannes et sur le Resource Manager HA pour les décisions de basculement. Lorsqu'un nœud est déclaré défaillant, le HA Manager identifie les VMs concernées et déclenche leur redémarrage automatique sur un nœud survivant. Ce mécanisme est rendu possible par le stockage partagé Ceph : les disques des VMs étant accessibles depuis n'importe quel nœud, le redémarrage ne nécessite aucun transfert de données préalable. Le délai de basculement est typiquement compris entre 60 et 120 secondes.

Toutes les VMs de l'application Sona-Web seront configurées en mode HA, avec une priorité de basculement définie : HAProxy en premier, suivi des serveurs Flask, puis de la base PostgreSQL.

III.4.2.4 Matrice des Scénarios de Défaillance

Le tableau III.3 recense les principaux scénarios de défaillance envisagés, en précisant pour chacun le composant affecté, le comportement attendu de l'infrastructure et l'objectif de temps de rétablissement (RTO).

TABLEAU III.3 – Matrice des scénarios de défaillance et comportements attendus

Scénario	Composant affecté	Comportement	RTO
Panne d'un nœud physique	PVE-01, 02 ou 03	Corosync détecte la perte. HA redémarre les VMs sur les survivants. Ceph passe en degraded mais reste accessible.	60–120 s
Panne d'un OSD	Disque sdb	Ceph marque l'OSD down . Le pool passe en degraded (2/3 répliques). Backfill automatique après rétablissement.	Aucune coupure
Panne du MGR actif	PVE-02 (MGR)	Le MGR standby (PVE-03) prend le relais. Endpoint Prometheus disponible sans interruption.	< 30 s
Panne de deux nœuds	Deux des trois nœuds	Perte du quorum. Le nœud survivant se gèle. Intervention manuelle requise.	N/A
Panne VM OPNsense	VM routeur/pare-feu	Proxmox HA redémarre la VM sur un nœud survivant. Le trafic inter-réseaux est interrompu pendant la durée du redémarrage.	60–120 s
Split-brain	Réseau Vx-LAN 20	Le groupe minoritaire perd le quorum et se gèle. Le groupe majoritaire continue d'opérer.	Aucune coupure côté majoritaire
Panne CLBS	VM CLBS	Aucun impact applicatif. Le service redémarre automatiquement (Restart=always systemd).	Aucune coupure

III.4.3 Mécanisme d'Équilibrage Dynamique : CLBS

Cette section présente le mécanisme d'équilibrage dynamique de charge CLBS développé pour ce projet : le besoin auquel il répond, sa base algorithmique, les adaptations apportées à l'algorithme d'origine et sa logique de décision.

III.4.3.1 Contexte et Besoin

Proxmox VE intègre un mécanisme de haute disponibilité réactif : lorsqu'un nœud tombe en panne, les VMs sont redémarrées sur les nœuds survivants. Cependant, Proxmox ne dispose d'aucun mécanisme proactif d'équilibrage de charge, contrairement à VMware vSphere qui propose le DRS (Distributed Resource Scheduler). Sans équilibrage proactif, un nœud peut être saturé à 90% de CPU tandis qu'un autre reste à 20%, dégradant les performances et gaspillant les ressources disponibles.

III.4.3.2 Base Algorithmique : CSLB

Notre mécanisme, nommé **CLBS**, est une adaptation du CSLB [19]. L'algorithme original repose sur cinq phases : évaluation de charge, détermination de rentabilité, calcul du vecteur

de transfert, sélection de VM, et exécution de la migration [12]. La charge CPU de chaque nœud est évaluée par rapport à un seuil dynamique (moyenne des charges de tous les nœuds) :

$$\text{seuil} = \frac{\sum_{i=1}^n \alpha_i}{n}, \quad \text{diff} = \left| \text{seuil} - \frac{\alpha_{\min} + \alpha_{\max}}{2} \right|$$

La bande modérée est définie à $\pm \max(\text{diff}, 10\%)$ autour du seuil. Les nœuds hors de cette bande sont candidats à une action d'équilibrage.

III.4.3.3 Modifications Apportées au CSLB

Notre adaptation introduit plusieurs modifications importantes par rapport à l'algorithme original.

- **Implémentation en Go** : retenu pour sa faible empreinte mémoire, sa gestion native de la concurrence via les goroutines, et la simplicité du déploiement sous forme de binaire statique dans un service systemd [11].
- **Extension à la RAM** : le CSLB original ne considère que le CPU. Notre implémentation calcule une bande modérée indépendante pour la RAM. Un nœud est surchargé si l'une ou l'autre dépasse sa bande respective.
- **Seuil minimum de migration** : une migration n'est déclenchée que si la charge dépasse un seuil absolu de 50% (CPU ou RAM), évitant des déplacements inutiles dans un cluster globalement sous-chargé.
- **Sélection du nœud destination pondérée** : le choix du nœud de destination se fonde sur un score pondéré :

$$\text{score} = \Delta_{CPU} \times 0.7 + \Delta_{RAM} \times 0.3$$

Avant validation, CLBS simule l'état post-migration du nœud cible pour s'assurer que l'ajout de la VM ne le fait pas dépasser la bande haute.

- **Exclusion des VMs système** : OPNsense, la VM monitoring et la VM CLBS sont explicitement exclues du périmètre de migration.
- **Cadence et unicité** : CLBS s'exécute toutes les 60 secondes et ne peut déclencher qu'une seule migration par cycle.
- **Cooldown post-migration** : une période de repos de 5 minutes est appliquée après chaque migration, le temps que les métriques InfluxDB intègrent l'impact du changement.
- **Mécanismes anti-oscillation** : si une même VM est migrée deux fois en moins de 10 minutes, elle est gelée 30 minutes et l'administrateur est notifié. Si 6 migrations ou plus sont enregistrées en 30 minutes, le service se suspend automatiquement pendant une heure (circuit breaker).
- **Collecte des métriques** : CLBS combine l'API REST Proxmox pour les métriques courantes, et InfluxDB via le client Go officiel pour les moyennes glissantes sur 5 minutes, garantissant des décisions fondées sur des données lissées plutôt que sur des pics instantanés.

III.4.3.4 Algorithme de Décision (Pseudocode)

```
toutes les 60 secondes :  
  recuperer metriques CPU et RAM moyenees (5 min) depuis InfluxDB  
  calculer seuil_cpu, bande_moderee_cpu  
  calculer seuil_ram, bande_moderee_ram  
  
pour chaque noeud N :  
  si (cpu_N > bande_haute_cpu OU ram_N > bande_haute_ram)  
    ET (cpu_N > 50% OU ram_N > 50%) :  
  
    candidats_vm = VMs sur N non migrees recemment (cooldown 10  
      min)  
    vm_cible = VM dont la charge est la plus proche du  
      vecteur de transfert calcule (phase 3 CSLB)  
  
    pour chaque noeud destination D (bande legere) :  
      score_D = (cpu_N - cpu_D) * 0.7 + (ram_N - ram_D) * 0.3  
    noeud_dest = D avec score_D maximal  
  
    si noeud_dest trouve :  
      declencher live migration vm_cible -> noeud_dest  
      via API Proxmox (VxLAN 50)  
      enregistrer migration dans InfluxDB  
      appliquer cooldown anti-oscillation (10 min sur vm_cible)
```

Figure III.3 – Pseudocode de l'algorithme CLBS

La figure [III.3](#) présente le pseudocode de l'algorithme CLBS sous sa forme condensée. Chaque cycle commence par la récupération des métriques CPU et RAM lissées sur 5 minutes depuis InfluxDB, afin d'éviter les décisions basées sur des pics instantanés. Le double critère de déclenchement (dépassement de la bande modérée ET seuil absolu de 50%) garantit qu'aucune migration n'est effectuée dans un cluster globalement sous-chargé. La sélection de la VM cible repose sur le vecteur de transfert du CSLB, et le score pondéré 0.7/0.3 en faveur du CPU reflète le fait que la pression CPU est généralement plus contraignante que la RAM dans les charges de virtualisation.

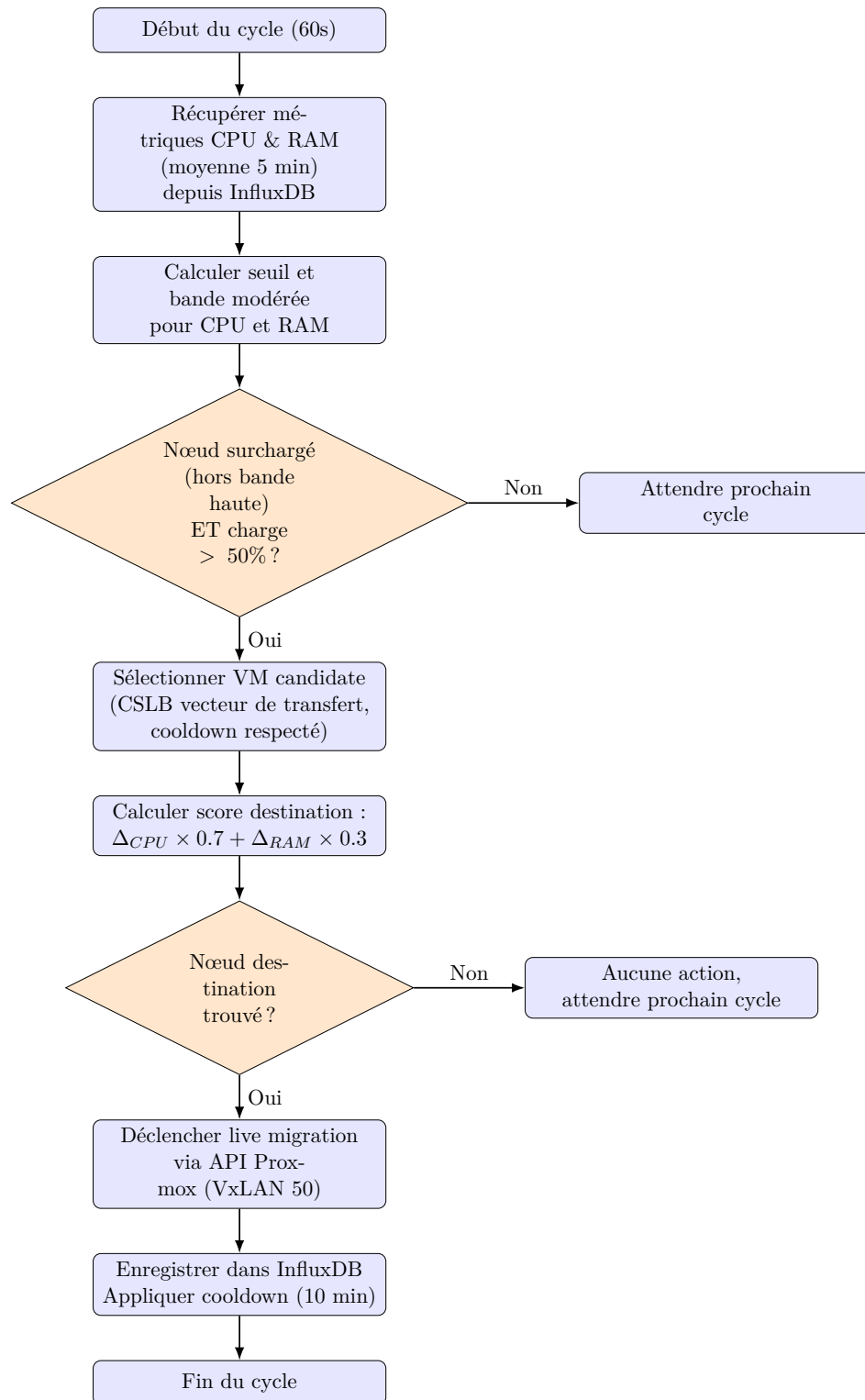


FIGURE III.4 – Logique de Décision du Mécanisme CLBS

La figure III.4 présente le cycle de décision complet du service CLBS. A chaque intervalle de 60 secondes, le service évalue la charge de chaque nœud par rapport aux bandes modérées calculées dynamiquement, sélectionne la VM la mieux adaptée au transfert selon le vecteur de charge, choisit le nœud destination en maximisant le score pondéré CPU/RAM, et déclenche la migration via l'API Proxmox si toutes les conditions sont réunies. Les mécanismes anti-oscillation et le circuit breaker sont également représentés.

III.4.4 Application de Démonstration : Sona-Web

L'application Sona-Web sert de cas d'usage pour valider les capacités de haute disponibilité et de migration à chaud de l'infrastructure. Cette section en décrit l'architecture applicative, puis le scénario de validation associé.

III.4.4.1 Architecture Applicative

Sona-Web est une application web déployée sur le cluster pour démontrer les capacités de haute disponibilité et de live migration. Elle sera constituée de quatre VMs Debian 12 interconnectées :

- **HAProxy** : point d'entrée unique de l'application. Il distribue les requêtes HTTP entre les deux serveurs applicatifs selon l'algorithme round-robin et surveille leur disponibilité toutes les 200 ms (`GET /health`), basculant vers l'instance disponible en moins de 200 ms (`fall 1`) en cas d'indisponibilité [15].
- **Flask app-1 et app-2** : deux instances identiques de l'application Python Flask. Chaque instance enregistre de manière asynchrone en base de données toutes les requêtes reçues.
- **PostgreSQL** : base de données centralisée dont le disque est stocké sur un volume Ceph RBD, garantissant un historique cohérent.

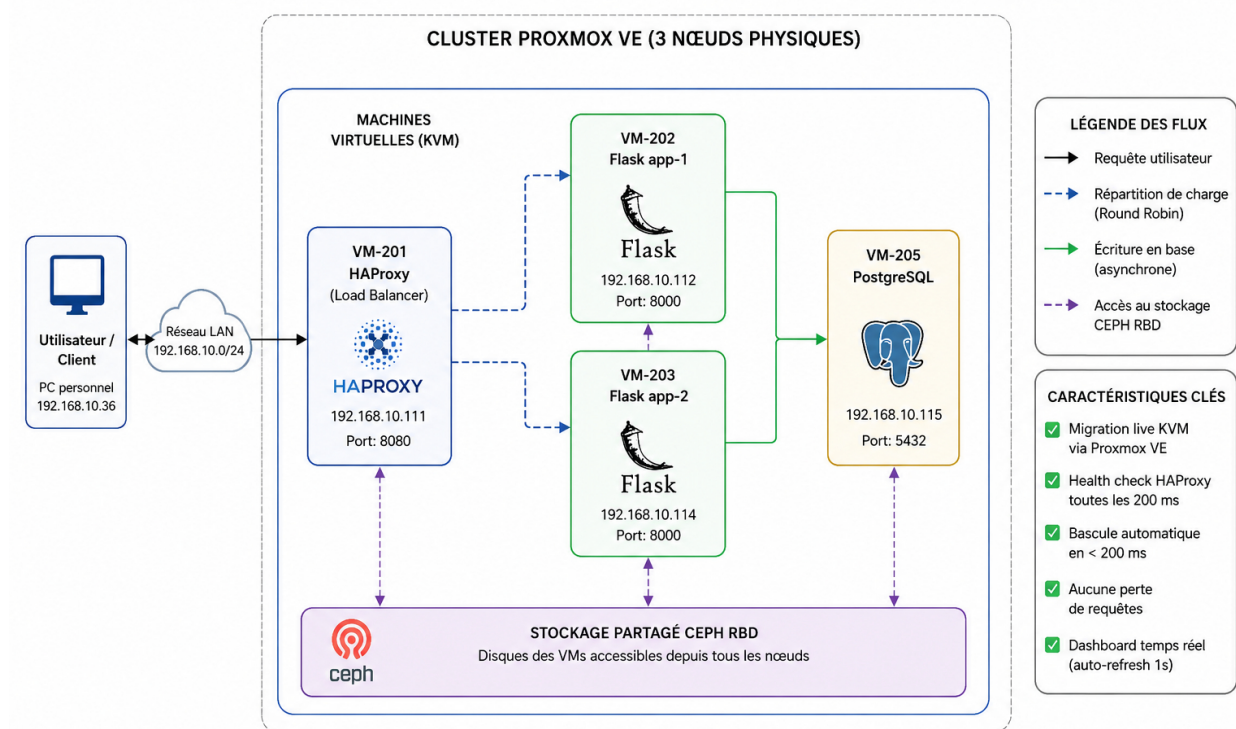


FIGURE III.5 – Architecture Applicative Sona-Web

La figure III.5 illustre l'architecture en quatre tiers de l'application Sona-Web. HAProxy constitue le point d'entrée unique et distribue le trafic entre les deux instances Flask selon un algorithme round-robin. Les deux serveurs applicatifs écrivent dans une base PostgreSQL

centralisée dont le disque est stocké sur le pool Ceph RBD partagé, garantissant la cohérence des données quelle que soit la position des VMs dans le cluster.

III.4.4.2 Scénario de Validation HA

Le scénario de validation se déroule en plusieurs temps : affichage de l’alternance round-robin entre app-1 et app-2, puis déclenchement d’une live migration de VM-202 (**app-1**) vers un autre nœud. Pendant la migration, Proxmox transfère l’état mémoire de la VM sans l’arrêter. Lorsque la synchronisation atteint ~99%, la VM est suspendue quelques dizaines de millisecondes le temps de transférer le delta final, puis reprend sur le nœud cible.

HAProxy détecte l’indisponibilité transitoire en 200 ms maximum et redirige l’intégralité du trafic vers app-2. Une fois la migration terminée, app-1 est remise en service dès le premier health check réussi (**rise 1**).

III.4.5 Architecture du Monitoring

La stack de monitoring sera déployée dans un conteneur LXC dédié à l’adresse 192.168.10.99. Elle comprend trois composants complémentaires couvrant des périmètres de métriques distincts.

InfluxDB recevra les métriques générales de Proxmox (CPU, RAM, réseau, stockage des nœuds et des VMs) via le mécanisme natif “Metric Server”, sans agent supplémentaire. Cet historique sera utilisé à la fois par Grafana pour la visualisation et par le service CLBS pour ses décisions de migration.

Prometheus collectera les métriques détaillées du cluster Ceph via l’endpoint exposé par le nœud MGR actif sur le port 9283. Les métriques applicatives du service CLBS seront également exposées sur le port 9101 de la VM dédiée.

Grafana sera connecté à InfluxDB et à Prometheus comme sources de données, permettant de construire des tableaux de bord unifiés couvrant l’ensemble de l’infrastructure : utilisation des ressources par nœud, état du cluster Ceph, historique des migrations CLBS, et alertes de surcharge [14].

III.4.6 Automatisation

Le schéma suivant présente l’organisation du pipeline d’automatisation, composé de deux outils complémentaires : Ansible pour la configuration logicielle et l’intégration au cluster, et le pipeline PXE/TFTP pour l’installation système initiale des nœuds physiques. Ces deux mécanismes couvrent l’intégralité du cycle de vie d’un serveur, depuis la mise sous tension jusqu’à son intégration complète dans la plateforme.

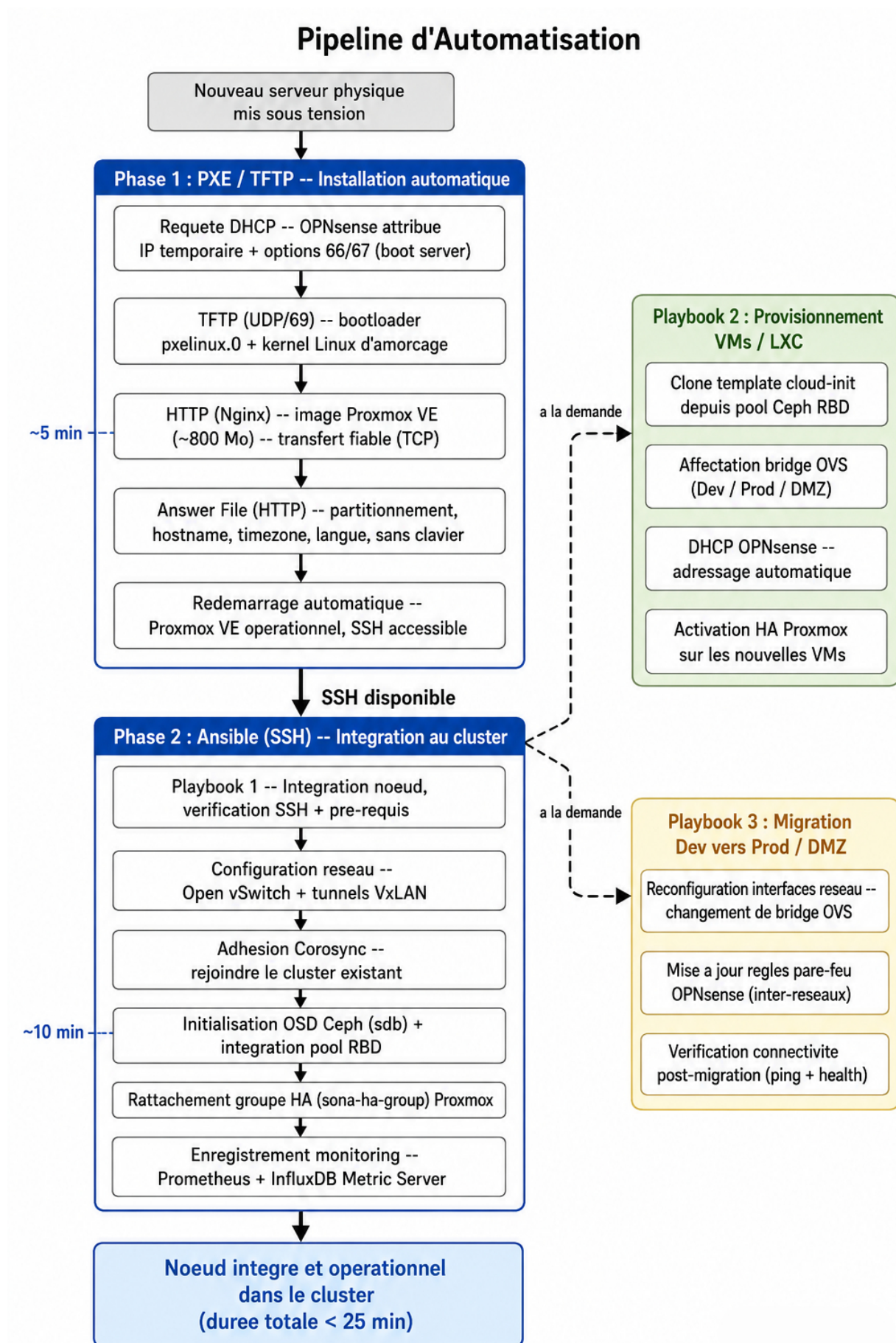


FIGURE III.6 – Pipeline d'Automatisation : PXE/TFTP et Ansible

La figure III.6 illustre le découpage en deux phases du pipeline d’automatisation. La Phase 1, entièrement automatique et opérant en environnement pre-OS, installe Proxmox VE sur le serveur via PXE, TFTP et HTTP sans aucune frappe clavier. La Phase 2 prend le relais dès que le système est démarré, en exécutant le Playbook 1 via SSH pour intégrer le nœud au cluster, configurer les VxLAN, rejoindre Ceph et enregistrer le monitoring. Les Playbooks 2 et 3, déclenchés à la demande, couvrent respectivement le provisionnement de nouvelles VMs depuis les templates cloud-init et la promotion d’environnements vers la production ou la DMZ.

III.4.6.1 Ansible : Périmètre et Playbooks

Ansible est l’outil d’automatisation principal, exécuté depuis les postes de travail des développeurs via SSH sur le réseau de management. Quatre playbooks couvrent les principaux besoins d’automatisation liés au cycle de vie des machines virtuelles.

- **Playbook 1 (Provisionnement d’une VM) :** automatise la création d’une VM à partir d’un template Debian préparé dans le pool Ceph. Il couvre le clonage du template, la configuration matérielle (CPU, RAM), l’attribution d’une adresse IP statique via cloud-init, le démarrage de la VM et sa mise à jour initiale.
- **Playbook 2 (Bootstrap d’une VM) :** initialise la gestion Ansible sur une machine fraîchement créée à partir de son adresse IP, sans qu’elle figure préalablement dans l’inventaire. Il installe les dépendances nécessaires, crée l’utilisateur dédié `ansible` avec les droits `sudo` appropriés et déploie la clé SSH de gestion.
- **Playbook 3 (Synchronisation NTP) :** configure `chrony` sur l’ensemble du cluster. Le nœud `pve-01` est désigné maître NTP interne — il se synchronise depuis `pool.ntp.org` et distribue le temps aux autres nœuds, qui le prennent comme source prioritaire avec `pool.ntp.org` en secours.
- **Playbook 4 (Provisionnement en lot) :** permet la création simultanée de plusieurs VMs à partir d’une liste de définitions YAML, en invoquant le playbook de provisionnement unitaire pour chaque entrée de la liste.

III.4.6.2 Pipeline PXE/TFTP

Dans une optique de scalabilité, l’intégration de tout nouveau serveur physique a été conçue autour d’un pipeline de provisionnement entièrement automatique, structuré en deux phases : une installation système initiale en environnement “pre-OS” via PXE/TFTP, puis une intégration logique au cluster pilotée par Ansible. L’objectif est d’enregistrer un serveur Dell PowerEdge R720 vierge dans l’infrastructure sans aucune intervention manuelle, en s’appuyant uniquement sur le démarrage réseau.

Pourquoi PXE et TFTP ? Le démarrage réseau PXE (“Preboot eXecution Environment”) repose sur une contrainte fondamentale : au moment du boot, la machine ne dispose d’aucun système d’exploitation, d’aucune pile TCP complète, ni d’aucune bibliothèque logicielle. Le client PXE, intégré directement dans le firmware de la carte réseau, ne dispose que de quelques dizaines de kilo-octets de mémoire et d’une pile réseau UDP minimale. Cette contrainte exclut l’usage de protocoles comme FTP (“File Transfer Protocol”), SSH

ou HTTP avec TLS (“Transport Layer Security”), qui nécessitent tous un OS opérationnel. TFTP (“Trivial File Transfer Protocol”), défini par la RFC 1350 et opérant sur le port UDP/69, est le seul protocole compatible avec cet environnement “pre-OS” : son implémentation ne dépasse pas quelques kilo-octets et il ne requiert ni authentification ni connexion TCP.

En revanche, la limitation inhérente à TFTP (un mécanisme de transfert “stop-and-wait” d’un bloc de 512 octets à la fois) le rend inadapté au transfert de fichiers volumineux. Dans la conception retenue, TFTP est donc strictement réservé aux fichiers de démarrage (bootloader et kernel), dont la taille totale n’excède généralement pas 40 Mo. Les fichiers volumineux, comme l’image d’installation ISO (environ 800 Mo) ou le fichier de réponses automatiques (“preseed”), sont ensuite téléchargés via HTTP dès que le kernel Linux est en mémoire et dispose d’une pile TCP complète. Le tableau III.4 compare les protocoles envisagés aux différentes étapes du pipeline de démarrage réseau.

TABLEAU III.4 – Comparaison des protocoles envisagés dans le pipeline PXE

Protocole	Transport	Nécessite un OS	Usage dans le pipeline
TFTP	UDP/69	Non	Bootloader + kernel (≤ 40 Mo)
HTTP	TCP	Oui (après kernel)	Image ISO + preseed (≈ 800 Mo)
FTP	TCP	Oui	Inutilisable en pre-OS
SSH/SFTP	TCP	Oui	Post-installation uniquement

Architecture du pipeline conçu. L’architecture retenue repose sur quatre composants principaux, hébergés dans des conteneurs LXC dédiés sur le cluster Proxmox. Le serveur DHCP (OPNsense) fournit une adresse IP au nœud en démarrage réseau et lui indique, via les options 66 et 67, l’adresse du serveur TFTP et le nom du fichier bootloader à télécharger. Un conteneur LXC dédié héberge le serveur TFTP (`tftpd-hpa`) ainsi que les fichiers de boot : le bootloader, les modules Syslinux, le kernel `vmlinuz` et l’image `initrd.img`. Un second conteneur LXC héberge un serveur HTTP Nginx qui met à disposition l’image d’installation et le fichier “preseed”. Enfin, le nœud physique cible amorce la séquence depuis son firmware.

Phase 1 : Installation automatique via PXE. Lorsqu’un nouveau serveur est mis sous tension, la séquence de démarrage réseau se déroule en plusieurs étapes successives, sans aucune frappe clavier :

1. La machine envoie une requête DHCP en broadcast ; OPNsense lui attribue une adresse IP et lui indique, via les options 66 et 67, l’adresse du serveur TFTP et le nom du fichier bootloader à récupérer.
2. Le client PXE télécharge le bootloader (`pxelinux.0` en mode BIOS Legacy, `grubx64.efi` en mode UEFI) via TFTP (UDP/69), seul protocole utilisable dans un firmware pre-OS.
3. Le bootloader charge le menu de boot puis récupère via TFTP le kernel Linux (`vmlinuz`) et l’image disque initiale (`initrd.img`).
4. Le kernel s’exécute en mémoire vive et prend le contrôle de la machine ; il télécharge ensuite l’image d’installation Proxmox VE (~ 800 Mo) et le fichier “preseed” via HTTP depuis le serveur Nginx, car TFTP est inadapté aux fichiers volumineux.

5. L'installateur Proxmox applique le fichier "preseed" (partitionnement, nom d'hôte, fuseau horaire, langue) et déroule l'installation de manière entièrement automatisée.
6. Le serveur redémarre automatiquement sur Proxmox VE fraîchement installé.

Phase 2 : Intégration au cluster via Ansible. À l'issue du redémarrage, le nœud dispose de Proxmox VE fonctionnel mais isolé. Un playbook Ansible prend le relais via SSH pour réaliser l'intégration complète :

- Adhésion au cluster **Corosync** ;
- Configuration des interfaces **Open vSwitch** et des tunnels **VxLAN** ;
- Intégration des disques physiques comme **OSDs** dans le cluster **Ceph** ;
- Rattachement au groupe HA Proxmox (**sona-ha-group**) ;
- Déclaration du nœud comme cible dans **Prometheus**.

Ce découpage en deux phases répond à une contrainte fondamentale : PXE et TFTP interviennent dans un environnement pre-OS où aucun client SSH ne peut fonctionner, tandis qu'Ansible nécessite un système d'exploitation opérationnel. Le pipeline ainsi conçu vise l'ajout d'un nouveau nœud en moins de 25 minutes sans intervention manuelle, et constitue le premier maillon de la chaîne d'automatisation de bout en bout, depuis un serveur vierge jusqu'à un nœud pleinement opérationnel. Sa validation expérimentale, qui requiert du matériel serveur dédié, est présentée parmi les perspectives de ce travail.

III.5 Partie II : Stockage

Cette partie détaille la couche de stockage distribuée de l'infrastructure, reposant entièrement sur Ceph. Elle expose l'architecture du cluster Ceph avec ses composants Monitors, Managers et OSDs, le dimensionnement du pool de stockage en tenant compte du facteur de réplication, et la configuration de l'algorithme CRUSH qui garantit la distribution des répliques sur des domaines de défaillance physiquement distincts.

III.5.1 Architecture du Cluster Ceph

Le cluster Ceph fournira la couche de stockage distribuée et partagée de l'infrastructure. Cette section en présente les composants, le dimensionnement du pool de stockage et la configuration de l'algorithme CRUSH.

III.5.1.1 Composants du Cluster

Le cluster Ceph sera composé des services suivants, répartis sur les trois nœuds physiques :

- **Monitors (MON)** : un monitor par nœud, soit trois au total. Les monitors maintiennent la carte du cluster ("cluster map") et assurent le consensus distribué. Trois monitors garantissent un quorum stable (2/3 suffisants).

- **Managers (MGR)** : PVE-02 hébergera le MGR actif, PVE-03 le MGR en standby. Le MGR exposera les métriques du cluster via un endpoint REST utilisé par Prometheus.
- **OSDs (Object Storage Daemons)** : quatre OSDs par nœud (disques sdb, sdc, sdd et sde), soit douze OSDs au total dans le cluster [18].

III.5.1.2 Dimensionnement du Pool

Avec un facteur de réplication de 3, la capacité utilisable est calculée ainsi :

$$\text{Capacité utilisable} = \frac{3 \times (300 + 3 \times 600) \text{ GB}}{3} = \frac{3 \times 2100 \text{ GB}}{3} = 2100 \text{ GB} \approx 2,05 \text{ TB}$$

L'algorithme CRUSH est configuré pour respecter les domaines de défaillance au niveau du nœud physique, garantissant que les trois répliques d'un même bloc ne se retrouvent jamais sur le même serveur. Le tableau suivant présente la répartition prévue :

TABLEAU III.5 – Répartition du stockage Ceph par usage

Usage	Stockage alloué
VM OPNsense	20 GB
HAProxy VM	16 GB
Flask VM 1	32 GB
Flask VM 2	32 GB
PostgreSQL VM	100 GB
VM Monitoring (InfluxDB + Prometheus + Grafana + Loki)	150 GB
VM CLBS	32 GB
LXC GitLab	200 GB
LXC Rundeck	16 GB
LXC OpenLDAP	16 GB
LXC NTP	8 GB
Marge, snapshots et templates	~300 GB
Total estimé	~922 GB (thin)

Le total estimé reste inférieur à la capacité utilisable de 2 100 GB car le thin provisioning de Ceph RBD ne consomme l'espace que proportionnellement aux données réellement écrites.

III.5.1.3 Configuration CRUSH et Domaines de Défaillance

Lors de la création du cluster Ceph via Proxmox VE, une règle CRUSH par défaut est générée automatiquement, organisant les OSDs selon la hiérarchie `root > host > osd` avec la politique `chooseleaf firstn 0 type host`. Cela garantit que les trois répliques d'un objet sont placées sur trois hôtes physiques distincts.

Le tableau III.6 résume les principaux paramètres CRUSH appliqués automatiquement par Proxmox lors de la création du cluster Ceph.

TABLEAU III.6 – Paramètres CRUSH appliqués automatiquement par Proxmox

Paramètre	Valeur
Domaine de défaillance	host (généré automatiquement par Proxmox)
Facteur de réplication (size)	3 (une réplique par nœud physique)
min_size	2 (défaut Ceph), pool accessible avec 2 nœuds opérationnels

III.6 Partie III : Réseau

Cette partie décrit la couche réseau de l'infrastructure, conçue pour contourner les limitations du réseau physique Sonatrach. Elle présente les contraintes réseau du site et la solution retenue basée sur VxLAN et Open vSwitch, la topologie en deux niveaux de segmentation (réseaux d'infrastructure et réseaux applicatifs), le plan d'adressage complet, la VM OPN-sense assurant le routage centralisé et le filtrage, et l'architecture de pare-feu à deux niveaux couvrant à la fois OPNsense et le pare-feu natif Proxmox.

III.6.1 Contexte et Contraintes Réseau

Le réseau Sonatrach met à disposition un sous-réseau 192.168.10.0/24 utilisé pour l'accès SSH et l'interface web Proxmox. La connectivité Internet des nœuds est assurée par un poste jouant le rôle de routeur externe à l'adresse 192.168.10.36, qui réalise du NAT et du forwarding IP.

Le switch Sonatrach n'autorise que le VLAN 10 natif et ne permet pas la configuration de VLANs supplémentaires. Pour créer les réseaux logiques nécessaires à l'infrastructure, VxLAN a été retenu comme technologie d'encapsulation. VxLAN permet de créer des réseaux logiques de niveau 2 tunnelisés au-dessus du réseau IP existant, sans modifier la configuration du switch. Open vSwitch gèrera la création et la terminaison de ces tunnels sur chaque nœud Proxmox.

III.6.2 Topologie Réseau

La topologie réseau repose sur deux niveaux de segmentation complémentaires, décrits ci-dessous, ainsi que sur les bridges Open vSwitch déployés sur chaque nœud.

III.6.2.1 Deux Niveaux de Segmentation

La topologie retenue s'articule autour de deux niveaux complémentaires.

Niveau 1 : Réseaux d'infrastructure (VxLAN inter-nœuds). Pour les réseaux de contrôle (Corosync, Ceph Cluster, Ceph Public, Live Migration), des tunnels VxLAN point-à-point en maillage complet sont établis entre les trois nœuds. Chaque nœud crée deux ports VxLAN vers chacun des deux autres, garantissant une connectivité directe sans point de transit central.

Niveau 2 : Réseaux applicatifs (OPNsense). Pour les réseaux Dev, Prod et DMZ, une architecture centralisée autour d'une VM OPNsense est utilisée. Trois bridges OVS internes sont configurés sur chaque nœud. OPNsense dispose de quatre interfaces virtuelles : WAN ("Wide Area Network") connectée au réseau de management (192.168.10.65, gateway 192.168.10.36), et trois interfaces LAN ("Local Area Network") connectées aux bridges Dev, Prod et DMZ.

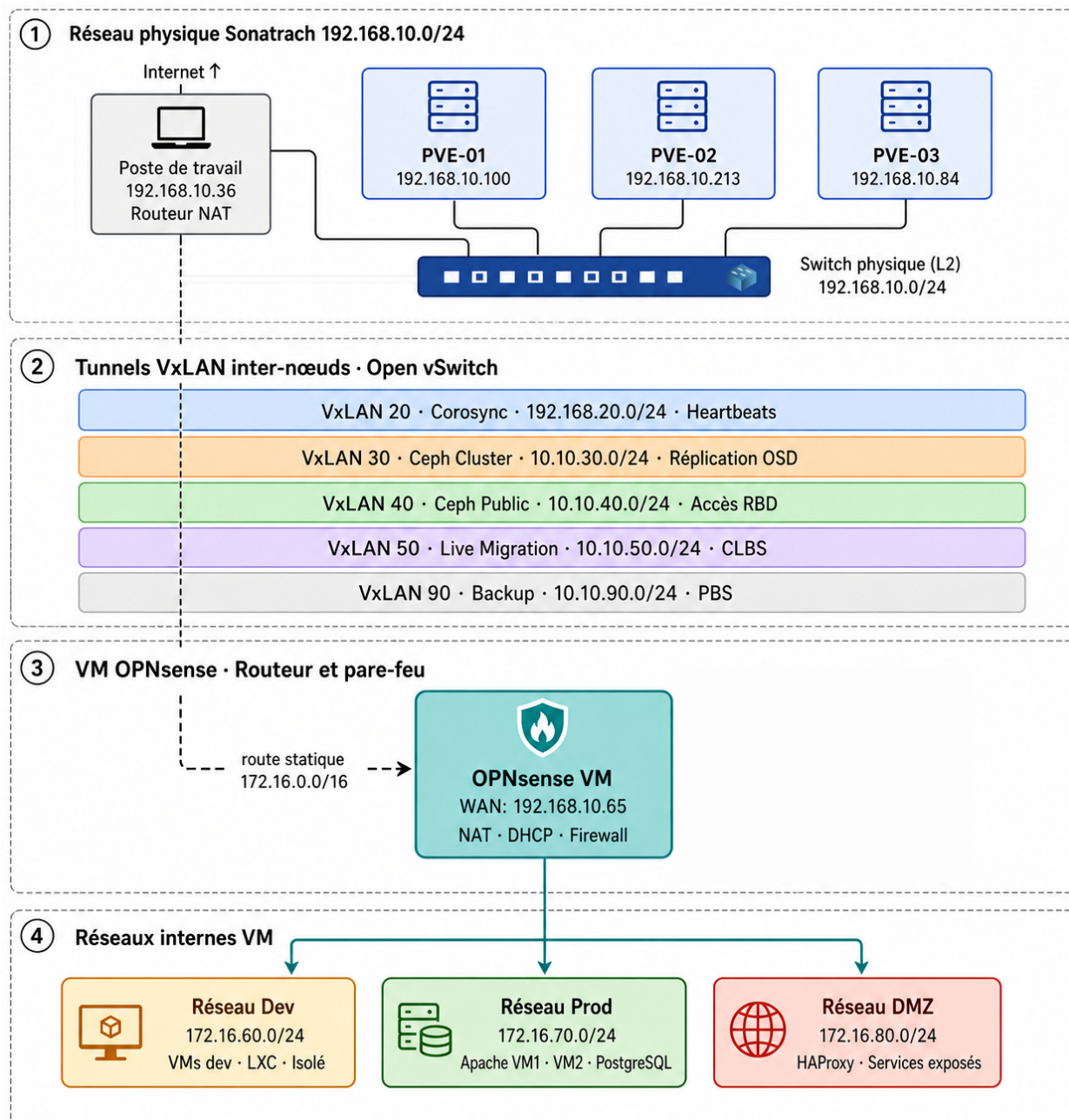


FIGURE III.7 – Topologie Réseau et Plan VxLAN

La figure III.7 présente les deux niveaux de segmentation réseau retenus. Le premier niveau repose sur des tunnels VxLAN point-à-point en maillage complet entre les trois nœuds physiques, dédiés aux flux de contrôle (Corosync, Ceph, migrations). Le second niveau utilise OPNsense comme routeur centralisé pour les réseaux applicatifs Dev, Prod et DMZ, chacun associé à un bridge OVS distinct sur chaque nœud.

III.6.2.2 Bridges Open vSwitch par Nœud

Le tableau III.7 décrit les bridges Open vSwitch configurés sur chaque nœud, le VxLAN associé, le sous-réseau correspondant et le rôle de chacun.

TABLEAU III.7 – Bridges OVS et tunnels VxLAN par nœud Proxmox

Bridge OVS	VxLAN associé	Sous-réseau	Rôle
vmbr0	Réseau physique natif	192.168.10.0/24	Bridge principal. Accès management, SSH, Web UI Proxmox, interface WAN d'OPNsense.
vmbr60	vxlan60-int	172.16.60.0/24	Réseau Dev.
vmbr70	vxlan70-int	172.16.70.0/24	Réseau Prod.
vmbr80	vxlan80-int	172.16.80.0/24	Réseau DMZ.

III.6.3 Plan d'Adressage

Le plan d'adressage est organisé en deux familles : les réseaux VxLAN d'infrastructure pour les flux de contrôle, et les réseaux applicatifs internes gérés par OPNsense.

Réseaux VxLAN d'infrastructure :

- **VxLAN 10, Management (192.168.10.0/24)** : Web UI Proxmox, SSH, API (réseau physique existant).
- **VxLAN 20, Corosync (192.168.20.0/24)** : Heartbeats et quorum cluster.
- **VxLAN 30, Ceph Cluster (10.10.30.0/24)** : Réplication OSD interne.
- **VxLAN 40, Ceph Public (10.10.40.0/24)** : Accès des VMs au stockage RBD.
- **VxLAN 50, Live Migration (10.10.50.0/24)** : Migrations à chaud et CLBS.

Réseaux applicatifs internes :

- **Réseau Dev (172.16.60.0/24, gateway 172.16.60.1)** : environnements de développement et de test.
- **Réseau Prod (172.16.70.0/24, gateway 172.16.70.1)** : environnements de production.
- **Réseau DMZ (172.16.80.0/24, gateway 172.16.80.1)** : services exposés, dont HAProxy.

Le tableau III.8 détaille l'adressage IP statique attribué à chaque nœud Proxmox sur les différents réseaux logiques de l'infrastructure.

TABLEAU III.8 – Adressage IP statique des nœuds Proxmox par réseau logique

Nœud	Management	VxLAN 20	VxLAN 30	VxLAN 40	VxLAN 50
PVE-01	192.168.10.100	192.168.20.11	10.10.30.11	10.10.40.11	10.10.50.11
PVE-02	192.168.10.213	192.168.20.12	10.10.30.12	10.10.40.12	10.10.50.12
PVE-03	192.168.10.84	192.168.20.13	10.10.30.13	10.10.40.13	10.10.50.13

III.6.4 VM OPNsense : Routeur et Pare-feu Centralisé

La VM OPNsense centralise le routage et le filtrage des réseaux applicatifs. Cette section détaille ses interfaces et leurs rôles, puis les mécanismes assurant sa haute disponibilité.

III.6.4.1 Interfaces et Rôles

La VM OPNsense constitue le cœur du réseau applicatif. Il s'agit d'une distribution FreeBSD dédiée aux fonctions de routage et de sécurité réseau. Dans cette architecture, OPNsense sera déployée avec quatre interfaces réseau virtuelles :

- **Interface WAN (192.168.10.65/24)** : connectée au bridge OVS de management, avec gateway 192.168.10.36. Assure l'accès Internet sortant via NAT pour toutes les VMs internes.
- **Interface LAN1 (172.16.60.1/24)** : passerelle du réseau Dev.
- **Interface LAN2 (172.16.70.1/24)** : passerelle du réseau Prod.
- **Interface LAN3 (172.16.80.1/24)** : passerelle du réseau DMZ.

OPNsense intègre nativement dnsmasq, qui sera utilisé comme serveur DHCP pour les trois réseaux internes. L'attribution des adresses IP aux VMs sera ainsi automatisée dès leur connexion à l'un des bridges applicatifs, simplifiant le provisionnement via les playbooks Ansible.

III.6.5 Architecture du Pare-feu

La sécurité réseau de l'infrastructure repose sur deux niveaux de filtrage complémentaires : le pare-feu OPNsense pour les flux entre réseaux applicatifs, et le pare-feu natif de Proxmox au niveau de l'hyperviseur.

III.6.5.1 Pare-feu OPNsense : Filtrage Inter-réseaux

OPNsense applique les politiques de filtrage entre les réseaux applicatifs Dev, Prod et DMZ. Les règles de conception retenues sont les suivantes :

Règles autorisées :

- Dev → WAN : pour les mises à jour et les dépôts de paquets.
- DMZ → WAN : pour les services exposés nécessitant un accès sortant.

- Prod → DMZ : pour la communication entre production et services exposés.

Règles bloquées :

- Dev → Prod : pour isoler les environnements de développement de la production.
- Dev → DMZ : pour empêcher l'accès aux services exposés depuis le développement.
- Prod → WAN direct : le trafic de production passe obligatoirement par la DMZ.

III.6.5.2 Pare-feu Proxmox : Filtrage au Niveau Hyperviseur

Proxmox VE intègre nativement un mécanisme de pare-feu basé sur iptables, configurable aux niveaux cluster, nœud et VM individuelle. Les règles de conception retenues pour protéger la plateforme sont les suivantes :

- **Isolation des réseaux Ceph** : interdire tout trafic des réseaux applicatifs (172.16.x.0/24) vers les réseaux Ceph Cluster (VxLAN 30) et Ceph Public (VxLAN 40).
- **Isolation du réseau Corosync** : interdire tout accès extérieur au VxLAN 20. Seuls les nœuds Proxmox communiquent sur ce réseau.
- **Protection du management** : restreindre l'accès à l'interface web Proxmox (port 8006) et SSH (port 22) aux seules adresses du sous-réseau 192.168.10.0/24.
- **Moindre privilège par VM** : n'autoriser que les ports applicatifs strictement nécessaires sur chaque VM.

Architecture du pare-feu : deux niveaux de filtrage

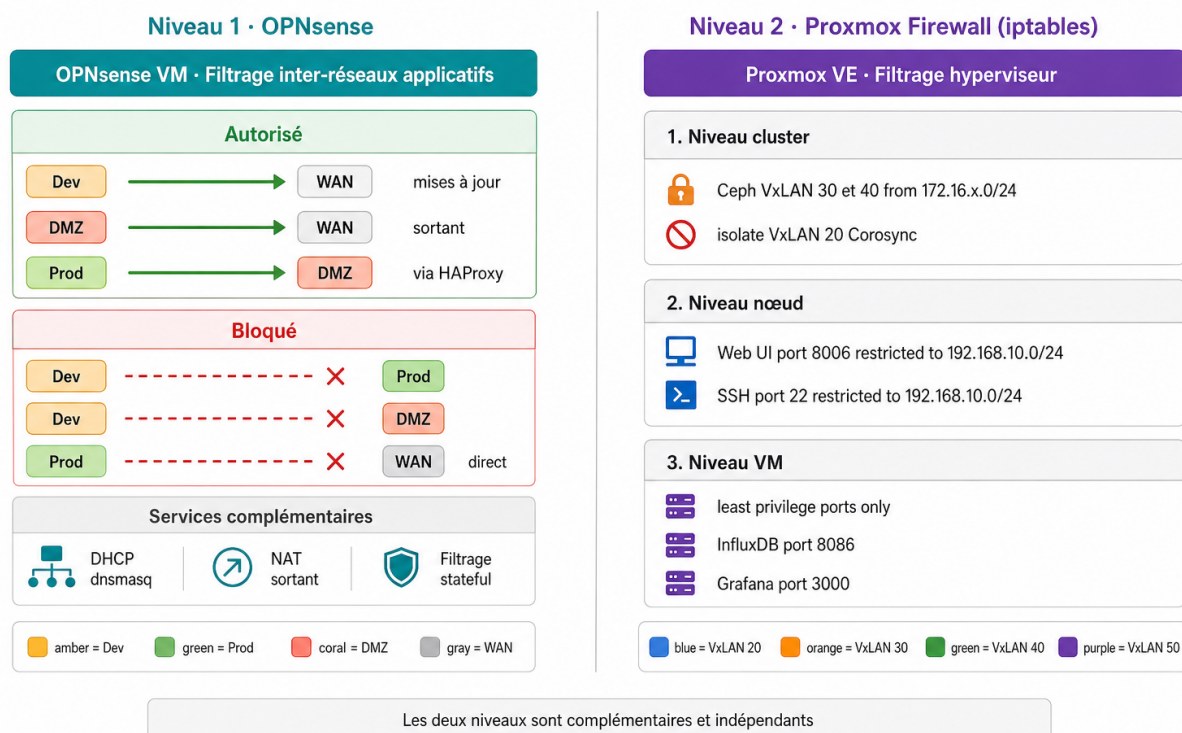


FIGURE III.8 – Architecture du Pare-feu : OPNsense et Proxmox Firewall

La figure III.8 représente les deux niveaux de filtrage coexistant dans l'infrastructure. OPNsense assure le filtrage inter-réseaux entre Dev, Prod et DMZ selon des politiques d'isolation applicative. Le pare-feu Proxmox, basé sur iptables, opère au niveau de l'hyperviseur pour protéger les réseaux d'infrastructure (Ceph, Corosync) et restreindre l'accès à l'interface de management. Cette défense en profondeur garantit qu'aucune VM applicative ne puisse atteindre les réseaux de contrôle du cluster.

III.7 Services Complémentaires d'Infrastructure

Cette section présente les services transverses qui complètent les trois couches de base (calcul, stockage, réseau). Ils sont déployés sous forme de conteneurs LXC sur le cluster Proxmox et couvrent la supervision avancée, la synchronisation temporelle, la gestion des identités, la gestion du code source et l'automatisation opérationnelle.

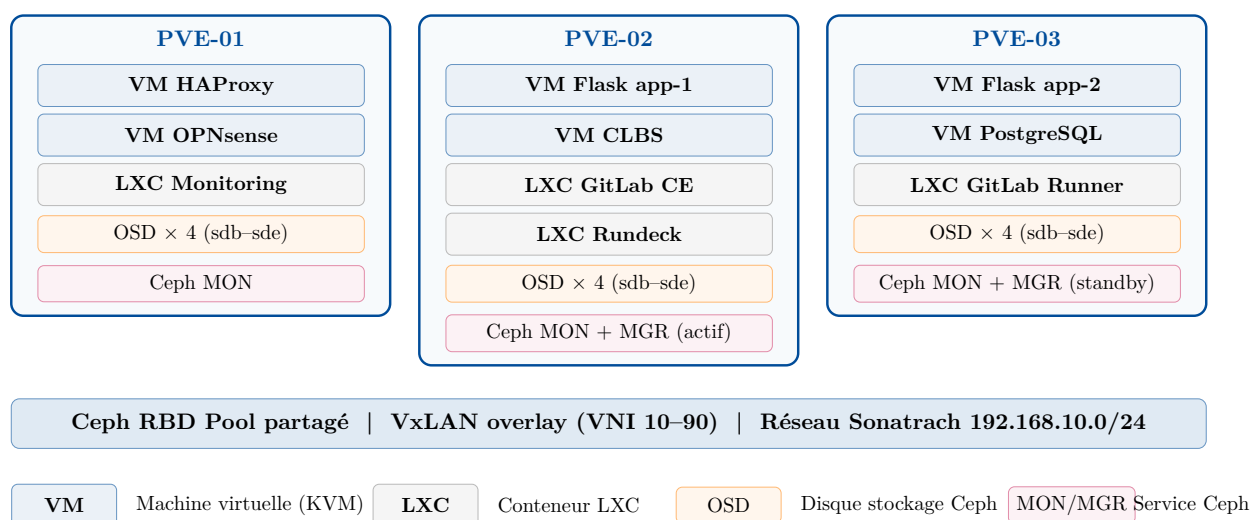


FIGURE III.9 – Diagramme de déploiement : répartition des services sur les trois nœuds physiques

La figure III.9 présente la répartition des services virtualisés sur les trois nœuds physiques du cluster. PVE-01 concentrera les services réseau et de monitoring. PVE-02 hébergera les services applicatifs de développement (GitLab, Rundeck, OpenLDAP) et le service CLBS. PVE-03 prendra en charge les instances de l'application de démonstration et le GitLab Runner. Chaque nœud contribuera avec quatre OSDs au pool Ceph partagé et hébergera un Monitor Ceph. Les Managers actif et standby seront respectivement sur PVE-02 et PVE-03.

III.7.1 Système de Notification : Grafana Alertmanager

Grafana intègre nativement un système d'alerte permettant de définir des règles de déclenchement directement sur les tableaux de bord existants. Lorsqu'une condition est vérifiée, Grafana génère une alerte et la transmet via un contact point configuré. Dans notre infrastructure, les notifications seront acheminées vers une adresse Gmail via le protocole SMTP [14].

Les alertes couvrent les métriques critiques suivantes :

- **Cluster Ceph** : OSD en état down, pool en état degraded, espace disponible inférieur à 20 %.
- **Nœuds Proxmox** : charge CPU supérieure à 85 %, mémoire disponible inférieure à 10 %, perte de quorum Corosync.
- **Service CLBS** : nombre de migrations excessif déclenchant le circuit breaker, impossibilité de contacter l'API Proxmox ou InfluxDB.
- **Application Sona-Web** : backend HAProxy hors service, taux d'erreurs HTTP 5xx anormal.

À terme, Apprise est envisagé comme couche d'abstraction de notification, permettant d'acheminer les alertes vers plusieurs canaux (email, Discord, Slack) sans dépendre d'un serveur SMTP dédié.

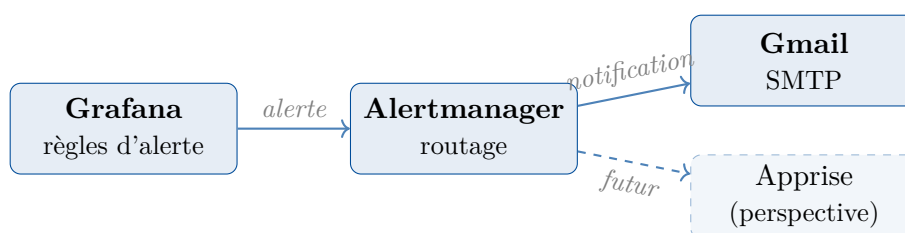


FIGURE III.10 – Pipeline de notification Grafana Alertmanager

La figure III.10 présente le pipeline de notification. Grafana évalue les règles d'alerte définies sur les tableaux de bord et transmet les déclenchements à l'Alertmanager intégré, qui les achemine vers le contact point configuré (Gmail via SMTP). La liaison vers Apprise est prévue en extension pour supporter plusieurs canaux de notification sans serveur SMTP dédié.

III.7.2 Centralisation des Journaux : Loki, Alloy et Grafana

La centralisation des journaux reposera sur la pile Loki, Alloy et Grafana. Loki est une solution de stockage de logs qui indexe uniquement les métadonnées (labels) et stocke les messages compressés, ce qui lui confère une empreinte en ressources nettement inférieure à celle d'Elasticsearch. Grafana Alloy (anciennement Grafana Agent) est l'agent de collecte déployé sur chaque nœud et VM ; il transmet les logs vers Loki. Grafana permet ensuite d'interroger et de visualiser ces journaux via le langage LogQL.

Alloy sera installé sur chaque nœud Proxmox et sur les VMs à superviser. Loki sera hébergé dans le conteneur LXC monitoring, aux côtés d'InfluxDB et de Prometheus.

Les services retenus pour la collecte des journaux sont les suivants :

- **Nœuds Proxmox** : journaux système (auth.log, syslog, kern.log) pour la sécurité et le diagnostic matériel.
- **Cluster Ceph** : journaux des démons OSD et MON pour le suivi des événements de santé, de récupération et de rebalancing.
- **OPNsense** : journaux du pare-feu (connexions bloquées, règles déclenchées) et du service DHCP.

- **Authentification SSH** : tentatives de connexion et échecs d'authentification sur les nœuds et VMs (sécurité).

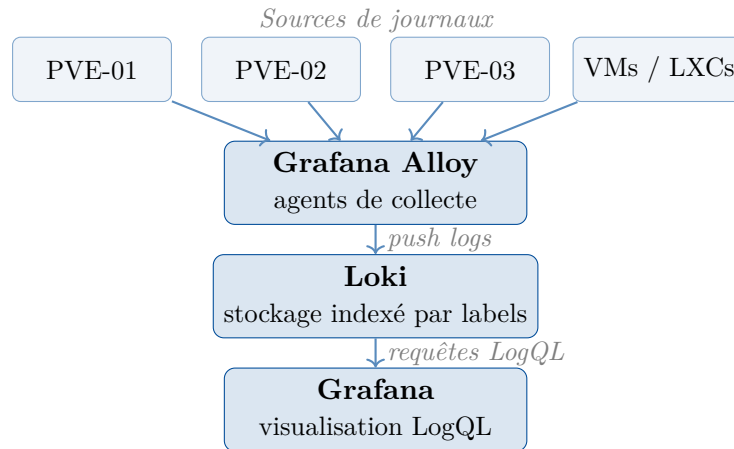


FIGURE III.11 – Architecture de centralisation des journaux : Alloy, Loki et Grafana

La figure III.11 présente l'architecture de collecte centralisée des journaux. Les agents Alloy, déployés sur chaque nœud Proxmox et sur les VMs, collectent les journaux et les transmettent vers Loki par push. Loki stocke les messages compressés en indexant uniquement leurs labels. Grafana interroge Loki via LogQL et affiche les journaux dans les mêmes tableaux de bord que les métriques, offrant une vue unifiée de l'infrastructure.

III.7.3 Synchronisation Temporelle : Serveur NTP

La synchronisation temporelle est un prérequis pour tout système distribué. Dans notre infrastructure, Ceph et Corosync dépendent de l'horloge système de chaque nœud : un écart supérieur à quelques secondes peut provoquer des erreurs de quorum Ceph, des incohérences dans les journaux distribués ou des refus d'authentification.

Plutôt que de dédier un conteneur LXC à ce rôle, le serveur NTP sera hébergé directement sur le nœud physique **PVE-01** (192.168.10.100). L'implémentation retenue est **Chrony**, plus réactif que ntpd classique pour les corrections de dérive d'horloge. PVE-01 se synchronisera sur `pool.ntp.org` et constituera la référence temporelle unique pour l'ensemble des nœuds Proxmox ainsi que pour les machines virtuelles et conteneurs LXC de l'infrastructure.

Plutôt que de dédier un conteneur LXC à ce rôle, le serveur NTP sera hébergé directement sur les nœuds physiques Proxmox. **PVE-01** (192.168.10.100) constitue la référence temporelle principale et se synchronise sur `pool.ntp.org` ; en cas d'indisponibilité, les clients basculent successivement vers **PVE-02**, puis **PVE-03**, ensuite vers `pool.ntp.org` directement, et enfin sur l'horloge locale en dernier recours. L'implémentation retenue est **Chrony**, dont l'algorithme de sélection de source gère ce basculement nativement. Les VMs hébergées sur des VLANs distincts (172.16.60/70/80.0/24) atteignent les nœuds PVE via **OPNsense**.

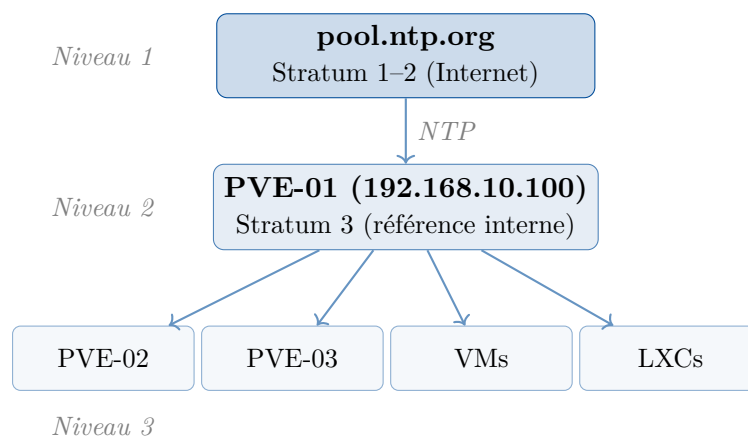


FIGURE III.12 – Hiérarchie NTP : synchronisation temporelle de l'infrastructure

La figure III.12 illustre la hiérarchie de synchronisation temporelle. PVE-01 joue le rôle de référence interne (Stratum 3) en se synchronisant sur `pool.ntp.org`. Les autres nœuds et VMs interrogent PVE-01 en priorité, et basculent vers `pool.ntp.org` directement si PVE-01 est indisponible. Cette architecture garantit la cohérence temporelle requise par Ceph et Corosync.

III.7.4 Gestion des Identités : Proxmox et Perspectives

La gestion des identités de l'infrastructure repose sur le système de gestion des utilisateurs et des permissions intégré à Proxmox. Cette solution native permet de définir des comptes utilisateurs, d'attribuer des rôles granulaires et d'appliquer des politiques d'accès adaptées aux différents profils (administrateur, opérateur, lecture seule), sans nécessiter de composant externe.

À titre de perspective, le déploiement d'un annuaire OpenLDAP pourrait être envisagé dans une phase ultérieure du projet afin de centraliser l'authentification sur l'ensemble des services de l'infrastructure, notamment Grafana, GitLab et Rundeck. Cette évolution permettrait d'unifier la gestion des accès depuis un point unique et d'assurer une cohérence des politiques de contrôle d'accès à l'échelle de la plateforme.

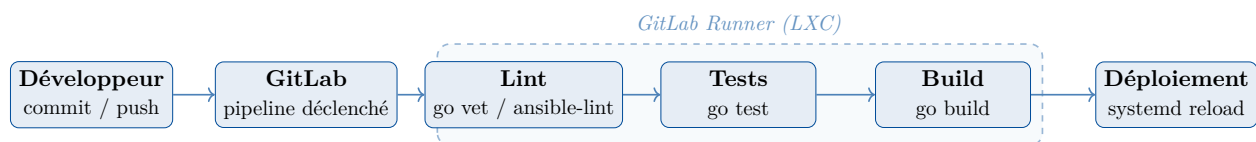
III.7.5 Gestion du Code Source et CI/CD : GitLab

GitLab sera déployé sous forme de conteneur LXC Turnkey Linux sur le cluster Proxmox. Le template Turnkey GitLab repose sur Debian 12 et intègre GitLab Community Edition version 17, antérieure à la dernière version disponible (18.x au moment de la rédaction). Une mise à jour vers la dernière version sera réalisée après le déploiement initial, en suivant la procédure officielle de mise à niveau par paliers [20].

Les projets hébergés sur cette instance GitLab seront les suivants :

- **clbs** : code source du service d'équilibrage de charge en Go.
- **ansible-infra** : ensemble des playbooks Ansible d'automatisation de l'infrastructure.

Des pipelines CI/CD GitLab seront configurés pour automatiser les vérifications de qualité de code et les tests unitaires du service CLBS, ainsi que la validation des syntaxes des playbooks Ansible. Les GitLab Runners seront déployés sous forme de conteneurs LXC sur le cluster et enregistrés dans l'instance GitLab.

FIGURE III.13 – Pipeline CI/CD GitLab pour les projets `clbs` et `ansible-infra`

La figure III.13 présente le pipeline CI/CD configuré pour les deux projets hébergés sur l'instance GitLab. À chaque push, GitLab déclenche automatiquement le pipeline. Les trois stages (Lint, Tests, Build) s'exécutent sur le GitLab Runner déployé en LXC sur le cluster. Le stage de déploiement met à jour le service `systemd` correspondant sur la VM cible.

III.7.6 Automatisation Opérationnelle : Rundeck

Rundeck est un outil d'automatisation opérationnelle qui expose les playbooks Ansible via une interface web. Il permet aux administrateurs d'exécuter des jobs prédéfinis sans accès direct à la ligne de commande, d'historiser les exécutions avec leurs journaux complets, et de contrôler les permissions d'exécution par utilisateur ou groupe.

Dans notre infrastructure, Rundeck sera déployé dans un conteneur LXC et connecté au dépôt `ansible-infra` hébergé sur GitLab. Les playbooks de provisionnement de VMs, d'intégration de nœuds et de promotion d'environnements sont ainsi accessibles via une interface graphique, sans nécessiter d'accès SSH direct aux nœuds Proxmox.

III.8 Conclusion

Ce chapitre a présenté la conception complète de l'infrastructure Cloud privée hyperconvergente. La partie Calcul couvre les serveurs physiques (5 disques SAS par nœud), le cluster Proxmox avec Corosync et HA, le mécanisme CLBS, l'application Sona-Web, la stack de monitoring et l'automatisation via Ansible et PXE. La partie Stockage détaille le cluster Ceph avec ses douze OSDs, la capacité utilisable de 2,05 TB et la politique CRUSH. La partie Réseau expose l'architecture VxLAN gérée par Open vSwitch, le plan d'adressage complet, le routage centralisé via OPNsense et le plan de pare-feu à deux niveaux. Enfin, les services complémentaires (Alertmanager, Loki, NTP, GitLab, Rundeck) complètent l'infrastructure pour en assurer la supervision, la sécurité et la gestion unifiée.

L'ensemble de ces choix de conception constitue une base cohérente et documentée pour la phase d'implémentation présentée dans le chapitre suivant.

Chapitre IV

Implémentation de la Solution Proposée

IV.1 Introduction

Ce chapitre présente la mise en œuvre concrète de la solution proposée dans le chapitre précédent. L'objectif est de montrer les étapes réelles de déploiement de l'infrastructure hyperconvergée, depuis l'installation de la plateforme de virtualisation jusqu'aux services applicatifs et aux mécanismes d'automatisation. Pour améliorer la lisibilité, l'implémentation est organisée en trois grandes parties : la couche calcul, la couche stockage, puis la couche réseau.

Les opérations ont été réalisées avec plusieurs outils complémentaires : l'interface web Proxmox VE, la ligne de commande Linux, les fichiers de configuration système, OPNsense, Ceph, ainsi que des playbooks Ansible pour automatiser certaines tâches répétitives. Des captures d'écran sont prévues aux points importants afin d'illustrer visuellement les résultats obtenus.

IV.2 Partie I : Couche Calcul

Cette partie couvre l'ensemble des opérations réalisées au niveau de la couche de calcul de l'infrastructure : installation de Proxmox VE, formation du cluster, déploiement du monitoring, implémentation du mécanisme CLBS, et déploiement des services applicatifs.

IV.2.1 Préparation de l'Infrastructure Physique

La préparation de l'infrastructure physique constitue la première étape du déploiement. Elle comprend l'installation de Proxmox VE sur chaque nœud ainsi que la configuration de la connectivité réseau nécessaire au bon fonctionnement du cluster.

IV.2.1.1 Installation de Proxmox VE sur les nœuds

L'installation de la plateforme de virtualisation a commencé par le téléchargement de l'image ISO de Proxmox VE version 9.1.1. Cette image a ensuite été copiée sur une clé USB bootable utilisée pour démarrer chaque serveur Dell PowerEdge R720. Une installation séparée a été réalisée sur chacun des trois nœuds du cluster.

Pendant l'installation, le disque système `sda` a été utilisé pour héberger Proxmox VE. Une configuration réseau statique a aussi été appliquée afin de donner à chaque nœud une adresse de management fixe sur le réseau physique `192.168.10.0/24`. Les adresses retenues sont `192.168.10.100` pour PVE-01, `192.168.10.213` pour PVE-02 et `192.168.10.84` pour PVE-03.

Une fois l'installation terminée, la vérification a été effectuée à travers l'interface web de Proxmox, accessible sur le port `8006`. Comme le montre la Figure [IV.1](#), cette étape a permis de confirmer que chaque nœud était opérationnel et prêt pour la suite du déploiement.

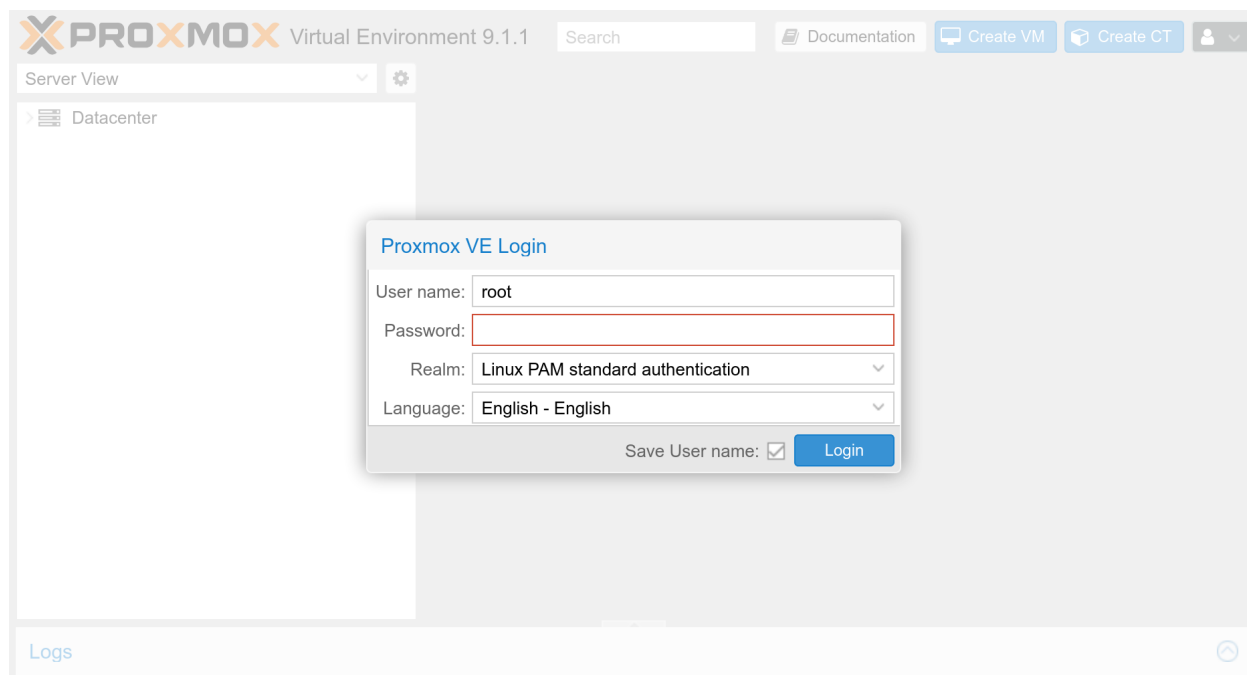


FIGURE IV.1 – Interface web Proxmox au premier login sur PVE-01.

La Figure IV.1 montre l'interface web de Proxmox VE après le premier démarrage sur PVE-01. On y retrouve l'écran de connexion avec les champs d'authentification, confirmant que le service web est actif et accessible sur le port 8006 du réseau de management.

IV.2.1.2 Configuration de la connectivité Internet via le laptop gateway

Dans l'environnement de laboratoire, aucun routeur physique dédié n'était disponible pour fournir l'accès Internet aux nœuds Proxmox. Pour cette raison, un laptop a été configuré pour jouer le rôle de passerelle NAT entre le réseau interne de l'infrastructure et Internet. Il possède deux interfaces : `enp0s20f0u4c2` côté infrastructure avec l'adresse `192.168.10.36/24`, et `wlan0` côté Internet avec l'adresse `10.247.78.8/24`.

La première étape consiste à activer le routage IP afin que le système puisse transférer les paquets d'une interface à l'autre. Cette activation doit être faite de manière immédiate puis rendue permanente.

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo vim /etc/sysctl.d/99-ipforward.conf
# Ajouter :
net.ipv4.ip_forward=1
```

Figure IV.2 – Activation du routage IP sur le laptop

Les commandes de la Figure IV.2 activent le forwarding IP : la première l'active immédiatement, la seconde le rend permanent via le fichier `99-ipforward.conf`. Sans cette étape, le laptop ne peut pas transmettre les paquets entre ses deux interfaces réseau.

Ensuite, des règles `iptables` ont été ajoutées pour réaliser la translation d'adresses NAT et autoriser le forwarding du trafic entre les interfaces du laptop.

```
sudo iptables -t nat -A POSTROUTING -o wlan0 -j MASQUERADE
sudo iptables -A FORWARD -i enp0s20f0u4c2 -o wlan0 -j ACCEPT
sudo iptables -A FORWARD -i wlan0 -o enp0s20f0u4c2 -m state --state
    RELATED,ESTABLISHED -j ACCEPT
```

Figure IV.3 – Configuration NAT avec iptables

Les trois règles de la Figure IV.3 assurent respectivement : le masquage des adresses sources sur l'interface Internet, l'autorisation du trafic sortant, et l'autorisation des réponses correspondant à des connexions établies. Ensemble, elles permettent aux nœuds Proxmox d'atteindre Internet à travers le laptop.

Pour éviter de perdre ces règles après redémarrage, elles ont été sauvegardées puis activées automatiquement au boot du laptop.

```
sudo pacman -S iptables-nft
sudo iptables-save | sudo tee /etc/iptables/iptables.rules
sudo systemctl enable iptables
```

Figure IV.4 – Persistance des règles iptables

Les commandes de la Figure IV.4 garantissent la persistance du NAT : la sauvegarde dans `/etc/iptables/iptables.rules` combinée à l'activation du service assure que le NAT sera opérationnel dès le démarrage du laptop, sans intervention manuelle à chaque redémarrage.

Sur chaque nœud Proxmox, la passerelle par défaut a été définie vers 192.168.10.36, avec des serveurs DNS publics.

Cette configuration a été ajoutée dans le fichier `/etc/network/interfaces`.

```
auto vmbr0
iface vmbr0 inet static
    address 192.168.10.XX/24
    gateway 192.168.10.36
    dns-nameservers 8.8.8.8 8.8.4.4
```

Figure IV.5 – Configuration réseau de base d'un nœud Proxmox

La configuration de la Figure IV.5 attribue une adresse IP statique à chaque nœud, définit le laptop comme passerelle par défaut et utilise les DNS publics de Google. Le paramètre XX est remplacé par la valeur spécifique à chaque nœud (100, 213 ou 84).

Enfin, des routes statiques ont été ajoutées sur le laptop afin de rediriger les réseaux internes 172.16.60.0/24, 172.16.70.0/24 et 172.16.80.0/24 vers la VM OPNsense, qui assure ensuite le routage vers ces sous-réseaux.

```
nmcli connection modify "Wired connection 1" \
+ipv4.routes "172.16.60.0/24 192.168.10.65" \
+ipv4.routes "172.16.70.0/24 192.168.10.65" \
+ipv4.routes "172.16.80.0/24 192.168.10.65"

nmcli connection up "Wired connection 1"
```

Figure IV.6 – Ajout des routes statiques vers OPNsense

Les routes de la Figure IV.6 indiquent au laptop que tout trafic destiné aux réseaux internes (172.16.x.0/24) doit être transmis vers l'adresse IP d'OPNsense (192.168.10.65), qui se charge du routage vers la zone applicative appropriée.

Des tests simples comme `ping 8.8.8.8`, `ping google.com` et `apt update` ont confirmé que les nœuds disposaient correctement d'un accès Internet.

IV.2.2 Formation du cluster Proxmox et haute disponibilité

Après avoir installé et connecté les trois nœuds à Internet, l'étape suivante a consisté à les regrouper en un cluster unifié et à configurer les mécanismes de haute disponibilité.

IV.2.2.1 Création et jonction au cluster

Après l'installation de Proxmox VE sur les trois serveurs, la phase suivante a consisté à former un cluster unique. Cette opération a été lancée depuis le premier nœud PVE-01, qui devient le nœud initial du cluster.

```
pvecm create Cluster-Proxmox-VE
```

Figure IV.7 – Création du cluster Proxmox

La commande de la Figure IV.7 initialise le cluster sur PVE-01, configure automatiquement Corosync pour le mécanisme de heartbeat et génère les fichiers de configuration dans `/etc/pve/`. PVE-01 devient le nœud de référence pour les adhésions suivantes.

Les deux autres nœuds ont ensuite rejoint ce cluster en utilisant l'adresse IP du nœud principal.

```
pvecm add 192.168.10.100
```

Figure IV.8 – Jonction de PVE-02 et PVE-03 au cluster

La commande de la Figure IV.8, exécutée successivement sur PVE-02 et PVE-03, contacte le nœud initial, échange les certificats d'authentification et intègre le nœud au cluster Corosync existant.

Le réseau Corosync, utilisé pour le heartbeat entre les nœuds, a été isolé sur le VxLAN 20 afin de séparer le trafic de cluster du reste du trafic d'administration ou de production. Une fois la configuration terminée, la commande de vérification a confirmé la présence des trois nœuds avec quorum atteint.

```
pvecm status
```

Figure IV.9 – Vérification de l'état du cluster

La commande `pvecm status` de la Figure IV.9 affiche l'état du cluster Corosync : nombre de membres actifs, version du quorum, et confirmation de l'atteinte du quorum. Une sortie avec `Quorum: Yes` et trois membres confirme que le cluster est correctement formé. Comme l'illustre la Figure IV.10, ce résultat est également visible dans l'interface web.

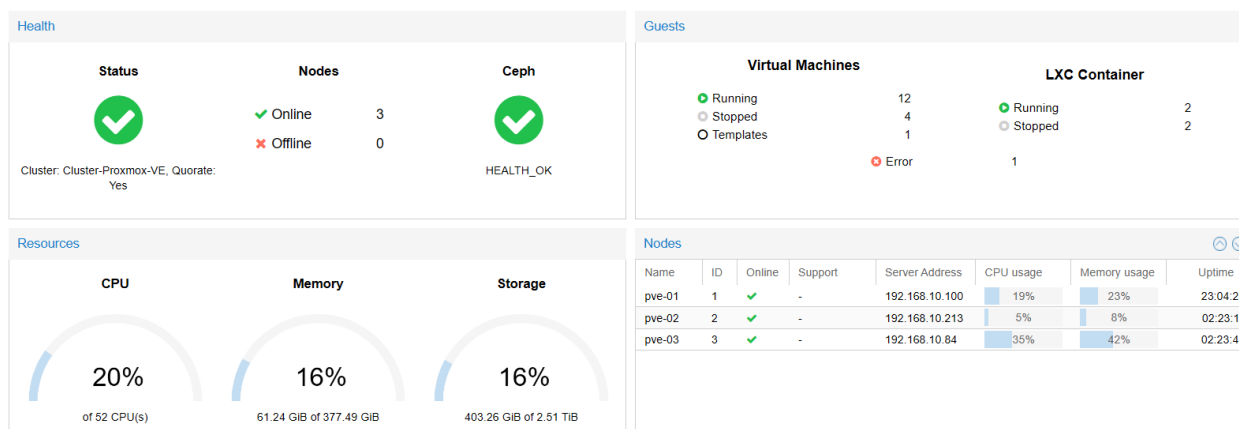


FIGURE IV.10 – Vue du cluster Proxmox montrant les trois nœuds en ligne.

La Figure IV.10 affiche la vue d'ensemble du datacenter dans l'interface web Proxmox, où les trois nœuds PVE-01, PVE-02 et PVE-03 apparaissent simultanément en ligne. Le tableau de bord indique également l'état de santé du cluster Ceph et le nombre de VMs en cours d'exécution. L'avertissement affiché dans l'interface ("*No valid subscription*") est lié à la tentative d'accès au dépôt `pve-enterprise` de Proxmox, qui nécessite une licence commerciale. Dans le cadre de ce projet, la version communautaire est utilisée; la configuration du fichier `/etc/apt/sources.list.d/` a été modifiée pour pointer vers le dépôt `pve-no-subscription` afin d'éliminer cet avertissement sans impacter le fonctionnement du cluster.

IV.2.2.2 Configuration de la migration à chaud

La migration à chaud des machines virtuelles a été configurée pour utiliser le réseau dédié VxLAN 50. Ce choix permet de ne pas mélanger le trafic de migration avec le trafic utilisateur ou le trafic de stockage.

Dans la configuration du datacenter Proxmox, les paramètres de migration ont été ajustés pour utiliser un réseau spécifique et garder un niveau de sécurité acceptable. Une migration manuelle de test a ensuite été réalisée depuis l'interface web afin de vérifier le déplacement transparent d'une VM d'un nœud à un autre sans interruption notable du service. Les figures IV.11 et IV.12 illustrent respectivement l'état avant et après cette opération.

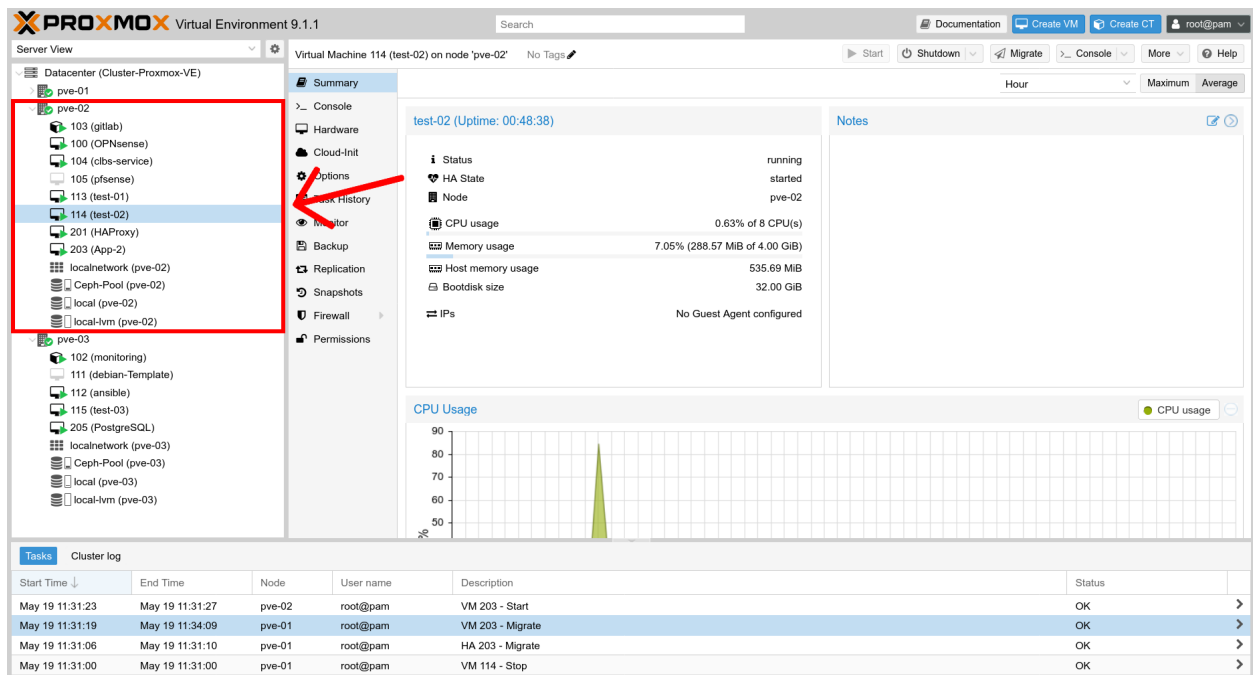


FIGURE IV.11 – Avant la migration live d’une machine virtuelle.

La Figure IV.11 montre l’état de la VM avant la migration, hébergée sur le nœud PVE-02. On peut observer les métriques de consommation CPU et mémoire ainsi que le nœud d’hébergement actuel dans le panneau de résumé.

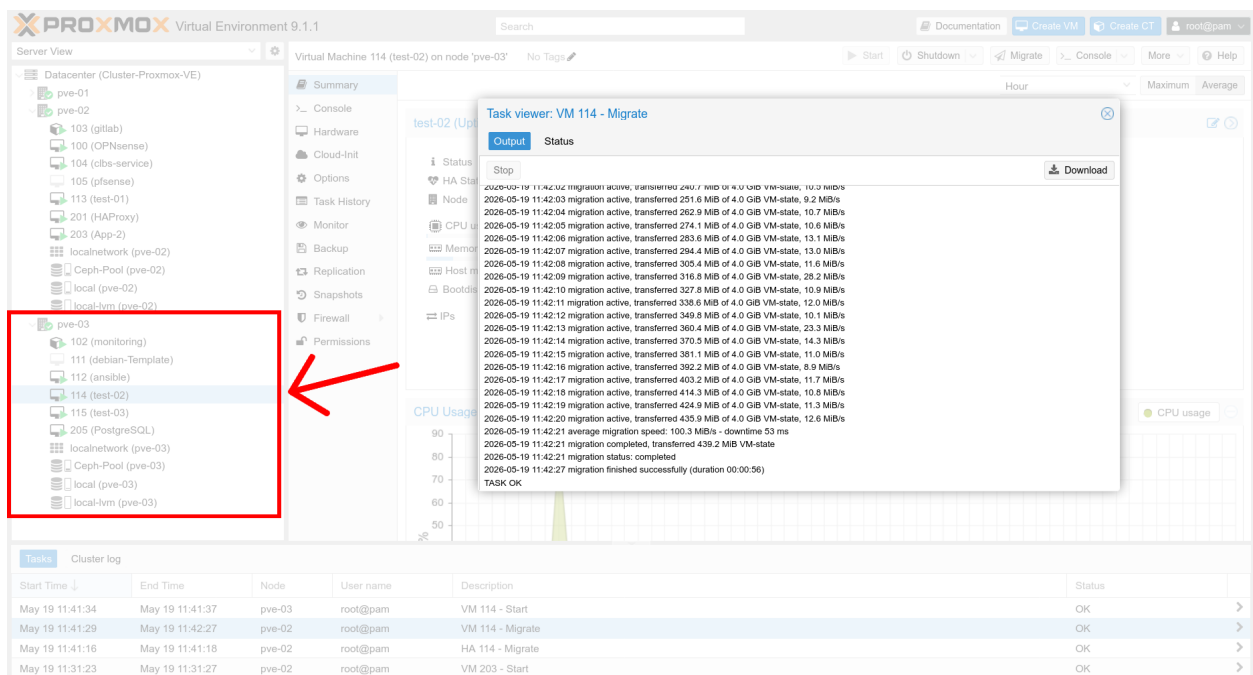


FIGURE IV.12 – Après la migration live d’une machine virtuelle.

La Figure IV.12 confirme que la VM a été déplacée avec succès vers le nœud PVE-03 sans interruption du service. Le journal de tâches affiche les étapes successives de la migration, incluant le transfert de l’état mémoire et la commutation finale.

IV.2.3 Déploiement de la stack de monitoring

Afin de superviser l'état global de l'infrastructure, une stack de monitoring a été déployée dans un conteneur LXC dédié appelé `monitoring`, avec l'adresse `192.168.10.99`. Ce conteneur centralise InfluxDB, Prometheus et Grafana.

IV.2.3.1 Déploiement d'InfluxDB

InfluxDB a été installé pour recevoir les métriques envoyées directement par Proxmox à travers le système natif “Metric Server”. Un bucket nommé `proxmox-metrics` a été créé afin de stocker les mesures CPU, mémoire, réseau et stockage de chaque nœud du cluster.

Une fois la configuration validée dans l'interface web Proxmox, les métriques ont commencé à être transmises automatiquement sans agent supplémentaire. Comme le montre la Figure IV.13, la présence des séries temporelles dans l'interface InfluxDB a confirmé le bon fonctionnement de ce mécanisme.



FIGURE IV.13 – Explorateur InfluxDB montrant les métriques reçues depuis Proxmox.

La Figure IV.13 présente l'interface Data Explorer d'InfluxDB avec les séries temporelles reçues depuis Proxmox. On y distingue les différentes mesures disponibles (CPU, mémoire, réseau) regroupées par nœud dans le bucket `proxmox-metrics`, confirmant que le pipeline de collecte fonctionne correctement.

IV.2.3.2 Déploiement de Prometheus

Prometheus a été installé dans le même conteneur afin de collecter les métriques exposées par Ceph et par le service applicatif CLBS. La configuration du fichier `prometheus.yml` contient deux jobs principaux : un job pour les métriques Ceph via le module MGR Prometheus, et un job pour l'application CLBS développée en Go.

Avant de configurer Prometheus, le module MGR Prometheus de Ceph a été activé sur les nœuds PVE-02 et PVE-03 via la commande suivante, exécutée une seule fois sur le cluster :

```
ceph mgr module enable prometheus
```

Figure IV.14 – Activation du module Prometheus sur les nœuds Ceph

Cette commande expose les métriques du cluster Ceph sur le port 9283 de chaque nœud hébergeant un MGR actif, permettant à Prometheus de les récupérer sans configuration supplémentaire côté Ceph.

```
global:
  scrape_interval: 30s
  evaluation_interval: 30s

scrape_configs:
  - job_name: ceph
    static_configs:
      - targets:
        - "192.168.10.213:9283"
        - "192.168.10.83:9283"
        - "192.168.10.100:9283"
      labels:
        cluster: "proxmox-lab"

  - job_name: "clbs"
    static_configs:
      - targets: ["192.168.10.104:9101"]
        scrape_interval: 30s
        scrape_timeout: 10s
```

Figure IV.15 – Extrait du fichier prometheus.yml

la Figure IV.15 définit deux jobs de collecte : le premier interroge les endpoints Ceph MGR des trois nœuds sur le port 9283, le second scrute le service CLBS sur le port 9101. L'intervalle de 30 secondes constitue un bon compromis entre fraîcheur des données et charge générée sur les cibles.

Après déploiement, l'interface web Prometheus a permis de confirmer que l'ensemble des cibles déclarées apparaissaient avec l'état UP, comme l'illustre la Figure IV.16.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.10.100:9283/metrics	DOWN	cluster="proxmox-lab" instance="192.168.10.100:9283" job="ceph"	27.954s ago	1.058ms	Get "http://192.168.10.100:9283/metrics": dial tcp 192.168.10.100:9283: connect: connection refused
http://192.168.10.213:9283/metrics	UP	cluster="proxmox-lab" instance="192.168.10.213:9283" job="ceph"	20.83s ago	12.44ms	
http://192.168.10.84:9283/metrics	UP	cluster="proxmox-lab" instance="192.168.10.84:9283" job="ceph"	8.625s ago	8.615ms	
clbs (1/1 up)					
http://192.168.10.104:9101/metrics	UP	instance="192.168.10.104:9101" job="clbs"	10.206s ago	3.583ms	
node (1/1 up)					
http://localhost:9100/metrics	UP	instance="localhost:9100" job="node"	4.766s ago	437ms	
prometheus (1/1 up)					
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus"	625ms ago	15.64ms	

FIGURE IV.16 – Interface Prometheus montrant les cibles Ceph et CLBS avec l'état UP.

La Figure IV.16 présente la page “Targets” de l’interface web Prometheus. On observe que deux des trois endpoints Ceph MGR affichent l’état UP (PVE-02 et PVE-03), tandis que celui de PVE-01 apparaît en état DOWN. Ce comportement est tout à fait normal : le module MGR Prometheus n’expose ses métriques sur le port 9283 que lorsque le manager Ceph est en rôle *actif* sur le nœud concerné. PVE-01 héberge un MGR en mode *standby*, c’est-à-dire qu’il ne devient actif qu’en cas de défaillance simultanée des managers de PVE-02 et PVE-03 ; en fonctionnement nominal, son endpoint reste donc inaccessible. Le service CLBS, quant à lui, affiche bien l’état UP, confirmant que la collecte des métriques applicatives est opérationnelle.

IV.2.3.3 Déploiement de Grafana

Grafana a été ajouté comme couche de visualisation unifiée pour les métriques de Proxmox, Ceph et CLBS. Deux sources de données ont été configurées : InfluxDB pour les métriques de Proxmox, et Prometheus pour les métriques Ceph et applicatives. Le tableau IV.1 récapitule les tableaux de bord configurés ainsi que leur source de données respective.

TABLEAU IV.1 – Tableaux de bord Grafana configurés.

Tableau de bord	Source	Métriques principales
Proxmox Cluster	InfluxDB	CPU, RAM, réseau par nœud
Ceph	Prometheus	Santé cluster, pools, latence OSD
CLBS	Prometheus	Historique des migrations

Trois dashboards principaux ont ainsi été utilisés : un dashboard infrastructure Proxmox, un dashboard Ceph, et un dashboard personnalisé pour le service CLBS. La Figure IV.17 présente le dashboard dédié à l’infrastructure Proxmox, tandis que la Figure IV.18 illustre celui consacré au cluster Ceph.

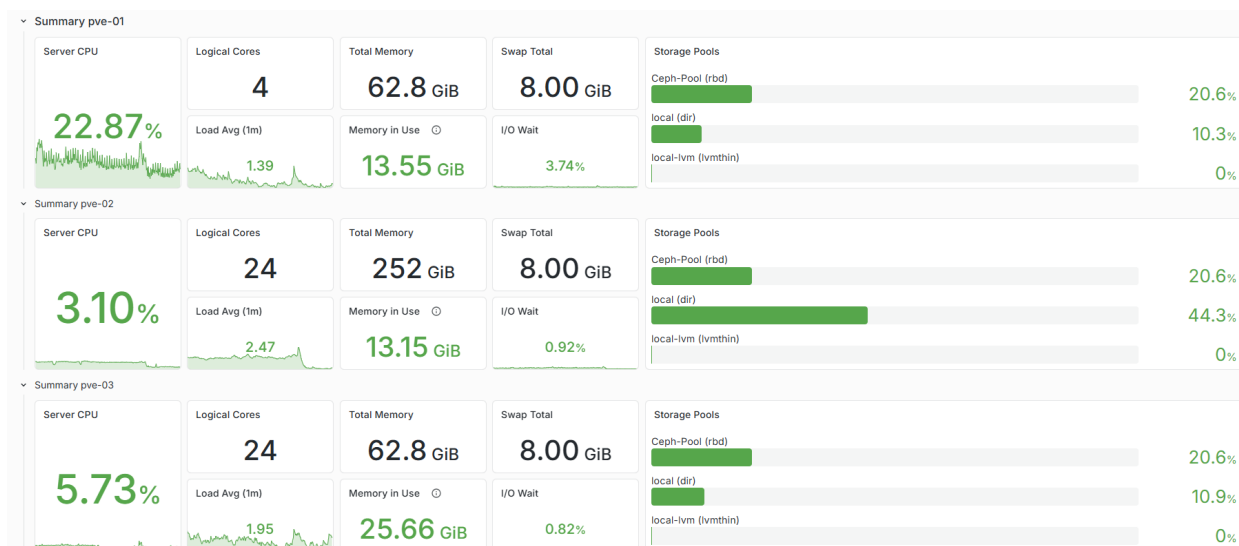


FIGURE IV.17 – Dashboard Grafana Infrastructure pour les nœuds Proxmox.

La Figure IV.17 présente le dashboard Grafana dédié à l'infrastructure Proxmox. Il affiche en temps réel les métriques CPU, mémoire et charge des trois nœuds, ainsi qu'un récapitulatif des VMs en cours d'exécution et des ressources disponibles dans le cluster. La vue complète du dashboard est disponible en Annexe A.

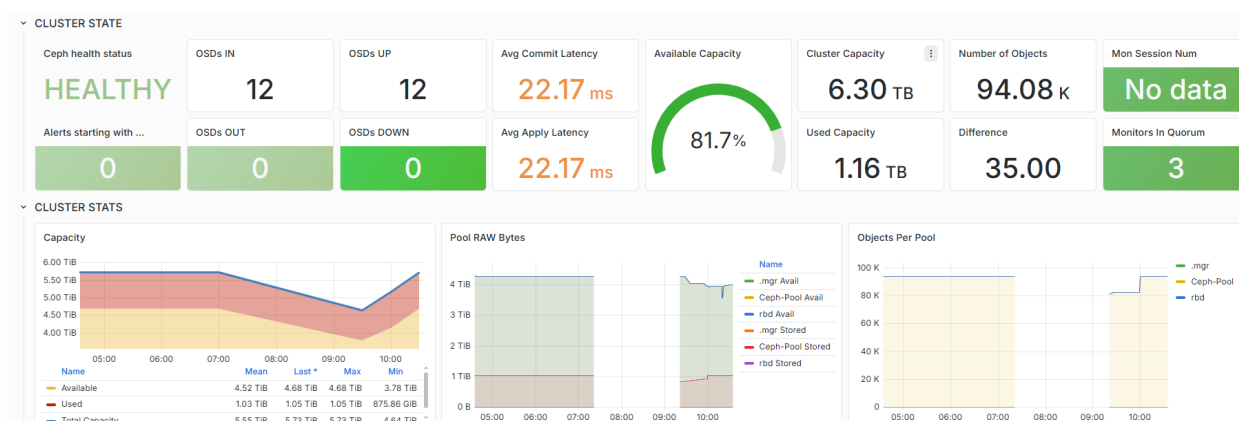


FIGURE IV.18 – Dashboard Grafana Ceph affichant l'état du cluster, des pools et la latence des OSDs.

La Figure IV.18 présente le dashboard Grafana dédié au cluster Ceph. On y retrouve l'état global de santé du cluster, les métriques I/O par pool, la latence des OSDs et le taux de réplication en temps réel, permettant de superviser en continu la couche stockage depuis la même interface unifiée. La vue complète du dashboard est disponible en Annexe A.

IV.2.4 Implémentation du mécanisme CLBS

Le mécanisme CLBS constitue l'une des contributions principales de ce projet. Il s'agit d'un service d'équilibrage proactif développé spécifiquement pour pallier l'absence de DRS natif dans Proxmox VE.

IV.2.4.1 Structure du service

Le service CLBS (“Cluster Load Balancing Service”) a été développé en langage Go dans une VM dédiée accessible sur le réseau de management. Son rôle est d’analyser périodiquement la charge CPU du cluster et de déclencher automatiquement des migrations live lorsque cela est nécessaire.

Le code est structuré en plusieurs composants : un lecteur InfluxDB pour les métriques historiques, un client API Proxmox pour piloter le cluster, un calculateur de bandes de charge, et un moteur principal de décision. Les paramètres de connexion sont placés dans un fichier `.env` chargé au démarrage du service.

IV.2.4.2 Logique de décision

À chaque cycle, le service récupère la charge des nœuds, calcule les bornes de charge modérée, puis classe les nœuds dans les catégories “heavy”, “moderate” ou “light”. Si un déséquilibre est détecté entre un nœud surchargé et un nœud léger, le moteur choisit la VM la plus adaptée à migrer.

Plusieurs mécanismes de protection ont été ajoutés pour éviter les migrations excessives, comme un temps de cooldown global, un cooldown individuel par VM et une suspension temporaire en cas de fréquence de migration trop élevée.

IV.2.4.3 Déploiement comme service systemd

Une fois compilé, le binaire Go a été déployé sur la VM CLBS puis intégré comme service `systemd` afin d’assurer un démarrage automatique et une relance en cas de panne.

```
systemctl enable clbs
systemctl start clbs
journalctl -u clbs.service -f -o cat
```

Figure IV.19 – Gestion du service CLBS avec systemd

Les trois commandes de la Figure IV.19 activent le démarrage automatique du service au boot, le démarrent immédiatement, et affichent les journaux en temps réel. Comme le montre la Figure IV.20, ces journaux permettent de suivre en direct les décisions de migration et les éventuelles erreurs de connexion à l’API Proxmox ou à InfluxDB.

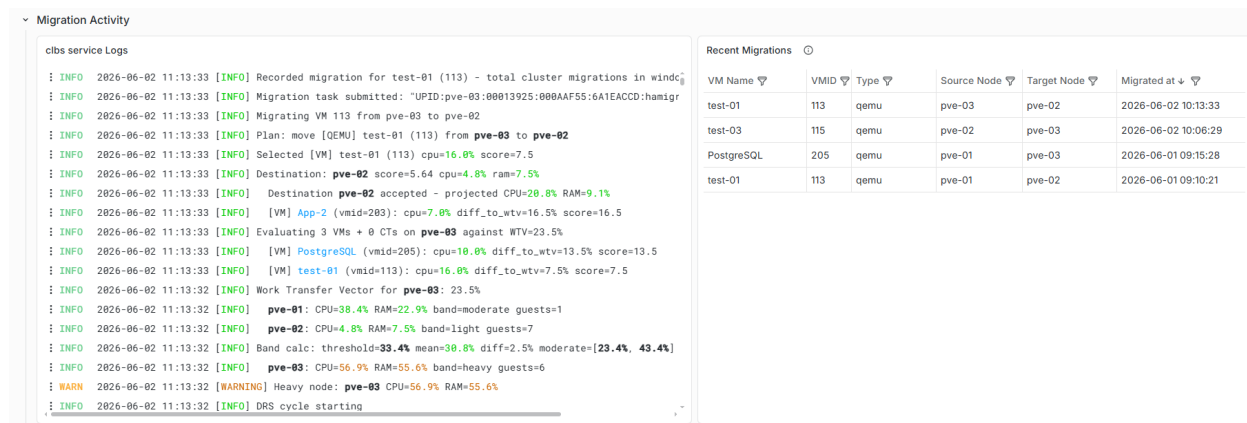


FIGURE IV.20 – Journaux systemd du service CLBS lors d’un cycle de détection et de migration.

La Figure IV.20 présente la sortie de la commande `journalctl` pour le service CLBS pendant un cycle de détection actif. On y observe la classification des nœuds par bande de charge (heavy/moderate/light), la sélection de la VM candidate à la migration, le déclenchement de l’opération via l’API Proxmox et l’activation du cooldown post-migration.

IV.2.4.4 Validation du mécanisme

Pour valider le comportement du service, deux VMs de test ont été créées puis soumises à une charge CPU artificielle avec l’outil `stress-ng`. Le service CLBS a détecté la surcharge et a déclenché une migration live vers un nœud moins chargé.

```
stress-ng --cpu 8 --timeout 10m
```

Figure IV.21 – Génération d’une charge CPU de test

La commande de la Figure IV.21 lance 8 workers saturant les cœurs CPU disponibles. Cette surcharge artificielle déclenche la logique de détection du service CLBS qui, après vérification des seuils, sélectionne la VM candidate et initie sa migration vers un nœud moins chargé. Comme l’illustre la Figure IV.22, le résultat est directement visible dans le dashboard dédié.

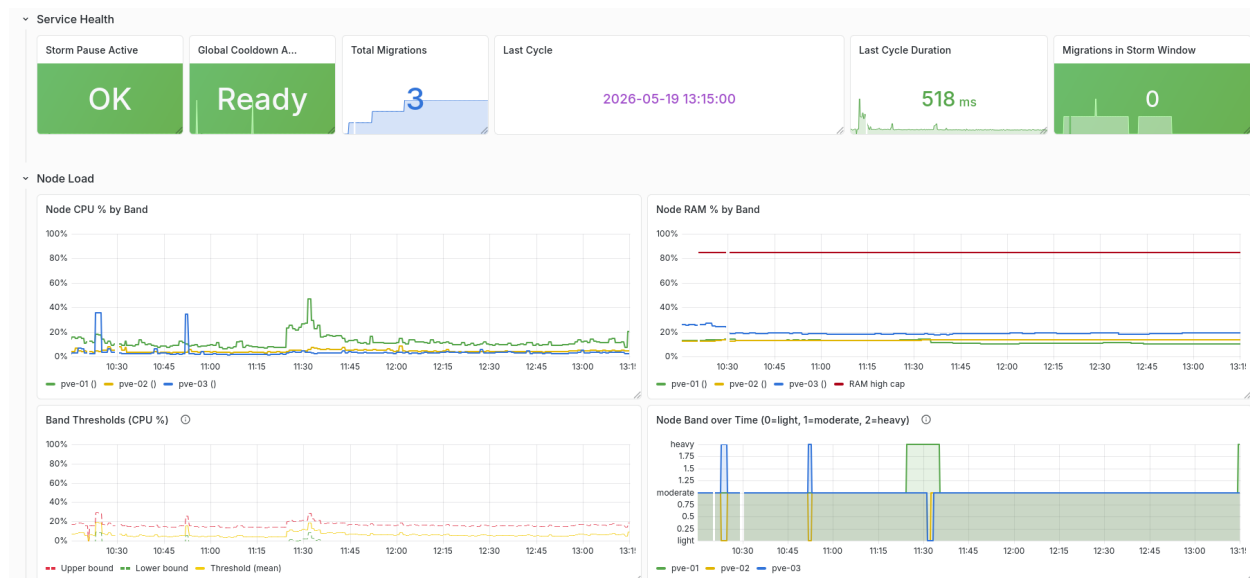


FIGURE IV.22 – Dashboard CLBS montrant une migration déclenchée automatiquement.

La Figure IV.22 présente le dashboard Grafana du service CLBS après le déclenchement d’une migration automatique. On y observe l’historique de charge des nœuds par bande (heavy/moderate/light), la migration enregistrée dans la section “Migration Activity”, ainsi que le cooldown restant avant le prochain cycle d’évaluation.

IV.2.5 Déploiement de l’application Sona-Web

L’application Sona-Web a été déployée sur l’infrastructure pour démontrer concrètement les capacités de haute disponibilité et de répartition de charge offertes par la plateforme hyperconvergée.

IV.2.5.1 Création des machines virtuelles

L’application Sona-Web a été déployée sous la forme de plusieurs VMs créées à partir d’un template Debian stocké dans le pool Ceph. Cette architecture comprend une VM HA-Proxy en frontal, deux VMs Flask pour la couche applicative, et une VM PostgreSQL pour la base de données.

Les VMs ont été déployées sur les réseaux internes appropriés, avec attribution d’adresse IP par DHCP selon la zone de sécurité visée. Cette organisation permet de séparer la partie exposée, la partie applicative et la base de données.

IV.2.5.2 Résultats de l’application Sona-Web

Afin de présenter concrètement les fonctionnalités de l’application déployée, des captures de l’interface Sona-Web ont été réalisées en conditions réelles d’utilisation. La Figure IV.23 en présente le tableau de bord principal.

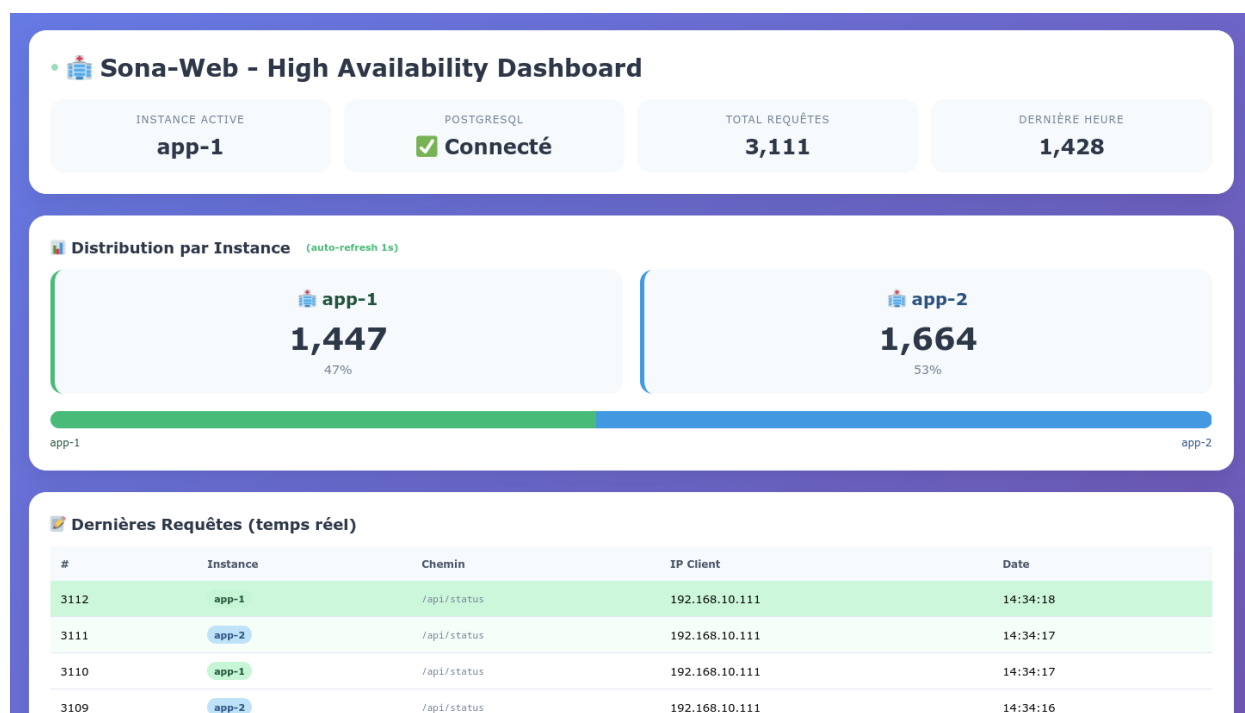


FIGURE IV.23 – Interface principale de l'application Sona-Web.

La Figure IV.23 présente le tableau de bord principal de Sona-Web, accessible via HA-Proxy sur le port 8080. L'interface affiche en temps réel l'instance Flask ayant traité la requête courante, le nombre total de requêtes enregistrées en base de données, ainsi que l'état de disponibilité des deux serveurs applicatifs.

IV.2.6 Automatisation avec Ansible

L'automatisation est un élément clé de la reproductibilité de l'infrastructure. Les playbooks Ansible développés permettent de provisionner de nouvelles VMs en quelques minutes et d'assurer leur intégration complète dans le cluster sans intervention manuelle.

IV.2.6.1 Environnement et structure du projet

Afin de simplifier le provisionnement des machines virtuelles et de rendre l'infrastructure plus reproductible, un environnement Ansible a été mis en place. Les connexions se font via SSH depuis les postes des développeurs vers le réseau de management, sans installation d'agent sur les nœuds cibles.

```

ansible-infra/
├── ansible.cfg
├── secrets.yml
├── inventory/
│   ├── hosts.yml
│   └── proxmox_nodes.yml
├── playbooks/
│   ├── bootstrap.yml
│   ├── ntp_sync.yml
│   ├── provision_vm.yml
│   └── provision_vm_batch.yml

```

Figure IV.24 – Structure du projet Ansible

L'arborescence de la Figure IV.24 organise le projet en trois parties : les fichiers de configuration globaux à la racine, le répertoire `inventory/` définissant les hôtes cibles, et le répertoire `playbooks/` contenant les scripts d'automatisation. Le fichier `secrets.yml` est chiffré avec Ansible Vault pour protéger les identifiants sensibles.

```
[defaults]
inventory          = inventory/hosts.yml
remote_user        = ansible
private_key_file    = ~/.ssh/ansible_ed25519
host_key_checking   = False
interpreter_python   = /usr/bin/python3

[privilege_escalation]
become              = True
become_user         = root
```

Figure IV.25 – Fichier `ansible.cfg`

Le fichier de configuration de la Figure IV.25 définit l'utilisateur distant `ansible`, la clé SSH privée `Ed25519` et l'élévation automatique de privilèges vers `root`. La désactivation de `host_key_checking` simplifie le déploiement en environnement de laboratoire où les hôtes changent fréquemment.

IV.2.6.2 Inventaire

L'inventaire regroupe les nœuds Proxmox, les VMs permanentes administrées manuellement, ainsi qu'un groupe dynamique destiné aux nouvelles VMs provisionnées à chaud par les playbooks.

```
all:
  children:
    proxmox_nodes:
      hosts:
        pve-01: {ansible_host: 192.168.10.100}
        pve-02: {ansible_host: 192.168.10.213}
        pve-03: {ansible_host: 192.168.10.84}
    managed_vms:
      hosts:
        ha-proxy: {ansible_host: 192.168.10.111}
        app-1:     {ansible_host: 192.168.10.112}
        postgres: {ansible_host: 192.168.10.115}
    provisioned_vms:
      hosts: {}
```

Figure IV.26 – Extrait de l'inventaire `hosts.yml`

L'inventaire de la Figure IV.26 est structuré en trois groupes : `proxmox_nodes` pour les nœuds physiques, `managed_vms` pour les VMs permanentes, et `provisioned_vms` initialement vide, alimenté dynamiquement via `add_host` au moment de la création de nouvelles

machines.

IV.2.6.3 Playbook de provisionnement

Le playbook `provision_vm.yml` automatise la création d'une VM à partir d'un template Debian stocké dans le pool Ceph. Il configure les ressources matérielles, le réseau via cloud-init, démarre la VM, puis la valide après boot.

```
ansible-playbook playbooks/provision_vm.yml \
  -e "vm_id=110 vm_name=web-01"

ansible-playbook playbooks/provision_vm.yml \
  -e "vm_id=115 vm_name=db-01 vm_cores=4 vm_memory=8192"
```

Figure IV.27 – Exemples d'utilisation du playbook `provision_vm.yml`

Les deux exemples de la Figure IV.27 montrent la flexibilité du playbook : le premier crée une VM web avec les ressources par défaut, le second provisionne une VM base de données avec 4 vCPUs et 8 Go de RAM. L'adresse IP est calculée automatiquement à partir du VMID.

```
vm_ip: "172.16.60.{{ vm_id | int - 90 }}"
```

Figure IV.28 – Formule d'attribution automatique de l'adresse IP

La formule de la Figure IV.28 génère automatiquement une adresse IP cohérente à partir du VMID, évitant les conflits d'adressage. Par exemple, la VM avec l'identifiant 110 reçoit l'adresse 172.16.60.20.

La configuration cloud-init repose sur trois commandes successives : attachement du disque cloud-init, injection de la configuration réseau, puis régénération de l'ISO cloud-init.

```
qm set {{ vm_id }} --ide2 {{ ceph_pool }}:cloudinit
qm set {{ vm_id }} --ipconfig0 ip={{ vm_cidr }},gw={{ vm_gw }} \
    --nameserver {{ vm_dns }}
qm cloudinit update {{ vm_id }}
```

Figure IV.29 – Configuration cloud-init d'une VM Proxmox

Les trois commandes de la Figure IV.29 attachent le disque cloud-init, injectent la configuration réseau (adresse IP, passerelle, DNS) et régénèrent l'ISO afin que la VM applique ces paramètres dès son premier démarrage. Un second play se connecte ensuite directement à la VM pour vérifier l'adresse IP configurée et effectuer une mise à jour complète des paquets.

IV.2.6.4 Playbook de bootstrap

Une VM fraîchement créée ne possède pas encore l'utilisateur `ansible` ni sa clé SSH : elle ne peut donc pas être prise en charge par les autres playbooks. Le playbook `bootstrap.yml` résout ce problème en prenant l'adresse IP de la cible en paramètre, sans qu'elle figure préalablement dans l'inventaire.

```
ansible-playbook playbooks/bootstrap.yml \
-e "target_hosts=192.168.10.120" \
-e "bootstrap_password=ROOT_PASS"
```

Figure IV.30 – Exemple d'utilisation du playbook bootstrap.yml

La commande de la Figure IV.30 montre comment appeler ce playbook. Celui-ci se connecte en `root` avec le mot de passe fourni, installe `python3` et `sudo`, crée l'utilisateur `ansible` avec les droits `sudo` sans mot de passe, puis déploie la clé publique SSH de gestion. À l'issue de ce playbook, la VM est prête à être ajoutée à l'inventaire et prise en charge par les autres playbooks.

IV.2.6.5 Playbook de synchronisation NTP

La cohérence temporelle entre les nœuds du cluster est indispensable pour Ceph et Corosync. Le playbook `ntp_sync.yml` configure `chrony` sur l'ensemble du parc en deux plays distincts.

Le premier play désigne `pve-01` comme maître NTP interne : il synchronise son horloge depuis `pool.ntp.org` et la distribue aux sous-réseaux du cluster (`192.168.0.0/16` et `172.16.0.0/12`). Le second play configure tous les autres nœuds en clients, avec `pve-01` comme source préférée et `pool.ntp.org` en secours. Chaque play se termine par un appel à `chronyc tracking` pour vérifier l'état de synchronisation.

```
# Configurer tout le cluster
ansible-playbook playbooks/ntp_sync.yml

# Reconfigurer uniquement pve-02
ansible-playbook playbooks/ntp_sync.yml --limit "pve-02"

# Configurer des n uds sp cifiques uniquement
ansible-playbook playbooks/ntp_sync.yml --limit "app-1,postgres"
```

Figure IV.31 – Exemples d'utilisation du playbook ntp_sync.yml

Les exemples de la Figure IV.31 illustrent la flexibilité du playbook : la première invocation configure l'ensemble du cluster, la deuxième reconfigure uniquement PVE-02, et la troisième applique la configuration NTP à des nœuds spécifiques de l'inventaire sans impacter le reste du parc.

IV.2.6.6 Provisionnement en lot de VMs

Le playbook `provision_vm_batch.yml` permet le provisionnement en lot de plusieurs VMs à partir d'une liste de définitions YAML. Il itère sur cette liste via `include_playbook` et invoque `provision_vm.yml` pour chaque entrée, en propageant les paramètres propres à chaque VM (identifiant, nom, ressources).

```
vms:
- id: 110
  name: web-01
```

```
- id: 111
  name: db-01
  cores: 4
  memory: 8192
```

Figure IV.32 – Exemple de liste de VMs pour `provision_vm_batch.yml`

L'exemple de la Figure IV.32 illustre le provisionnement en lot de plusieurs machines virtuelles à partir d'une liste YAML. Cette automatisation porte sur le provisionnement des machines virtuelles au-dessus d'un cluster déjà opérationnel. L'extension de cette même logique à l'amorçage des nœuds physiques eux-mêmes, c'est-à-dire l'installation initiale du système d'exploitation via un pipeline PXE/TFTP, a fait l'objet d'une conception détaillée dans le chapitre précédent. Sa mise en œuvre nécessitant toutefois du matériel serveur dédié, indisponible dans le cadre de ce projet, sa validation expérimentale est présentée parmi les perspectives d'évolution de l'infrastructure.

IV.3 Partie II : Couche Stockage

La couche stockage repose entièrement sur Ceph, déployé directement en intégration native avec Proxmox VE. Cette section présente le déploiement du cluster Ceph et la validation de sa tolérance aux pannes.

IV.3.1 Déploiement du cluster Ceph

Le déploiement du cluster Ceph suit une séquence bien définie : initialisation du réseau cluster, création des monitors et managers, ajout des OSDs, puis configuration du pool de stockage utilisé par les VMs.

IV.3.1.1 Initialisation de Ceph

La couche stockage repose sur un cluster Ceph intégré directement à Proxmox. L'initialisation a été effectuée depuis l'interface web ou via la ligne de commande, en définissant un réseau cluster dédié et un réseau public séparé.

```
pveceph init --network 10.10.30.0/24
```

Figure IV.33 – Initialisation du cluster Ceph

La commande de la Figure IV.33 initialise Ceph en spécifiant le réseau dédié aux échanges internes entre OSDs (10.10.30.0/24), transporté sur le VxLAN 30. Ce réseau est physiquement séparé du réseau public Ceph (10.10.40.0/24) pour isoler le trafic de réplication interne.

Le réseau 10.10.30.0/24 a été réservé aux échanges internes Ceph, tandis que le réseau 10.10.40.0/24 a été utilisé comme réseau public.

IV.3.1.2 Déploiement des MON et MGR

Un moniteur Ceph a été créé sur chaque nœud du cluster afin d'assurer le quorum et la haute disponibilité des métadonnées du cluster.

```
pveceph mon create
```

Figure IV.34 – Création d'un monitor Ceph

La commande de la Figure IV.34 est exécutée successivement sur chacun des trois nœuds. Avec trois monitors, le quorum Ceph est atteint dès que deux d'entre eux sont opérationnels, garantissant la disponibilité même en cas de panne d'un nœud.

Des managers Ceph ont aussi été déployés pour fournir les fonctions d'administration avancée et l'export des métriques vers Prometheus. Comme le montre la Figure IV.35, l'interface Proxmox permet de vérifier l'état de l'ensemble des monitors et managers déployés.

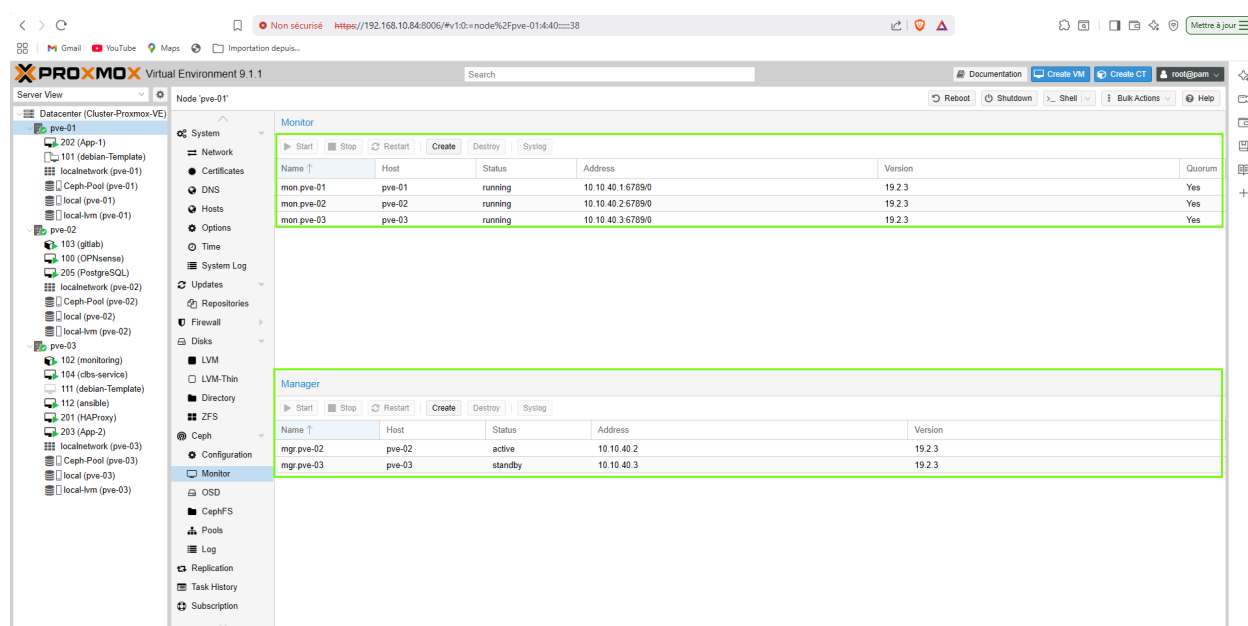


FIGURE IV.35 – Interface Proxmox affichant les monitors et managers Ceph actifs sur les trois nœuds.

La Figure IV.35 présente la vue Ceph de l'interface Proxmox après le déploiement des monitors et managers. Les trois MON apparaissent avec l'état **online** et le quorum est atteint, tandis que les MGR affichent leur rôle actif ou en veille selon le nœud, confirmant la redondance complète de la couche de contrôle Ceph.

```
ceph mgr module enable prometheus
```

Figure IV.36 – Activation du module Prometheus pour Ceph

L'activation du module via la Figure IV.36 expose les métriques du cluster (état des OSDs, pools, latence, I/O) sur le port 9283 de chaque nœud, permettant à Prometheus de les collecter sans configuration supplémentaire côté Ceph.

IV.3.1.3 Déploiement des OSDs

Sur chaque nœud, les quatre disques restants ont été intégrés comme OSDs Ceph. Le disque `/dev/sdb` (300 Go SAS) et les trois disques `/dev/sdc`, `/dev/sdd` et `/dev/sde` (600 Go SAS chacun) ont été provisionnés successivement via la commande suivante, exécutée pour

chacun d'eux.

```
pveceph osd create /dev/sdb
pveceph osd create /dev/sdc
pveceph osd create /dev/sdd
pveceph osd create /dev/sde
```

Figure IV.37 – Création des OSDs Ceph sur chaque nœud

Chaque appel à la Figure IV.37 formate le disque avec le système BlueStore et l'intègre comme OSD dans le cluster. La même séquence a été exécutée sur PVE-01, PVE-02 et PVE-03, portant le total à **douze OSDs** répartis équitablement sur les trois nœuds physiques.

Name	Class	OSD Type	Status	Version	weight	reweight	Used (%)	Total	Apply/Commit Latency (ms)	PGs
pve-03										
osd.11	hdd	bluestore	up / in	19.2.3	0.2728	1.00	20.31	279.39 GiB	15 / 15	29
osd.10	hdd	bluestore	up / in	19.2.3	0.54579	1.00	20.63	558.91 GiB	15 / 15	59
osd.8	hdd	bluestore	up / in	19.2.3	0.54579	1.00	19.05	558.91 GiB	39 / 39	56
osd.0	hdd	bluestore	up / in	19.2.3	0.54579	1.00	14.95	558.91 GiB	35 / 35	49
pve-02										
osd.9	hdd	bluestore	up / in	19.2.3	0.54579	1.00	15.90	558.91 GiB	15 / 15	53
osd.7	hdd	bluestore	up / in	19.2.3	0.54579	1.00	19.11	558.91 GiB	10 / 10	54
osd.3	hdd	bluestore	up / in	19.2.3	0.54579	1.00	19.16	558.91 GiB	46 / 46	61
osd.2	hdd	bluestore	up / in	19.2.3	0.2728	1.00	19.92	279.39 GiB	17 / 17	25
pve-01										
osd.6	hdd	bluestore	up / in	19.2.3	0.54579	1.00	18.47	558.91 GiB	15 / 15	56
osd.5	hdd	bluestore	up / in	19.2.3	0.54579	1.00	17.44	558.91 GiB	34 / 34	51
osd.4	hdd	bluestore	up / in	19.2.3	0.54579	1.00	18.11	558.91 GiB	16 / 16	57
osd.1	hdd	bluestore	up / in	19.2.3	0.2728	1.00	20.35	279.39 GiB	13 / 13	29

FIGURE IV.38 – État du cluster Ceph montrant les douze OSDs actifs sur les trois nœuds.

La Figure IV.38 affiche la vue OSD de l'interface Proxmox. Les douze OSDs apparaissent avec le statut **up/in**, leur type de stockage BlueStore et leur poids dans la hiérarchie CRUSH, confirmant que l'ensemble des disques est actif et correctement intégré au cluster.

```
ceph osd tree
```

Figure IV.39 – Affichage de l'arbre CRUSH

La commande de la Figure IV.39 affiche la hiérarchie CRUSH avec les trois nœuds physiques et leurs quatre OSDs respectifs. Cette structure garantit que les trois répliques d'un même bloc seront toujours placées sur des serveurs physiques distincts, conformément à la politique de domaines de défaillance.

IV.3.1.4 Création du pool de stockage

Un pool RBD a été créé avec une répllication de taille 3 et une taille minimale de 2, ce qui garantit la disponibilité des données même en cas de perte d'un nœud.

```
pveceph pool create ceph-pool --size 3 --min-size 2
```

Figure IV.40 – Création du pool Ceph

Dans la Figure IV.40, le paramètre `-size 3` impose trois répliques par objet, une par nœud physique. Le paramètre `-min-size 2` maintient le pool accessible tant qu'au moins deux répliques sont disponibles, offrant une tolérance à la panne d'un nœud sans coupure de service.

Ce pool a ensuite été intégré comme stockage partagé dans Proxmox pour héberger les disques des VMs et des conteneurs.

```
ceph -s
ceph df
```

Figure IV.41 – État global du cluster Ceph

Les commandes de la Figure IV.41 permettent de vérifier l'état du cluster : `ceph -s` affiche un résumé de l'état de santé, tandis que `ceph df` donne la capacité totale, utilisée et disponible par pool. Ces deux commandes constituent la vérification de base après chaque modification de la configuration Ceph. La Figure IV.42 illustre l'état du pool obtenu après ce déploiement.

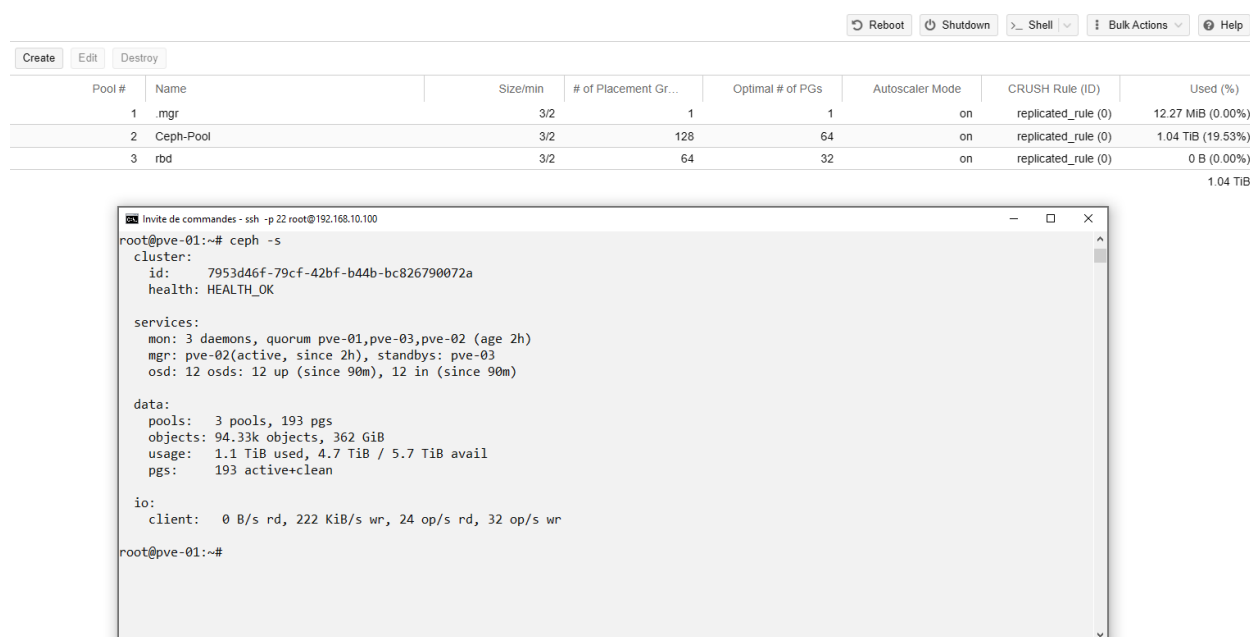


FIGURE IV.42 – État du cluster Ceph montrant le pool RBD.

La Figure IV.42 montre la vue des pools Ceph dans l'interface Proxmox. Le pool `ceph-pool` apparaît avec l'état `HEALTH_OK`, le facteur de réplication à 3 et les métriques d'I/O en temps réel, confirmant le bon fonctionnement de la couche stockage.

IV.3.2 Validation de la tolérance aux pannes

La tolérance aux pannes de la couche stockage a été vérifiée par un scénario de panne simulée. Lors de l’extinction d’un nœud, le cluster Ceph est passé temporairement à l’état `HEALTH_WARN`, mais les données sont restées accessibles grâce au facteur de réplication.

Après le retour du nœud défaillant, Ceph a automatiquement lancé la resynchronisation des données et l’état du cluster est revenu à `HEALTH_OK`. Ce test valide le bon comportement de la couche stockage distribuée.

IV.4 Partie III : Couche Réseau

La couche réseau est construite autour d’Open vSwitch et des tunnels VxLAN, permettant de créer plusieurs réseaux logiques isolés sur le même réseau physique Sonatrach. OPNsense assure le routage centralisé et le filtrage entre les différents segments.

IV.4.1 Configuration Open vSwitch et tunnels VxLAN

La configuration réseau s’appuie sur Open vSwitch pour la création des bridges et des tunnels VxLAN, et sur les fichiers d’interfaces Linux pour la persistance de la configuration.

IV.4.1.1 Installation d’Open vSwitch

Open vSwitch a été installé sur chaque nœud Proxmox afin de construire une couche réseau virtuelle flexible, adaptée au transport de plusieurs flux logiques sur le même réseau physique.

```
apt install openvswitch-switch
```

Figure IV.43 – Installation d’Open vSwitch

La commande de la Figure IV.43 installe le daemon OVS et les outils de gestion en ligne de commande (`ovs-vsctl`, `ovs-ofctl`). Elle doit être exécutée sur chacun des trois nœuds Proxmox avant toute configuration des bridges et des tunnels.

Un bridge principal a été créé sur chaque nœud, puis l’interface physique principale y a été attachée pour servir de point de transit à tous les tunnels VxLAN.

```
ovs-vsctl add-br vmbr0
ovs-vsctl add-port vmbr0 nic0
```

Figure IV.44 – Création du bridge OVS principal

Les commandes de la Figure IV.44 configurent le bridge principal : la première crée le bridge OVS `vmbr0`, la seconde y attache l’interface physique `nic0` comme port de transit. Tous les tunnels VxLAN créés ultérieurement utilisent ce bridge, permettant de transporter plusieurs réseaux logiques sur une seule interface physique.

IV.4.1.2 Configuration complète des interfaces (exemple sur PVE-01)

Le fichier `/etc/network/interfaces` est déployé de façon identique sur les trois nœuds ; seule l'adresse IP du bridge de management change d'un nœud à l'autre (192.168.10.100 pour PVE-01, 192.168.10.213 pour PVE-02, 192.168.10.84 pour PVE-03). La configuration présentée ici est donc celle de PVE-01 et sert d'exemple représentatif pour l'ensemble du cluster.

On pourrait se demander pourquoi chaque nœud doit explicitement lister les adresses IP des autres. C'est une contrainte inhérente à l'utilisation de tunnels VxLAN en mode point-à-point : chaque extrémité doit connaître les *remote endpoints* à l'avance, sans aucun mécanisme de découverte automatique. Si l'infrastructure devait migrer vers une architecture VLAN, cette limitation disparaîtrait : Proxmox intègre un module SDN (*Software Defined Networking*) qui permet de gérer la topologie réseau directement depuis l'interface web et de propager automatiquement la configuration sur tous les nœuds, sans toucher manuellement aux fichiers de configuration.

Pour en faciliter la lecture, la configuration est présentée en cinq parties logiques correspondant aux différents rôles des interfaces : la base physique, les réseaux d'infrastructure Ceph et Corosync, les réseaux de migration et de sauvegarde, puis les réseaux applicatifs et leurs bridges Linux. La Figure IV.45 ci-dessous offre une vue synthétique de cette organisation en quatre couches avant que chacune ne soit détaillée.

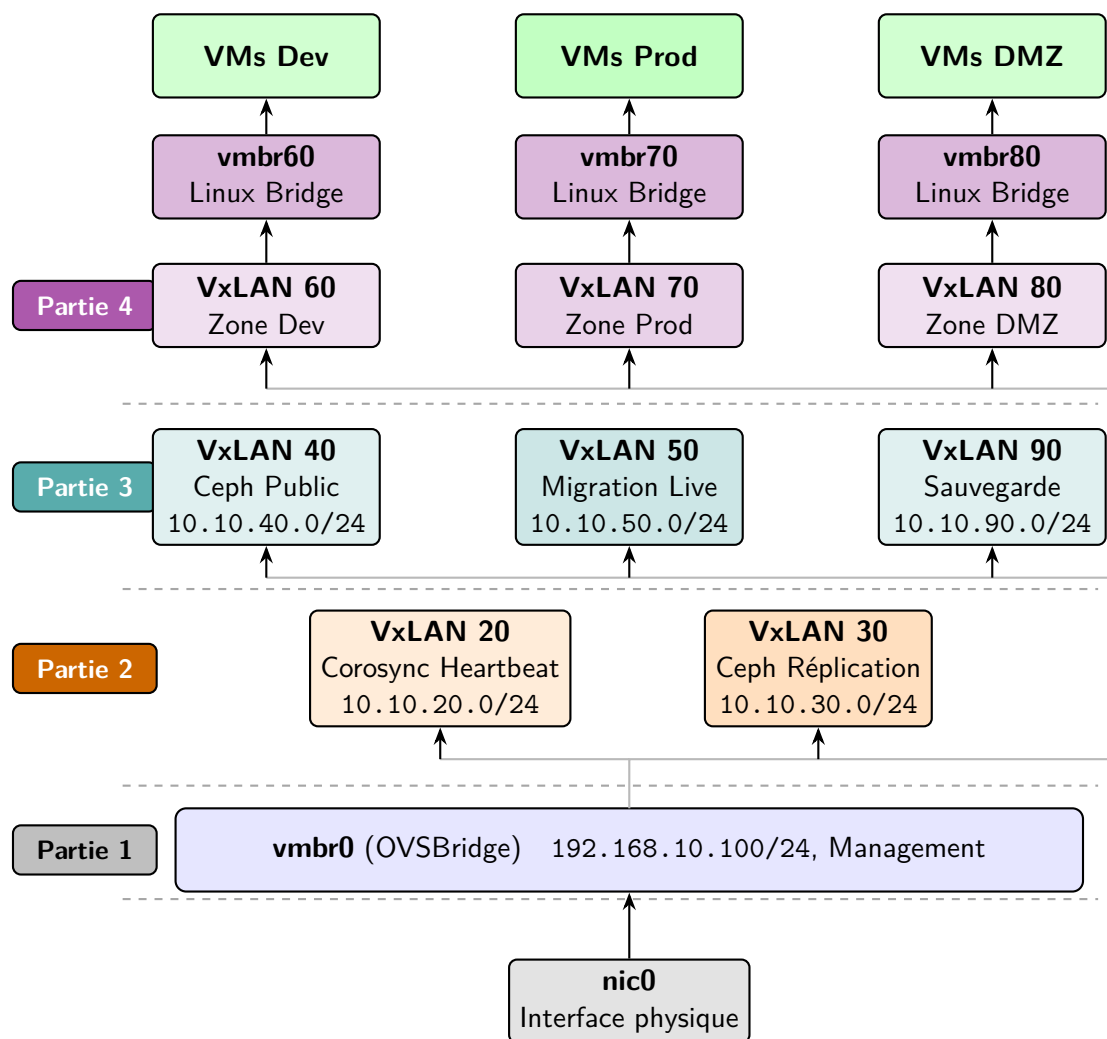


FIGURE IV.45 – Vue d'ensemble de la configuration réseau de PVE-01 : quatre parties logiques

La Figure IV.45 illustre l'organisation en quatre couches de la configuration réseau de PVE-01 : le bridge de management (Partie 1), les VxLAN d'infrastructure Corosync et Ceph interne (Partie 2), les VxLAN de migration, Ceph public et sauvegarde (Partie 3), et enfin les VxLAN applicatifs avec leurs bridges Linux (Partie 4).

Partie 1 : Interface physique et bridge de management Cette première partie définit l'interface physique `nic0` ainsi que le bridge OVS principal `vmbr0`, qui sert de point d'ancrage à l'ensemble des tunnels VxLAN. L'adresse de management `192.168.10.100/24` est attribuée directement sur ce bridge.

```
auto lo
iface lo inet loopback

iface nic0 inet manual

auto vmbr0
iface vmbr0 inet static
    address 192.168.10.100/24
    gateway 192.168.10.36
```

```
    ovs_type OVSBridge
    ovs_ports nic0

auto nic0
iface nic0 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0

iface nic1 inet manual
iface nic2 inet manual
iface nic3 inet manual
```

Figure IV.46 – Interfaces de base et bridge OVS de management

Dans la Figure IV.46, le bridge `vmbr0` est de type `OVSBridge` et encapsule `nic0` comme port physique de sortie. C'est sur ce même bridge que tous les tunnels VxLAN seront connectés, ce qui permet de les transporter sur l'unique interface physique disponible.

Partie 2 : VxLAN Corosync (clé 20) et stockage Ceph interne (clé 30) Ces deux VxLAN transportent respectivement le trafic de heartbeat Corosync entre les nœuds du cluster, et le trafic de réplication interne Ceph entre les OSDs. Chaque tunnel est créé en mode point-à-point, avec une interface interne OVS associée pour l'adressage IP local.

```
# ===== COROSYNC VXLAN (key=20) =====
auto vxlan20-int
iface vxlan20-int inet static
    address 10.10.20.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== CEPH INTERNE VXLAN (key=30) =====
auto vxlan30-int
iface vxlan30-int inet static
    address 10.10.30.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0
```

Figure IV.47 – VxLAN Corosync (clé 20) et Ceph interne (clé 30) – interfaces internes PVE-01

Dans la Figure IV.47, seules les interfaces internes (`OVSPort`) des VxLAN 20 et 30 sont représentées. Chaque interface interne porte l'adresse IP locale du nœud sur le réseau logique correspondant. Le VxLAN 20 isole le trafic Corosync du reste du réseau, garantissant que les heartbeats entre nœuds ne soient jamais perturbés par le trafic applicatif. Le VxLAN 30 isole quant à lui le trafic de réplication Ceph, dont la bande passante peut être importante lors de la resynchronisation d'un OSD. La configuration complète incluant les ports tunnels VxLAN vers PVE-02 et PVE-03, ainsi que les fichiers des deux autres nœuds, est disponible en Annexe B.

Partie 3 : Ceph public (clé 40), migration live (clé 50) et sauvegarde (clé 90) Ces trois VxLAN couvrent respectivement le réseau public Ceph utilisé pour les accès RBD des

VMs, le réseau dédié aux migrations live de VMs entre nœuds, et le réseau de sauvegarde. Leur séparation évite toute interférence entre ces flux lors des opérations intensives.

```
# ===== CEPH PUBLIC VXLAN (key=40) =====
auto vxlan40-int
iface vxlan40-int inet static
    address 10.10.40.1/24
    ovs_type OVSIntPort
    ovs_bridge vmbr0

# ===== MIGRATION LIVE VXLAN (key=50) =====
auto vxlan50-int
iface vxlan50-int inet static
    address 10.10.50.1/24
    ovs_type OVSIntPort
    ovs_bridge vmbr0

# ===== SAUVEGARDE VXLAN (key=90) =====
auto vxlan90-int
iface vxlan90-int inet static
    address 10.10.90.1/24
    ovs_type OVSIntPort
    ovs_bridge vmbr0
```

Figure IV.48 – Ceph public (clé 40), migration (clé 50) et sauvegarde (clé 90) – interfaces internes PVE-01

La Figure IV.48 présente les interfaces internes des VxLAN 40, 50 et 90 sur PVE-01. La séparation du réseau public Ceph (VxLAN 40) du réseau de réplication interne (VxLAN 30) suit les recommandations de Ceph : le trafic client et le trafic OSD-OSD n'entrent pas en contention. Le VxLAN 50, dédié aux migrations live, préserve la qualité de service lors des déplacements de VMs par le mécanisme CLBS. La configuration complète des trois nœuds est disponible en Annexe B.

Partie 4 : Réseaux applicatifs (clés 60, 70, 80) et bridges Linux Les trois derniers VxLAN portent le trafic des zones applicatives Dev (clé 60), Prod (clé 70) et DMZ (clé 80). Chacun dispose d'une interface interne OVS et d'un bridge Linux associé (`vmbr60`, `vmbr70`, `vmbr80`), qui constitue le point de connexion effectif pour les interfaces réseau des VMs applicatives.

```
# ===== VXLAN 60 (Dev) =====
auto vxlan60-int
iface vxlan60-int inet static
    ovs_type OVSIntPort
    ovs_bridge vmbr0

# ===== VXLAN 70 (Prod) =====
auto vxlan70-int
iface vxlan70-int inet static
    ovs_type OVSIntPort
    ovs_bridge vmbr0
```

```
# ===== VXLAN 80 (DMZ) =====
auto vxlan80-int
iface vxlan80-int inet static
    ovs_type OVSIntPort
    ovs_bridge vmbr0

# ===== VM Bridges =====
auto vmbr60
iface vmbr60 inet manual
    bridge_ports vxlan60-int
    bridge_stp off
    bridge_fd 0

auto vmbr70
iface vmbr70 inet manual
    bridge_ports vxlan70-int
    bridge_stp off
    bridge_fd 0

auto vmbr80
iface vmbr80 inet manual
    bridge_ports vxlan80-int
    bridge_stp off
    bridge_fd 0

source /etc/network/interfaces.d/*
```

Figure IV.49 – VxLAN applicatifs (clés 60, 70, 80) et bridges Linux – PVE-01

La Figure IV.49 présente les interfaces internes et les bridges Linux des réseaux applicatifs sur PVE-01. Les bridges `vmbr60`, `vmbr70` et `vmbr80` constituent la jonction entre le plan de données OVS et les interfaces réseau des VMs Proxmox. Chaque bridge est relié à l'interface interne OVS du VxLAN correspondant, formant un chemin de commutation continu depuis la VM applicative jusqu'à l'interface physique du nœud, à travers le tunnel VxLAN vers les autres nœuds du cluster. Cette architecture permet de transporter plusieurs réseaux logiques isolés sur le même réseau physique, sans dépendre d'un switch physique configuré en VLAN. La configuration complète des trois nœuds (avec les ports tunnels vers PVE-02 et PVE-03) est disponible en Annexe B.

IV.4.2 Déploiement de la VM OPNsense

OPNsense joue un rôle central dans l'architecture réseau en assurant le routage inter-réseaux, le NAT sortant et le service DHCP pour les trois zones applicatives.

IV.4.2.1 Création et interfaces réseau

OPNsense a été déployé sous forme de machine virtuelle sur le cluster Proxmox. Son disque système est stocké sur le pool Ceph, ce qui permet de bénéficier du stockage distribué et de la haute disponibilité.

La VM dispose de 4 Go de RAM, de 2 vCPUs et d'un disque de 20 Go. Quatre interfaces réseau lui ont été attribuées afin de jouer le rôle de routeur et pare-feu pour les différents segments de l'infrastructure. Le tableau IV.2 récapitule ces interfaces avec leur bridge Proxmox, leur rôle et leur adresse IP.

TABLEAU IV.2 – Interfaces de la VM OPNsense.

Interface VM	Bridge Proxmox	Rôle	Adresse IP
net0 (em0)	vmbr0	WAN	192.168.10.65/24
net1 (em1)	vmbr60	LAN Dev	172.16.60.1/24
net2 (em2)	vmbr70	LAN Prod	172.16.70.1/24
net3 (em3)	vmbr80	LAN DMZ	172.16.80.1/24

Comme le montre la Figure IV.50, la console de gestion OPNsense permet de vérifier l'assignation correcte de ces quatre interfaces.

```

Enter a host name or IP address: 8.8.8.8

PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: icmp_seq=0 ttl=111 time=79.301 ms
^C
*** OPNsense.internal: OPNsense 26.1.2_5 (amd64) ***

LAN (em1)      -> v4: 172.16.60.1/24
OPT1 (em2)     -> v4: 172.16.70.1/24
OPT2 (em3)     -> v4: 172.16.80.1/24
WAN (em0)      -> v4: 192.168.10.65/24

HTTPS: SHA256 DD D5 15 E5 28 38 7B A1 9B E2 1D 5B D4 9C 5D F5
              08 23 37 EC F7 A8 97 9D F6 CA CE 29 E9 BF 31 D5

0) Logout                      7) Ping host
1) Assign interfaces           8) Shell
2) Set interface IP address    9) pfTop
3) Reset the root password     10) Firewall log
4) Reset to factory defaults   11) Reload all services
5) Power off system            12) Update from console
6) Reboot system               13) Restore a backup

Enter an option:

```

FIGURE IV.50 – Configuration des interfaces de la VM OPNsense.

La Figure IV.50 montre la console de gestion OPNsense affichant les quatre interfaces réseau avec leurs adresses IP respectives, confirmant que le routeur virtuel est opérationnel avec ses interfaces WAN et LAN correctement assignées.

IV.4.2.2 NAT sortant

Le NAT sortant automatique a été activé dans OPNsense afin de permettre aux réseaux internes de sortir vers Internet via l'interface WAN. Cette configuration a simplifié le routage pour les VMs hébergées sur les réseaux Dev, Prod et DMZ.

IV.4.3 Configuration du service DHCP

Le service DHCP a été activé via `dnsmasq` sur les interfaces LAN internes. Ce choix a été retenu car il est simple à configurer et convient bien à un environnement de laboratoire. Le tableau IV.3 présente les plages d'adresses DHCP configurées pour chacun des réseaux internes.

TABLEAU IV.3 – Plages DHCP configurées sur OPNsense.

Interface	Plage début	Plage fin	Gateway
LAN (Dev)	172.16.60.10	172.16.60.250	172.16.60.1
OPT1 (Prod)	172.16.70.10	172.16.70.250	172.16.70.1
OPT2 (DMZ)	172.16.80.10	172.16.80.250	172.16.80.1

Le bon fonctionnement de DHCP a été vérifié par une capture réseau sur l'interface LAN d'OPNsense pendant le démarrage d'une VM de test.

```
tcpdump -i em1 port 67 or port 68
```

Figure IV.51 – Capture du trafic DHCP avec tcpdump

La commande de la Figure IV.51 capture les échanges DHCP sur l'interface `em1` d'OPNsense. La sortie affiche successivement les messages Discover, Offer, Request et Acknowledge, validant le cycle complet d'attribution d'adresse IP à la machine virtuelle de test.

Cette vérification a montré le cycle normal des échanges DHCP, depuis le “Discover” jusqu'au “Offer”, puis l'attribution effective de l'adresse IP à la machine virtuelle.

IV.4.4 Configuration du Pare-feu

La sécurité réseau de l'infrastructure repose sur deux niveaux de filtrage complémentaires et indépendants : le pare-feu OPNsense pour le contrôle des flux entre les zones applicatives, et le pare-feu natif Proxmox au niveau de l'hyperviseur pour protéger les réseaux d'infrastructure.

IV.4.4.1 Pare-feu OPNsense : Filtrage Inter-réseaux

OPNsense centralise le routage et applique les politiques de filtrage entre les réseaux applicatifs Dev (172.16.60.0/24), Prod (172.16.70.0/24) et DMZ (172.16.80.0/24). Les règles ont été configurées sous **Firewall** -> **Rules** de l'interface OPNsense selon le principe du moindre privilège : tout flux non explicitement autorisé est bloqué par défaut.

Le tableau IV.4 récapitule les règles de filtrage appliquées entre les zones et vers l'interface WAN.

TABLEAU IV.4 – Règles de filtrage inter-réseaux configurées sur OPNsense.

Source	Destination	Action	Justification
Dev (172.16.60.0/24)	WAN	ALLOW	Mises à jour et dépôts de paquets
DMZ (172.16.80.0/24)	WAN	ALLOW	Services exposés nécessitant un accès sortant
Prod (172.16.70.0/24)	DMZ	ALLOW	Communication production vers services exposés
Dev (172.16.60.0/24)	Prod	BLOCK	Isolation développement / production
Dev (172.16.60.0/24)	DMZ	BLOCK	Empêcher l'accès aux services exposés depuis Dev
Prod (172.16.70.0/24)	WAN	BLOCK	Le trafic Prod passe obligatoirement par la DMZ

Ces règles assurent une isolation stricte entre les trois zones tout en autorisant les flux légitimes vers Internet. La politique de sortie directe de Prod vers WAN est bloquée afin de forcer le passage par la DMZ, seule zone autorisée à exposer des services.

IV.4.4.2 Pare-feu Proxmox : Filtrage au Niveau Hyperviseur

En complément du filtrage OPNsense, le pare-feu natif Proxmox a été configuré au niveau de l'hyperviseur pour protéger les réseaux d'infrastructure et restreindre les accès à la plateforme de gestion. Ce pare-feu s'organise en trois niveaux hiérarchiques : le niveau datacenter pour les règles globales communes, le niveau nœud pour les règles propres à chaque serveur physique, et le niveau VM pour les règles spécifiques à chaque machine virtuelle.

Configuration au niveau datacenter

Des alias symboliques ont été définis pour simplifier l'écriture des règles. Le tableau IV.5 liste les alias créés, chacun associant un nom symbolique à l'adresse IP de l'instance correspondante.

TABLEAU IV.5 – Alias définis au niveau datacenter dans le pare-feu Proxmox.

Alias	Adresse IP	Instance
haproxy	172.16.80.10	VM HAProxy (frontal DMZ)
sona-web1	172.16.70.10	VM Flask app-1 (Prod)
sona-web2	172.16.70.11	VM Flask app-2 (Prod)
postgres	172.16.70.20	VM PostgreSQL (Prod)

Des IPSets ont également été créés pour regrouper les adresses appartenant à un même domaine de sécurité. Ces groupes centralisent les plages d'adresses des réseaux logiques et permettent d'appliquer une règle à un ensemble de machines sans dupliquer les adresses dans chaque règle individuelle. Le tableau IV.6 récapitule les IPSets configurés.

TABLEAU IV.6 – IPSets configurés au niveau datacenter.

Nom de l'IPSet	Membres	Utilisation
management-net	192.168.10.0/24	Accès administration
corosync-net	10.10.20.0/24	Heartbeat cluster
ceph-cluster-net	10.10.30.0/24	Réplication Ceph interne
ceph-public-net	10.10.40.0/24	Accès RBD public
migration-net	10.10.50.0/24	Migration live
vxlan-phys-net	192.168.10.0/24	Tunnels VxLAN

Cinq groupes de sécurité ont ensuite été créés pour encapsuler les règles de ports associées à chaque service d'infrastructure. Le tableau IV.7 décrit ces groupes, leurs règles de ports et les sources autorisées.

TABLEAU IV.7 – Groupes de sécurité Proxmox et leurs règles de ports par service.

Groupe	Protocole / Port	Source	Rôle
proxmox-management	TCP 22	192.168.10.0/24	Accès SSH
proxmox-management	TCP 8006	192.168.10.0/24	Interface web Proxmox
proxmox-management	ICMP	192.168.10.0/24	Diagnostics réseau
proxmox-management	TCP 9283	192.168.10.0/24	Métriques Ceph MGR (Prometheus)
corosync-cluster	UDP 5404–5405	10.10.20.0/24	Heartbeat Corosync
vxlan-overlay	UDP 4789	192.168.10.0/24	Tunnels VxLAN
ceph-storage	TCP 6800–7300	10.10.30.0/24	Communications OSD–OSD
ceph-storage	TCP 3300, 6789	10.10.30.0/24	Monitors Ceph
ceph-storage	TCP 6800–7300	10.10.40.0/24	Accès RBD public
migration-live	TCP any	10.10.50.0/24	Migrations live (ports dynamiques)

Cette approche par groupes de sécurité et IPSets garantit une politique de sécurité cohérente sur l'ensemble du cluster sans dupliquer les règles : toute modification d'un groupe se propage automatiquement sur les trois nœuds physiques.

Configuration au niveau nœud

Le deuxième niveau du pare-feu Proxmox s'applique directement à chaque nœud physique du cluster. La politique par défaut adoptée est **DROP** en entrée et **ACCEPT** en sortie. Sur chacun des trois nœuds PVE-01, PVE-02 et PVE-03, le pare-feu est activé et les cinq groupes de sécurité définis au niveau datacenter sont référencés. Cette approche hérite automatiquement de toutes les règles et IPSets définis centralement, sans nécessiter de redéfinition nœud par nœud.

Le tableau IV.8 résume les flux effectivement autorisés en entrée sur chaque nœud une fois les groupes de sécurité appliqués.

TABLEAU IV.8 – Flux autorisés au niveau nœud via les groupes de sécurité (PVE-01, PVE-02, PVE-03).

Groupe	Direction	Action	Source	Port / Protocole
proxmox-management	IN	ACCEPT	192.168.10.0/24	8006 TCP
proxmox-management	IN	ACCEPT	192.168.10.0/24	22 TCP
proxmox-management	IN	ACCEPT	192.168.10.0/24	ICMP
proxmox-management	IN	ACCEPT	192.168.10.0/24	9283 TCP
corosync-cluster	IN	ACCEPT	10.10.20.0/24	5404–5405 UDP
ceph-storage	IN	ACCEPT	10.10.30.0/24	6800–7300 TCP
ceph-storage	IN	ACCEPT	10.10.30.0/24	3300, 6789 TCP
ceph-storage	IN	ACCEPT	10.10.40.0/24	6800–7300 TCP
vxlan-overlay	IN	ACCEPT	192.168.10.0/24	4789 UDP
migration-live	IN	ACCEPT	10.10.50.0/24	any TCP

L'accès à l'interface web Proxmox (port 8006) et au service SSH (port 22) est restreint au seul réseau de management 192.168.10.0/24, empêchant tout accès non autorisé depuis les réseaux applicatifs. La règle ICMP, limitée au même réseau, permet les diagnostics de connectivité sans exposer les nœuds à des requêtes provenant des VMs internes. Le port 9283 est autorisé depuis le réseau de management uniquement, de sorte que seul le conteneur `monitoring` (192.168.10.99) peut interroger l'endpoint Prometheus du MGR Ceph, empêchant toute exfiltration des métriques d'infrastructure depuis les réseaux applicatifs. Le trafic Corosync est limité au réseau VxLAN 20 (10.10.20.0/24), tandis que les communications Ceph sont isolées sur leurs VxLAN dédiés (30 et 40). Enfin, le réseau de migration (VxLAN 50) est autorisé sans restriction de port, car Proxmox utilise des ports dynamiques pour le transfert de l'état mémoire des VMs lors des migrations live.

Ces règles au niveau nœud constituent un premier rempart de défense en profondeur, garantissant que même en cas de compromission d'une VM applicative, l'hyperviseur reste inaccessible depuis les réseaux internes.

Règles au niveau des VMs

Chaque VM dispose de règles de pare-feu adaptées à son rôle. Le principe appliqué est celui du moindre privilège : seuls les ports strictement nécessaires sont ouverts en entrée, et les flux sortants sont autorisés uniquement vers les destinations légitimes. Le tableau IV.9 détaille ces règles pour chaque VM applicative.

TABLEAU IV.9 – Règles de pare-feu par VM applicative.

VM	Direction	Action	Source	Port / Protocole
HAProxy (201)	IN	ACCEPT	0.0.0.0/0	8080 TCP
HAProxy (201)	IN	ACCEPT	management-net	8404 TCP
HAProxy (201)	IN	ACCEPT	management-net	22 TCP
HAProxy (201)	IN	ACCEPT	management-net	ICMP
HAProxy (201)	OUT	ACCEPT	–	8000 TCP
Flask app-1/2	IN	ACCEPT	haproxy	8000 TCP
Flask app-1/2	IN	ACCEPT	management-net	22 TCP
Flask app-1/2	OUT	ACCEPT	–	5432 TCP
PostgreSQL (205)	IN	ACCEPT	app1, app2	5432 TCP
PostgreSQL (205)	IN	ACCEPT	management-net	22 TCP

Ces règles garantissent que chaque VM ne communique qu’avec les composants dont elle dépend directement. Pour la VM HAProxy, deux règles supplémentaires ont été ajoutées : le port 8404 TCP, restreint au réseau de management, expose l’interface web de statistiques native d’HAProxy, permettant de consulter en temps réel le nombre de connexions actives, les codes de réponse et l’état de chaque backend sans passer par Grafana. La règle ICMP, également limitée au réseau de management, autorise les diagnostics de connectivité vers le frontal sans l’exposer aux réseaux internes. Tout autre flux est implicitement bloqué par la politique par défaut, assurant une isolation stricte entre les composants de l’application.

IV.4.5 Validation de la haute disponibilité réseau et applicative

Un scénario de panne a été réalisé afin de tester le comportement global de l’infrastructure en cas de perte d’un nœud physique. L’arrêt brutal de PVE-01 a permis d’observer la détection de panne par Corosync, le redémarrage des services protégés sur un autre nœud, puis la resynchronisation automatique après le retour du serveur.

Cette séquence a montré que le cluster restait opérationnel malgré la défaillance, et que les mécanismes de haute disponibilité répondaient correctement à ce type d’événement. La chronologie observée était la suivante : détection de la défaillance par Corosync, déclenchement du basculement HA, redémarrage des VMs protégées sur les nœuds survivants, puis resynchronisation Ceph après le retour du nœud défaillant.

IV.5 Partie IV : Services Complémentaires d’Infrastructure

Cette partie présente le déploiement des services complémentaires qui complètent l’infrastructure hyperconvergée : le système de notification, la centralisation des journaux, la synchronisation temporelle, la gestion du code source et la plateforme d’automatisation opérationnelle.

IV.5.1 Système de Notification : Alertmanager

La supervision active de l'infrastructure ne se limite pas à la visualisation des métriques : elle inclut la notification automatique des équipes en cas d'anomalie détectée. Le module Grafana Alerting a été configuré pour évaluer des règles d'alerte sur les métriques collectées et transmettre les notifications via un contact point Gmail utilisant le protocole SMTP.

IV.5.1.1 Configuration des règles d'alerte

Les règles d'alerte sont définies directement dans Grafana sous “Alerting -> Alert Rules”. Chaque règle spécifie une expression PromQL ou une requête InfluxDB, un seuil de déclenchement, une durée de maintien avant passage en état “Firing”, et le groupe de contact destinataire.

Parmi les règles configurées figurent notamment la détection d'une charge CPU élevée sur les nœuds Proxmox, la surveillance de l'état de santé du cluster Ceph, et la détection d'un OSD passé hors ligne. Ces règles couvrent les scénarios de défaillance les plus critiques de l'infrastructure.

IV.5.1.2 Configuration du contact point Gmail

Le contact point Gmail a été créé dans Grafana sous “Alerting -> Contact Points”. La configuration utilise le serveur SMTP de Google avec authentification par mot de passe d'application, garantissant la compatibilité avec les comptes protégés par la double authentification.

```
# Dans grafana.ini ou via l'interface web :
[smtp]
host      = smtp.gmail.com:587
user      = adresse@gmail.com
password  = mot_de_passe_application
from_address = adresse@gmail.com
starttls_policy = MandatoryStartTLS
```

Figure IV.52 – Paramètres SMTP du contact point Gmail

Dans la Figure IV.52, l'activation de `MandatoryStartTLS` garantit que les identifiants SMTP ne transitent jamais en clair, conformément aux exigences de sécurité de l'infrastructure.

IV.5.1.3 Validation

La validation du système de notification s'appuie sur deux vérifications. La Figure IV.53 présente les règles d'alerte configurées dans Grafana Alerting ainsi que leur état. Le contact point associé pour l'acheminement des notifications est configuré via Gmail, en utilisant l'adresse email de destination directement dans l'interface Grafana Alerting.

[CAPTURE À INSÉRER]

FIGURE IV.53 – Règles d'alerte configurées dans Grafana Alerting.

[CAPTURE : ouvrir Grafana, aller dans Alerting -> Alert Rules, capturer la liste des règles définies (ex : CPU nœud, santé Ceph) avec leur état affiché (Normal / Pending / Firing)]

IV.5.2 Centralisation des Journaux : Loki et Alloy

La collecte et la centralisation des journaux système constituent un élément essentiel pour le diagnostic des incidents et l'audit de l'infrastructure. Loki, déployé dans le conteneur LXC `monitoring` existant, joue le rôle d'agrégateur de journaux. L'agent Grafana Alloy, installé sur chaque nœud Proxmox, collecte les fichiers de log locaux et les transmet à Loki pour indexation.

IV.5.2.1 Déploiement de Loki

Loki a été installé dans le conteneur LXC `monitoring` (192.168.10.99), aux côtés de Prometheus et Grafana déjà en place. Cette co-localisation simplifie la configuration de Grafana, qui dispose ainsi de Loki comme source de données supplémentaire sans configuration réseau additionnelle.

```
apt install loki
systemctl enable loki
systemctl start loki
```

Figure IV.54 – Installation de Loki

Les commandes de la Figure IV.54 installent et activent Loki, qui écoute par défaut sur le port 3100. La source de données Loki a ensuite été ajoutée dans Grafana en pointant vers `http://localhost:3100`.

IV.5.2.2 Déploiement de l'agent Grafana Alloy

L'agent Grafana Alloy a été installé sur chacun des trois nœuds Proxmox. Sa configuration définit les chemins de fichiers de journaux à surveiller et l'adresse du serveur Loki cible.

```
local.file_match "varlogs" {
  path_targets = [{"__path__" = "/var/log/**/*.log"}]
}

loki.source.file "varlogs" {
  targets      = local.file_match.varlogs.targets
  forward_to   = [loki.write.default.receiver]
}

loki.write "default" {
  endpoint {
    url = "http://192.168.10.99:3100/loki/api/v1/push"
  }
}
```

Figure IV.55 – Configuration de l'agent Alloy pour la collecte des journaux

La configuration de la Figure IV.55 collecte tous les fichiers `*.log` sous `/var/log/` de chaque nœud et les transmet en flux continu vers Loki. Les labels `host` et `filename` sont automatiquement ajoutés à chaque entrée, permettant de filtrer les journaux par nœud dans Grafana.



FIGURE IV.56 – Journaux de l’infrastructure visualisés dans Grafana via Loki.

La Figure IV.56 présente la vue “Explore” de Grafana avec Loki comme source de données. Les journaux sont filtrables par hôte et par fichier source via LogQL. La vue complète du dashboard de centralisation des journaux est disponible en Annexe A.

IV.5.3 Synchronisation Temporelle : NTP/Chrony

La cohérence temporelle est une exigence fondamentale dans un cluster distribué. Une dérive de quelques secondes entre les nœuds peut perturber le quorum Corosync, fausser les journaux de corrélation et provoquer des incohérences dans les métriques Prometheus. Le serveur Chrony est hébergé directement sur **PVE-01** (192.168.10.100), qui fait référence pour l’ensemble des nœuds Proxmox et des VMs de l’infrastructure.

IV.5.3.1 Configuration du serveur Chrony sur PVE-01

La configuration du fichier `/etc/chrony/chrony.conf` sur PVE-01 définit les serveurs de référence amont, la plage réseau des clients autorisés et le comportement de repli local.

```
# Serveurs NTP de reference amont
pool pool.ntp.org iburst

# Autoriser les clients du reseau de management
allow 192.168.10.0/24

# Fichier de derive
driftfile /var/lib/chrony/drift

# Repli local si perte de synchronisation amont
local stratum 10
```

Figure IV.57 – Configuration du serveur Chrony sur PVE-01

Dans la Figure IV.57, la directive `allow 192.168.10.0/24` couvre l’ensemble des nœuds Proxmox et des clients directs du réseau de management. Les VMs hébergées sur les VLANs

172.16.60/70/80.0/24 atteignent PVE-01 via un routeur qui effectue une translation d'adresses (NAT) : leurs requêtes NTP arrivent sur PVE-01 avec l'adresse source 192.168.10.65 (interface du routeur côté management), et sont donc couvertes par la même directive `allow` sans qu'il soit nécessaire d'y ajouter les sous-réseaux VLAN. La directive `local stratum 10` garantit que PVE-01 continue à distribuer une référence temporelle cohérente en cas de perte temporaire de connectivité vers `pool.ntp.org`.

IV.5.3.2 Configuration des clients NTP

Sur chaque nœud Proxmox et sur chaque VM, le fichier `chrony.conf` client interroge PVE-01 en priorité, puis bascule vers `pool.ntp.org` si PVE-01 est indisponible.

```
# Source primaire : serveur interne PVE-01
server 192.168.10.100 iburst prefer

# Fallback : serveurs publics si PVE-01 inaccessible
pool pool.ntp.org iburst

driftfile /var/lib/chrony/drift
```

Figure IV.58 – Configuration Chrony côté clients (nœuds et VMs)

La configuration de la Figure IV.58 interroge PVE-01 en priorité, avec bascule automatique vers `pool.ntp.org` si PVE-01 est indisponible. À la différence des conteneurs LXC qui héritent directement de l'horloge du nœud hôte, les VMs disposent d'une horloge virtuelle indépendante et nécessitent donc une synchronisation NTP explicite.

IV.5.3.3 Déploiement via Ansible

La configuration NTP est appliquée sur les VMs via le playbook Ansible `ntp_sync.yml`. Ce playbook est exécuté à la demande, en passant l'adresse IP de la VM cible, ce qui permet d'intégrer toute nouvelle machine dans la hiérarchie de synchronisation sans intervention manuelle. Le fonctionnement détaillé de l'automatisation Ansible est traité dans la section dédiée.

IV.5.3.4 Validation de la synchronisation

La vérification du bon fonctionnement du serveur Chrony s'effectue depuis PVE-01 via deux commandes complémentaires. La première confirme la connexion aux sources NTP amont, la seconde liste les clients actifs.

```
root@pve-01:~# chronyc -n clients
```

Hostname	NTP	Drop	Int	IntL	Last	Cmd	Drop	Int	Last
192.168.10.213	27	0	6	-	20	0	0	-	-
192.168.10.114	8	0	9	-	21	0	0	-	-
192.168.10.84	26	0	6	-	61	0	0	-	-
192.168.10.65	25	0	6	-	57	0	0	-	-
192.168.10.104	7	0	7	-	39	0	0	-	-
192.168.10.112	7	0	6	-	23	0	0	-	-
192.168.10.115	6	0	7	-	12	0	0	-	-
192.168.10.111	8	0	6	-	1	0	0	-	-

FIGURE IV.59 – Sortie de `chronyc -n clients` sur PVE-01.

La Figure IV.59 illustre la sortie de la commande `chronyc -n clients` exécutée sur PVE-01. Les nœuds Proxmox PVE-02 (192.168.10.213) et PVE-03 (192.168.10.84) apparaissent individuellement avec leur compteur de requêtes NTP. Les VMs hébergées sur les réseaux distants (172.16.60/70/80.0/24) n'apparaissent pas directement : leurs requêtes NTP arrivent sur PVE-01 avec l'adresse source 192.168.10.65 (interface WAN d'OPNsense), car le routeur effectue une translation d'adresses (NAT) entre les VLANs internes et le réseau de management. Ce comportement est attendu et confirme que les VMs bénéficient bien de la synchronisation NTP via OPNsense.

IV.5.4 Gestion du Code Source et CI/CD : GitLab

La traçabilité du code et l'automatisation des tests constituent des pratiques essentielles dans un environnement d'infrastructure moderne. Un LXC Turnkey GitLab CE a été déployé sur le cluster pour centraliser les dépôts de code de l'infrastructure et y associer des pipelines d'intégration continue.

IV.5.4.1 Déploiement du LXC Turnkey GitLab CE

Le template Turnkey GitLab CE disponible dans le catalogue Proxmox a été utilisé pour simplifier le déploiement initial. Ce template embarque GitLab CE préconfiguré avec Nginx, PostgreSQL et les workers Sidekiq, réduisant le temps d'installation à quelques minutes.

Le conteneur LXC a été créé avec les ressources suivantes : 4 vCPUs, 8 Go de RAM et 50 Go de stockage sur le pool Ceph, permettant d'héberger confortablement les dépôts et les artefacts de pipeline de l'infrastructure.

IV.5.4.2 Création des dépôts

Deux dépôts ont été créés dans l'instance GitLab pour couvrir les besoins de versionnement de l'infrastructure. Le dépôt `clbs` contient le code source du service d'équilibrage de charge développé en Go, y compris les fichiers de configuration, les tests unitaires et le fichier de pipeline CI/CD. Le dépôt `ansible-infra` regroupe l'ensemble des playbooks Ansible, l'inventaire et les fichiers de secrets chiffrés avec Ansible Vault.

IV.5.4.3 Configuration des pipelines CI/CD

Un fichier `.gitlab-ci.yml` a été défini dans chaque dépôt pour automatiser les vérifications à chaque push. Pour le projet `clbs`, le pipeline comprend les stages “Lint” (vérification syntaxique du code Go avec `golint`), “Tests” (exécution des tests unitaires avec `go test`) et “Build” (compilation du binaire). Pour le projet `ansible-infra`, le pipeline effectue une vérification syntaxique des playbooks avec `ansible-lint`.

IV.5.4.4 Validation

La validation de l’instance GitLab porte sur la présence des dépôts et sur le bon déroulement des pipelines. La Figure IV.60 présente la page des projets de l’instance GitLab, et la Figure ?? montre un pipeline CI/CD exécuté avec succès pour le projet `clbs`.

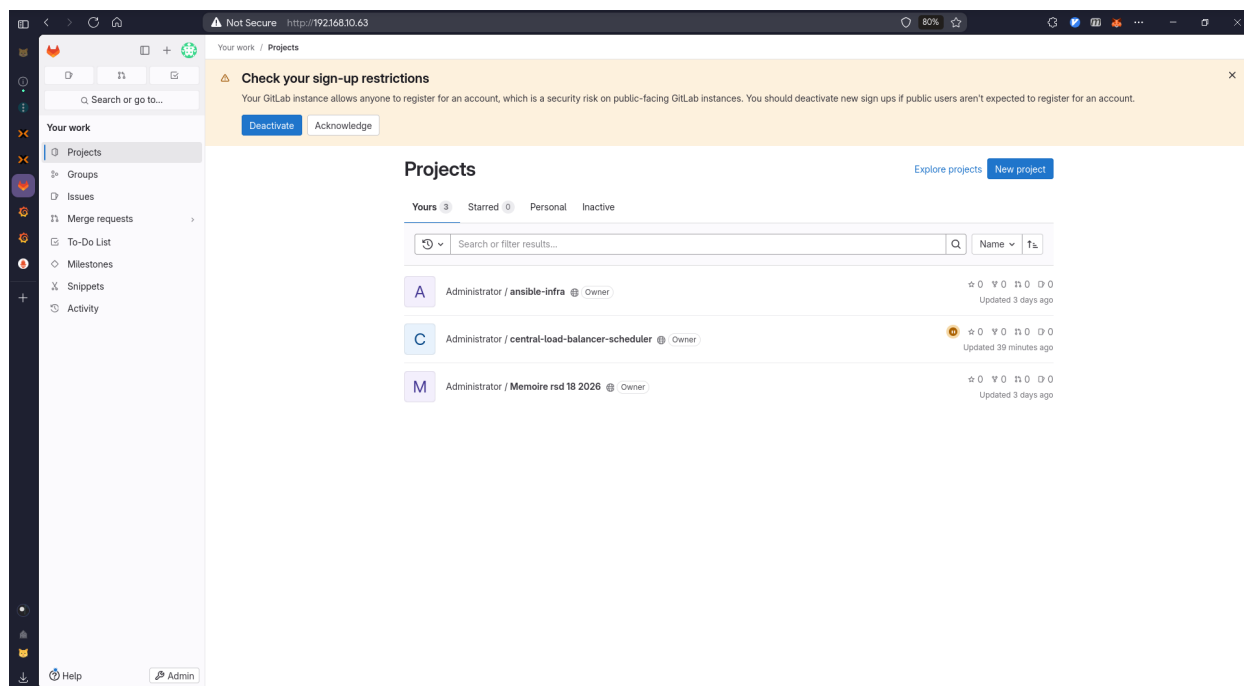


FIGURE IV.60 – Page Projects de l’instance GitLab avec les dépôts de l’infrastructure.

IV.5.5 Automatisation Opérationnelle : Rundeck

La délégation de tâches opérationnelles récurrentes à une interface centralisée permet de réduire les erreurs humaines et de tracer l’ensemble des interventions sur l’infrastructure. Rundeck, déployé dans un conteneur LXC dédié, offre une interface web permettant de déclencher des playbooks Ansible sans accès SSH direct aux nœuds.

IV.5.5.1 Déploiement du LXC Rundeck

Le conteneur LXC Rundeck a été créé sur le cluster Proxmox avec 2 vCPUs et 4 Go de RAM. Rundeck a été installé depuis les dépôts officiels et configuré pour écouter sur le port 4440.

```
apt install -y rundeck
systemctl enable rundeckd
systemctl start rundeckd
```

Figure IV.61 – Installation de Rundeck

Les commandes de la Figure IV.61 installent le service Rundeck, l'activent au démarrage et le démarrent immédiatement. Une fois actif, Rundeck est accessible via son interface web sur le port 4440 du conteneur LXC.

IV.5.5.2 Intégration du dépôt `ansible-infra` via GitLab Runner

Après le déploiement du conteneur, le dépôt `ansible-infra` a été cloné depuis l'instance GitLab locale vers `/root` sur la machine Rundeck. Un "GitLab Runner" a ensuite été enregistré sur ce conteneur et intégré au pipeline CI/CD du dépôt.

Le rôle de ce runner est de synchroniser automatiquement les playbooks à chaque push valide : le pipeline copie le contenu du dépôt vers `/opt/ansible-infra`, répertoire appartenant à l'utilisateur système `rundeck` (`chown -R rundeck:rundeck /opt/ansible-infra`). Cette configuration garantit que Rundeck dispose des permissions nécessaires pour lire et exécuter les playbooks directement depuis ce répertoire, sans manipulation manuelle des fichiers.

Les clés SSH générées précédemment pour l'utilisateur `ansible` ont été déposées dans `/opt/ansible-infra/.ssh/`, rendant ce répertoire la source unique de vérité pour à la fois les playbooks et les secrets d'authentification SSH vers les VMs gérées.

IV.5.5.3 Définition des jobs

Les jobs Rundeck ont été définis pour chaque opération récurrente de l'infrastructure. Chaque job référence le playbook Ansible correspondant dans `/opt/ansible-infra` et expose les variables nécessaires sous forme de paramètres saisissables dans l'interface web, permettant à un opérateur de déclencher une opération sans connaissance de la syntaxe Ansible.

Quatre jobs ont été configurés :

- **Provision VM** : exécute `provision_vm.yml`. Les paramètres exposés sont `vm_id`, `vm_name`, ainsi que des paramètres optionnels tels que `vm_cores`, `vm_memory` et `vm_full_clone` pour ajuster les ressources ou le mode de clonage.
- **Provision VM Batch** : exécute `provision_vm_batch.yml` pour le provisionnement en masse. Le paramètre principal est la liste des VMs à créer, passée sous forme de structure YAML.
- **Bootstrap VM** : exécute `bootstrap.yml` pour préparer une VM nouvellement créée à la gestion Ansible. Le paramètre exposé est `target_hosts`, une liste d'adresses IP ou de noms d'hôtes (séparés par des virgules), permettant de cibler aussi bien des machines de l'inventaire que des IPs brutes.
- **NTP Sync** : exécute `ntp_sync.yml` pour configurer ou resynchroniser chrony sur les nœuds. Le paramètre exposé est également `target_hosts`, permettant de cibler un sous-ensemble de nœuds du cluster.

IV.5.5.4 Validation

La validation de Rundeck porte sur la définition des jobs et sur leur exécution. La Figure IV.62 présente la liste des jobs de provisionnement Ansible configurés dans l'interface web. Le résultat complet d'une exécution, incluant le journal Ansible avec l'état de chaque tâche, est présenté en Annexe C.

[CAPTURE À INSÉRER]

FIGURE IV.62 – Interface Rundeck affichant les jobs de provisionnement Ansible.
[CAPTURE : ouvrir l'interface web Rundeck (port 4440), aller dans Jobs, capturer la liste des jobs configurés (provisionnement VM, intégration nœud, etc.) avec leurs noms visibles]

IV.6 Conclusion

Ce chapitre a présenté l'implémentation réelle de l'infrastructure hyperconvergée open source mise en place dans le cadre du projet. La couche calcul a permis de déployer Proxmox, le cluster, le monitoring, l'automatisation et les services applicatifs. La couche stockage a été assurée par Ceph avec réplication et tolérance aux pannes. Enfin, la couche réseau a été construite avec Open vSwitch, les tunnels VxLAN, OPNsense et les services d'adressage associés.

Les services complémentaires déployés dans la Partie IV (synchronisation temporelle, centralisation des journaux, notifications, versionnement du code et automatisation opérationnelle) complètent l'infrastructure en lui apportant les briques transverses indispensables à son exploitation en conditions réelles.

Les différents tests réalisés ont confirmé la cohérence de l'architecture proposée et sa capacité à répondre aux objectifs du projet dans un environnement de laboratoire structuré. Le chapitre suivant présentera l'évaluation des performances obtenues ainsi qu'une analyse des résultats expérimentaux.

Conclusion générale

Ce projet a abordé la problématique du coût et de la dépendance technologique liés aux solutions d'hyperconvergence propriétaires dans les grandes entreprises industrielles. Sonatrach, dont l'infrastructure de production repose sur Nutanix et VMware vSAN, se trouvait face à un besoin non couvert : disposer d'une infrastructure hyperconvergée viable pour ses environnements de test et de développement, sans recourir aux mêmes solutions dont le coût de licence et la rigidité sont disproportionnés pour des clusters de petite taille. La question centrale était de savoir si des technologies open source pouvaient offrir une alternative crédible, souveraine et techniquement solide, sur du matériel existant.

Pour répondre à cette problématique, une infrastructure Cloud privée hyperconvergée entièrement open source a été déployée sur trois serveurs Dell PowerEdge R720. Un cluster Proxmox VE à trois nœuds a été mis en place avec haute disponibilité gérée par Corosync, un stockage distribué Ceph avec réplication à trois copies, et une segmentation réseau en neuf segments logiques VXLAN gérés par Open vSwitch. Une VM OPNsense assure le routage centralisé et le filtrage entre les zones applicatives. Un service d'équilibrage dynamique de charge, le CLBS, a été développé en Go et déployé pour pallier l'absence d'un mécanisme équivalent au DRS de VMware dans Proxmox. La haute disponibilité a été validée par une simulation de panne : à l'extinction d'un nœud, le cluster a détecté la défaillance via Corosync, redémarré automatiquement les machines virtuelles protégées sur les nœuds survivants, et resynchronisé le stockage après le retour du serveur. Le mécanisme CLBS a été validé sous charge artificielle, déclenchant des migrations à chaud conformément à sa logique de décision. La plateforme est complétée par une stack de supervision (InfluxDB, Prometheus, Grafana, Loki), une synchronisation temporelle Chrony, des pipelines CI/CD GitLab et une automatisation opérationnelle via Ansible et Rundeck.

Par rapport aux solutions existantes, cette infrastructure apporte plusieurs avantages concrets. Face à Nutanix et VMware vSAN, elle supprime entièrement les coûts de licence et donne un accès total au code, ce qui permet d'adapter la solution aux besoins spécifiques de l'entreprise. Face à OpenStack, elle est nettement plus légère et opérationnelle en quelques heures sur un cluster de trois nœuds, sans orchestration complexe de services interdépendants. Face à Harvester, elle offre une meilleure gestion des machines virtuelles classiques Linux et Windows avec une empreinte mémoire sur l'hôte inférieure à 1 Go à vide. Le développement du CLBS comble un manque réel de Proxmox VE en matière d'équilibrage proactif, en apportant une adaptation de l'algorithme CSLB étendue à la RAM, avec des protections contre les oscillations absentes des implémentations de référence. L'ensemble montre qu'une pile open source bien assemblée peut couvrir les mêmes fonctions qu'une solution propriétaire intégrée, à un coût d'acquisition nul.

Cette réalisation présente toutefois des limites. Le pipeline PXE/TFTP, conçu pour l'installation automatique des nœuds physiques, n'a pas pu être testé faute de matériel dédié disponible ; sa logique est documentée mais non vérifiée en conditions réelles. La segmentation

réseau par VXLAN résulte d'une contrainte imposée par l'inaccessibilité du switch physique Sonatrach, et impose une configuration manuelle des fichiers d'interfaces sur chaque nœud, ce qui devient fastidieux à l'échelle. La gestion des identités repose sur le système natif de Proxmox sans annuaire centralisé, créant une fragmentation des droits entre Grafana, GitLab et Rundeck. Plusieurs perspectives d'évolution peuvent être envisagées pour lever ces limites et étendre les capacités de la plateforme :

- Migrer les tunnels VxLAN OVS vers des VLANs OVS natifs via le module SDN de Proxmox, sous réserve d'accès aux commutateurs Sonatrach.
- Valider un pipeline PXE/Ansible permettant d'intégrer un nouveau nœud, de la mise sous tension à son admission dans le cluster, en moins de 25 minutes sans intervention manuelle.
- Centraliser l'authentification sur Proxmox, Grafana, GitLab et Rundeck via un annuaire OpenLDAP en remplacement des comptes locaux fragmentés.
- Remplacer la stack Grafana/Loki/Alloy/Alertmanager par Wazuh, une plateforme SIEM couvrant la détection d'intrusion, l'intégrité des fichiers et la corrélation d'événements.
- Déployer Apprise pour acheminer les alertes Grafana vers plusieurs canaux (WhatsApp, SMS, SMTP) sans dépendre du serveur Gmail externe.

Bibliographie

- [1] I. N. Sonatrach, « Historique et évolution », Institut numérique, 2012. [En ligne]. Disponible : <https://www.institut-numerique.org/2-historique-et-evolution-5028f58cb09d3>
- [2] J. Schneider, S. Susnjara et I. Smalley, « Hyperconverged infrastructure explained », IBM Think, févr. 2024. [En ligne]. Disponible : <https://www.ibm.com/think/topics/hyperconverged-infrastructure>
- [3] A. Pande, « Proxmox vs. Harvester : Can the enterprise-grade platform beat the community favorite? », XDA Developers, févr. 2025. [En ligne]. Disponible : <https://www.xda-developers.com/proxmox-vs-harvester/>
- [4] SUSE, « Harvester HCI Documentation », Harvester HCI. [En ligne]. Disponible : <https://harvesterhci.io/>
- [5] Proxmox Server Solutions, « Proxmox Virtual Environment Overview », Proxmox. [En ligne]. Disponible : <https://www.proxmox.com/en/products/proxmox-virtual-environment/overview>
- [6] Open vSwitch Authors, « What is Open vSwitch? », Documentation Open vSwitch. [En ligne]. Disponible : <https://docs.openvswitch.org/en/latest/intro/what-is-ovs/>
- [7] Open vSwitch Authors, « Features », Open vSwitch. [En ligne]. Disponible : <https://www.openvswitch.org/features/>
- [8] Broadcom, « VMware DRS Overview : Optimizing Resource Allocation in vSphere Cluster », Broadcom Knowledge Base. [En ligne]. Disponible : <https://knowledge.broadcom.com/external/article/391137/vmware-drs-overview-optimizing-resource.html>
- [9] InfluxData, « InfluxDB v2 Documentation », InfluxData. [En ligne]. Disponible : <https://docs.influxdata.com/influxdb/v2/>
- [10] TechWrix, « Containers vs VMs : Choosing the Right Approach for Your Proxmox VE », TechWrix. [En ligne]. Disponible : <https://www.techwrix.com/containers-vs-vms-choosing-the-right-approach-for-your-proxmox-ve-part-4/>
- [11] A. Chandak, K. Jaju, A. Kanfode, P. Lohiya et A. Joshi, « Dynamic Load Balancing of Virtual Machines using QEMU-KVM », *International Journal of Computer Applications*, vol. 46, n° 6, pp. 10–14, mai 2012.
- [12] J. Watts, « A practical approach to dynamic load balancing », Mémoire de master, California Institute of Technology, 1995. [En ligne]. Disponible : https://thesis.caltech.edu/6924/1/Watts_j_1995.pdf

BIBLIOGRAPHIE

- [13] Corosync Project, « The Corosync Cluster Engine », Corosync. [En ligne]. Disponible : <https://corosync.github.io/corosync/>
- [14] Grafana Labs, « Grafana Documentation », Grafana Labs. [En ligne]. Disponible : <https://grafana.com/docs/grafana/latest/>
- [15] W. Tarreau, « HAProxy version 2.8 : configuration manuelle », HAProxy. [En ligne]. Disponible : <https://www.haproxy.org/download/2.8/doc/configuration.txt>
- [16] S. A. Weil, S. A. Brandt, E. L. Miller et D. D. Long, « Ceph : A scalable, high-performance distributed file system », dans *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI'06)*, USENIX Association, 2006, pp. 307–320.
- [17] S. A. Weil, S. A. Brandt, E. L. Miller et C. Maltzahn, « CRUSH : Controlled, Scalable, Decentralized Placement of Replicated Data », dans *Proceedings of the 2007 ACM/IEEE Conference on Supercomputing (SC'07)*, ACM, 2007.
- [18] Proxmox Server Solutions, « Proxmox VE Ceph Server », Proxmox VE Wiki, 2024. [En ligne]. Disponible : https://pve.proxmox.com/wiki/Ceph_Server
- [19] T. C. Wilcox Jr., « Dynamic Load Balancing of Virtual Machines Hosted on Xen », Mémoire de master, Brigham Young University, avr. 2009. [En ligne]. Disponible : <https://scholarsarchive.byu.edu/cgi/viewcontent.cgi?article=2653&context=etd>
- [20] GitLab, « GitLab Requirements », Documentation GitLab. [En ligne]. Disponible : <https://docs.gitlab.com/install/requirements/>
- [21] KVM Authors, « Kernel-based Virtual Machine », Linux KVM. [En ligne]. Disponible : https://www.linux-kvm.org/page/Main_Page
- [22] QEMU Project, « QEMU Documentation », QEMU. [En ligne]. Disponible : <https://www.qemu.org/docs/master/>
- [23] libvirt Project, « libvirt : The virtualization API », libvirt.org. [En ligne]. Disponible : <https://libvirt.org/>
- [24] Proxmox Server Solutions, « High Availability Manager », Proxmox VE Wiki. [En ligne]. Disponible : https://pve.proxmox.com/wiki/High_Availability
- [25] M. Mahalingam et al., « Virtual eXtensible Local Area Network (VXLAN) : A Framework for Overlaying Virtualized Layer 2 Networks over Layer 3 Networks », RFC 7348, IETF, août 2014. [En ligne]. Disponible : <https://datatracker.ietf.org/doc/html/rfc7348>
- [26] Microsoft, « Dynamic Optimization in VMM », Microsoft Learn. [En ligne]. Disponible : <https://learn.microsoft.com/en-us/system-center/vmm/>
- [27] oVirt Project, « Scheduling Policies », oVirt Documentation. [En ligne]. Disponible : <https://www.ovirt.org/documentation/>
- [28] OpenStack Foundation, « Nova Scheduler », OpenStack Documentation. [En ligne]. Disponible : <https://docs.openstack.org/nova/latest/admin/scheduling.html>
- [29] Prometheus Authors, « Prometheus Documentation », Prometheus. [En ligne]. Disponible : <https://prometheus.io/docs/>

BIBLIOGRAPHIE

- [30] Red Hat, « Ansible Documentation », Ansible. [En ligne]. Disponible : <https://docs.ansible.com/>
- [31] Deciso B.V., « OPNsense Documentation », OPNsense. [En ligne]. Disponible : <https://docs.opnsense.org/>
- [32] D. Mills et al., « Network Time Protocol Version 4 : Protocol and Algorithms Specification », RFC 5905, IETF, juin 2010. [En ligne]. Disponible : <https://datatracker.ietf.org/doc/html/rfc5905>

Annexe A

Dashboards Grafana de Supervision

Cette annexe présente les captures complètes des tableaux de bord Grafana déployés pour superviser l'ensemble de l'infrastructure hyperconvergée. Ces dashboards sont accessibles depuis l'interface Grafana sur le conteneur LXC `monitoring` (192.168.10.99).

A.1 Dashboard Infrastructure Proxmox

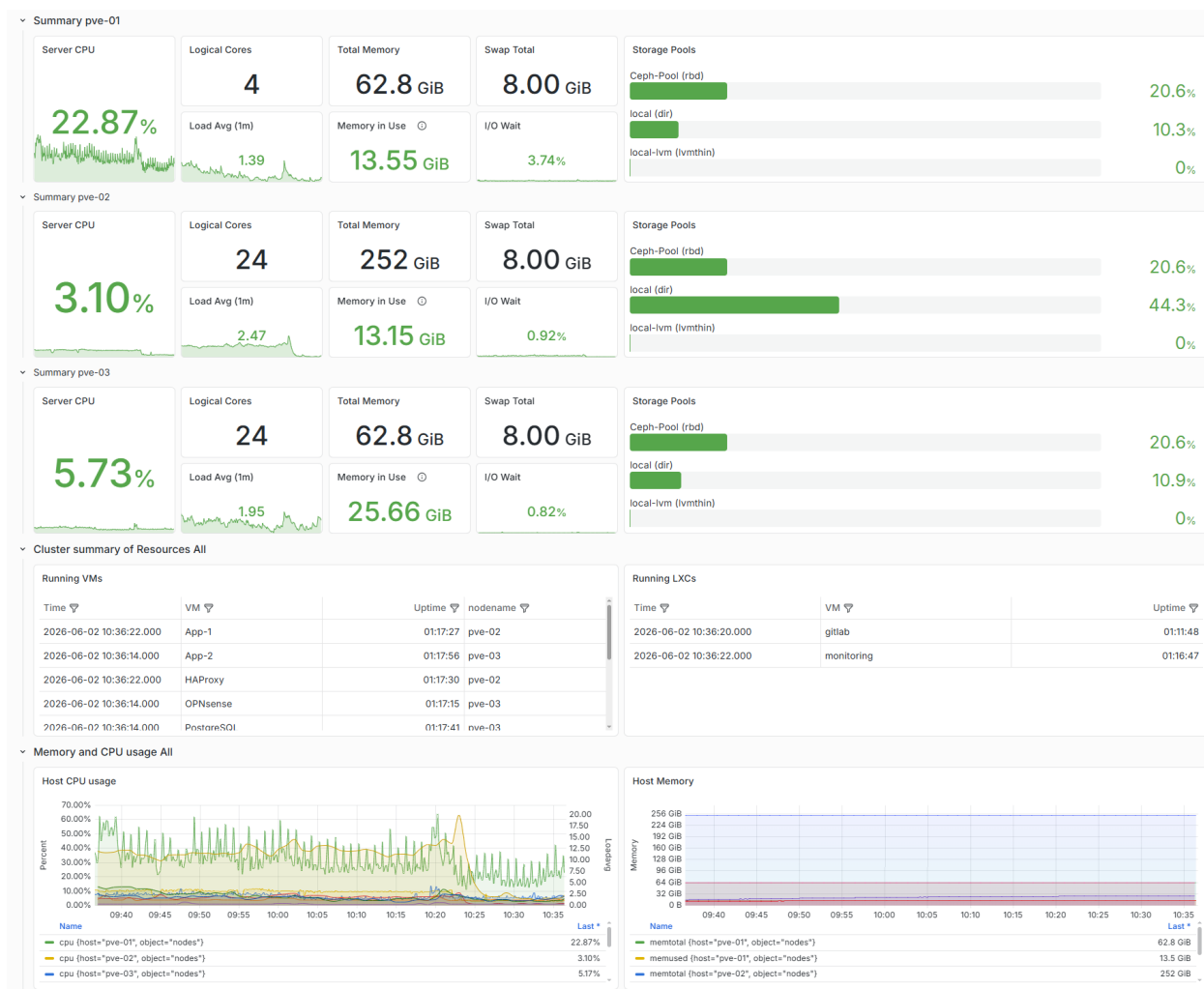


FIGURE A.1 – Dashboard Grafana Infrastructure – vue complète des métriques CPU, mémoire et réseau des trois nœuds Proxmox.

Ce tableau de bord, alimenté par InfluxDB via le mécanisme natif “Metric Server” de Proxmox, affiche en temps réel l’utilisation du CPU, la mémoire disponible, la charge réseau et le nombre de VMs en cours d’exécution pour chacun des trois nœuds PVE-01, PVE-02 et PVE-03. Les séries temporelles permettent d’identifier les tendances de charge sur les dernières heures et de corrélérer les événements de migration CLBS avec l’évolution des ressources.

A.2 Dashboard Cluster Ceph

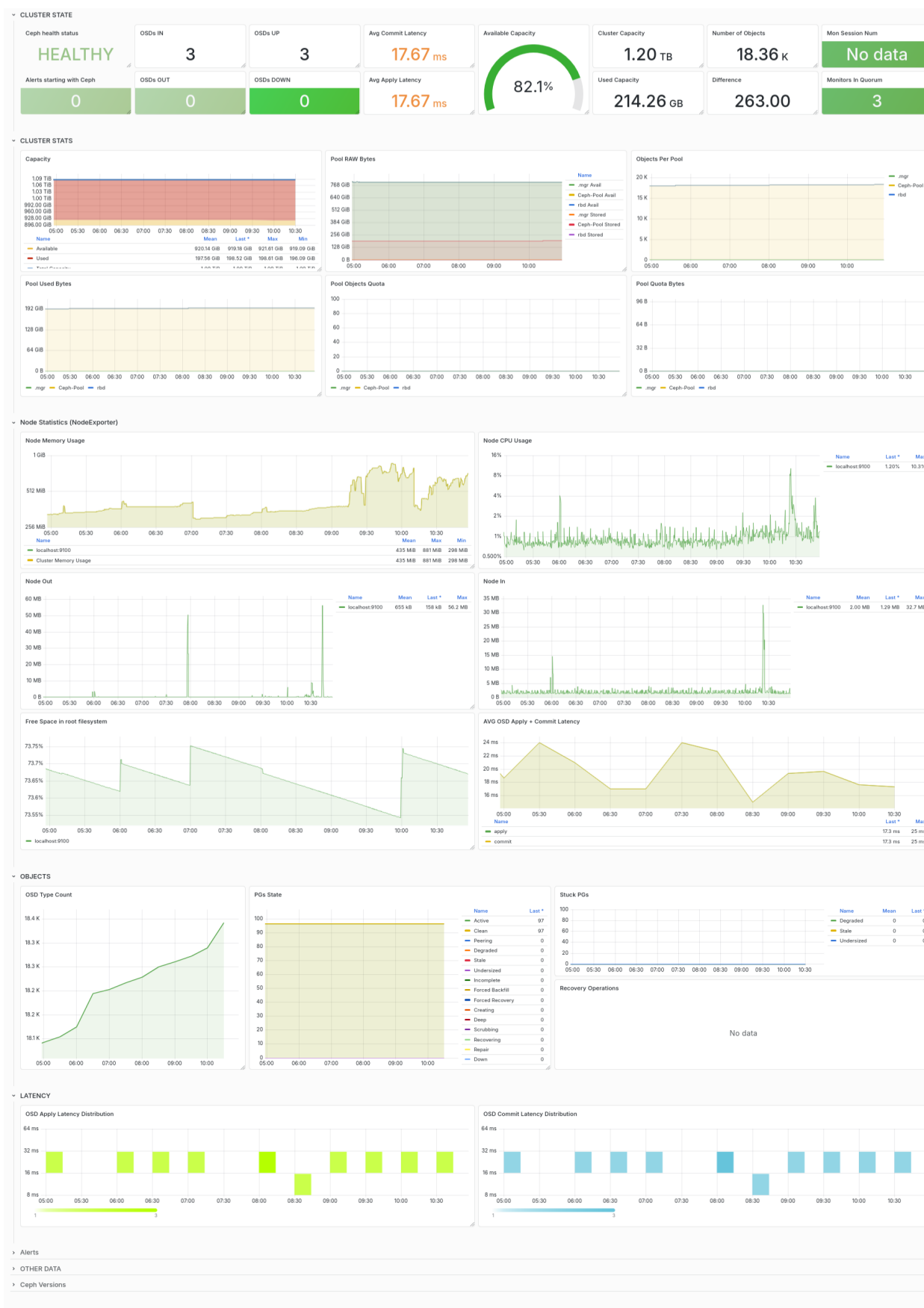


FIGURE A.2 – Dashboard Grafana Ceph – vue complète de l'état du cluster, des pools et de la latence des OSDs.

Ce tableau de bord, alimenté par Prometheus via le module MGR Ceph, centralise les métriques de santé du cluster : état global (`HEALTH_OK`/`HEALTH_WARN`), capacité utilisée et disponible par pool, débit I/O en lecture et écriture, latence des OSDs et taux de réplication en temps réel. Il constitue la vue de référence pour diagnostiquer tout incident affectant la couche stockage.

A.3 Journaux Système via Loki

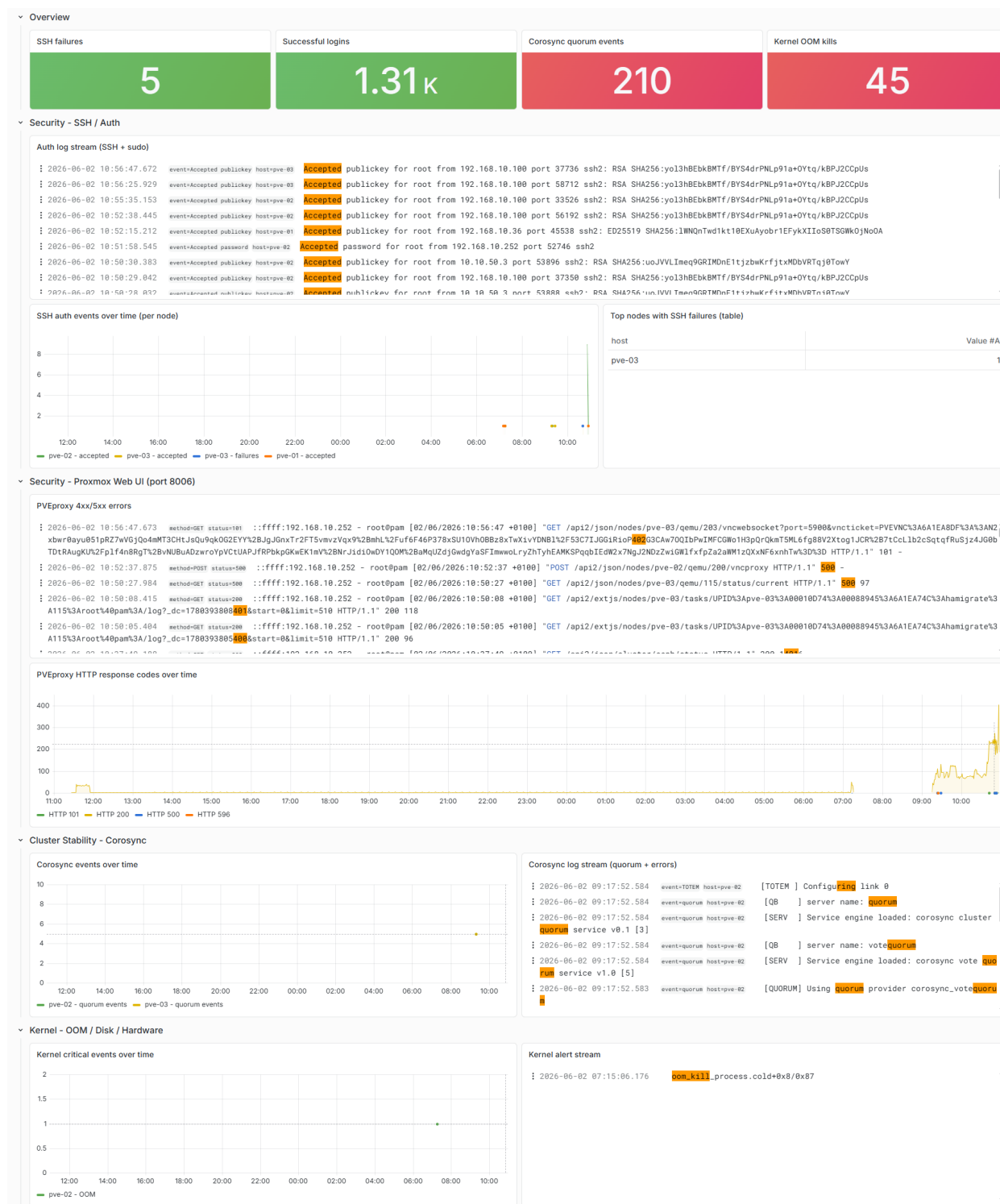


FIGURE A.3 – Dashboard Grafana Loki – journaux système centralisés des nœuds Proxmox et des VMs.

Ce tableau de bord, utilisant Loki comme source de données dans Grafana, permet d'interroger les journaux système de l'ensemble des nœuds et VMs via le langage LogQL. Les logs sont filtrables par hôte (label `host`), par fichier source (label `filename`) et par niveau de sévérité. Cette vue unifiée coexiste avec les tableaux de bord de métriques, offrant une

corrélation directe entre les événements système et les variations de charge.

Annexe B

Configuration Complète des Interfaces Réseau

Cette annexe présente le fichier `/etc/network/interfaces` complet de chacun des trois nœuds Proxmox. Ces fichiers définissent l'ensemble des tunnels VXLAN point-à-point en maillage complet ainsi que les bridges OVS associés. Seule l'adresse IP du bridge de management et les adresses des interfaces internes diffèrent d'un nœud à l'autre.

B.1 Configuration complète de PVE-01

```
auto lo
iface lo inet loopback

iface nic0 inet manual

auto vmbr0
iface vmbr0 inet static
    address 192.168.10.100/24
    gateway 192.168.10.36
    ovs_type OVSBridge
    ovs_ports nic0

auto nic0
iface nic0 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0

iface nic1 inet manual
iface nic2 inet manual
iface nic3 inet manual

# ===== COROSYNC VXLAN (key=20) =====
auto vxlan20-pve02
iface vxlan20-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=20
```

```
auto vxlan20-pve03
iface vxlan20-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=20

auto vxlan20-int
iface vxlan20-int inet static
    address 10.10.20.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== CEPH INTERNE VXLAN (key=30) =====
auto vxlan30-pve02
iface vxlan30-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=30

auto vxlan30-pve03
iface vxlan30-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=30

auto vxlan30-int
iface vxlan30-int inet static
    address 10.10.30.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== CEPH PUBLIC VXLAN (key=40) =====
auto vxlan40-pve02
iface vxlan40-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=40

auto vxlan40-pve03
iface vxlan40-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=40

auto vxlan40-int
iface vxlan40-int inet static
    address 10.10.40.1/24
```

```
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== MIGRATION LIVE VXLAN (key=50) =====
auto vxlan50-pve02
iface vxlan50-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=50

auto vxlan50-pve03
iface vxlan50-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=50

auto vxlan50-int
iface vxlan50-int inet static
    address 10.10.50.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== SAUVEGARDE VXLAN (key=90) =====
auto vxlan90-pve02
iface vxlan90-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=90

auto vxlan90-pve03
iface vxlan90-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=90

auto vxlan90-int
iface vxlan90-int inet static
    address 10.10.90.1/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 60 (Dev) =====
auto vxlan60-pve02
iface vxlan60-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=60
```

```
auto vxlan60-pve03
iface vxlan60-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=60

auto vxlan60-int
iface vxlan60-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 70 (Prod) =====
auto vxlan70-pve02
iface vxlan70-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=70

auto vxlan70-pve03
iface vxlan70-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=70

auto vxlan70-int
iface vxlan70-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 80 (DMZ) =====
auto vxlan80-pve02
iface vxlan80-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=80

auto vxlan80-pve03
iface vxlan80-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=80

auto vxlan80-int
iface vxlan80-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VM Bridges =====
```

```
auto vmbr60
iface vmbr60 inet manual
    bridge_ports vxlan60-int
    bridge_stp off
    bridge_fd 0

auto vmbr70
iface vmbr70 inet manual
    bridge_ports vxlan70-int
    bridge_stp off
    bridge_fd 0

auto vmbr80
iface vmbr80 inet manual
    bridge_ports vxlan80-int
    bridge_stp off
    bridge_fd 0

source /etc/network/interfaces.d/*
```

Figure B.1 – Fichier /etc/network/interfaces complet – PVE-01 (192.168.10.100)

B.2 Configuration complète de PVE-02

```
auto lo
iface lo inet loopback

iface nic0 inet manual

auto vmbr0
iface vmbr0 inet static
    address 192.168.10.213/24
    gateway 192.168.10.36
    ovs_type OVSBridge
    ovs_ports nic0

auto nic0
iface nic0 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0

iface nic1 inet manual
iface nic2 inet manual
iface nic3 inet manual

# ===== COROSYNC VXLAN (key=20) =====
auto vxlan20-pve01
iface vxlan20-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=20
```

```
auto vxlan20-pve03
iface vxlan20-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=20

auto vxlan20-int
iface vxlan20-int inet static
    address 10.10.20.2/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== CEPH INTERNE VXLAN (key=30) =====
auto vxlan30-pve01
iface vxlan30-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=30

auto vxlan30-pve03
iface vxlan30-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=30

auto vxlan30-int
iface vxlan30-int inet static
    address 10.10.30.2/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== CEPH PUBLIC VXLAN (key=40) =====
auto vxlan40-pve01
iface vxlan40-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=40

auto vxlan40-pve03
iface vxlan40-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=40

auto vxlan40-int
iface vxlan40-int inet static
    address 10.10.40.2/24
```

```
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== MIGRATION LIVE VXLAN (key=50) =====
auto vxlan50-pve01
iface vxlan50-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=50

auto vxlan50-pve03
iface vxlan50-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=50

auto vxlan50-int
iface vxlan50-int inet static
    address 10.10.50.2/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== SAUVEGARDE VXLAN (key=90) =====
auto vxlan90-pve01
iface vxlan90-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=90

auto vxlan90-pve03
iface vxlan90-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=90

auto vxlan90-int
iface vxlan90-int inet static
    address 10.10.90.2/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 60 (Dev) =====
auto vxlan60-pve01
iface vxlan60-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=60
```

```
auto vxlan60-pve03
iface vxlan60-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=60

auto vxlan60-int
iface vxlan60-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 70 (Prod) =====
auto vxlan70-pve01
iface vxlan70-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=70

auto vxlan70-pve03
iface vxlan70-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=70

auto vxlan70-int
iface vxlan70-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 80 (DMZ) =====
auto vxlan80-pve01
iface vxlan80-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=80

auto vxlan80-pve03
iface vxlan80-pve03 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.84 options:
        key=80

auto vxlan80-int
iface vxlan80-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VM Bridges =====
```

```
auto vmbr60
iface vmbr60 inet manual
    bridge_ports vxlan60-int
    bridge_stp off
    bridge_fd 0

auto vmbr70
iface vmbr70 inet manual
    bridge_ports vxlan70-int
    bridge_stp off
    bridge_fd 0

auto vmbr80
iface vmbr80 inet manual
    bridge_ports vxlan80-int
    bridge_stp off
    bridge_fd 0

source /etc/network/interfaces.d/*
```

Figure B.2 – Fichier /etc/network/interfaces complet – PVE-02 (192.168.10.213)

B.3 Configuration complète de PVE-03

```
auto lo
iface lo inet loopback

iface nic0 inet manual

auto vmbr0
iface vmbr0 inet static
    address 192.168.10.84/24
    gateway 192.168.10.36
    ovs_type OVSBridge
    ovs_ports nic0

auto nic0
iface nic0 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0

iface nic1 inet manual
iface nic2 inet manual
iface nic3 inet manual

# ===== COROSYNC VXLAN (key=20) =====
auto vxlan20-pve01
iface vxlan20-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=20
```

```
auto vxlan20-pve02
iface vxlan20-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=20

auto vxlan20-int
iface vxlan20-int inet static
    address 10.10.20.3/24
    ovs_type OVSIntPort
    ovs_bridge vmbr0

# ===== CEPH INTERNE VXLAN (key=30) =====
auto vxlan30-pve01
iface vxlan30-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=30

auto vxlan30-pve02
iface vxlan30-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=30

auto vxlan30-int
iface vxlan30-int inet static
    address 10.10.30.3/24
    ovs_type OVSIntPort
    ovs_bridge vmbr0

# ===== CEPH PUBLIC VXLAN (key=40) =====
auto vxlan40-pve01
iface vxlan40-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=40

auto vxlan40-pve02
iface vxlan40-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=40

auto vxlan40-int
iface vxlan40-int inet static
    address 10.10.40.3/24
```

```
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== MIGRATION LIVE VXLAN (key=50) =====
auto vxlan50-pve01
iface vxlan50-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=50

auto vxlan50-pve02
iface vxlan50-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=50

auto vxlan50-int
iface vxlan50-int inet static
    address 10.10.50.3/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== SAUVEGARDE VXLAN (key=90) =====
auto vxlan90-pve01
iface vxlan90-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=90

auto vxlan90-pve02
iface vxlan90-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=90

auto vxlan90-int
iface vxlan90-int inet static
    address 10.10.90.3/24
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 60 (Dev) =====
auto vxlan60-pve01
iface vxlan60-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=60
```

```
auto vxlan60-pve02
iface vxlan60-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=60

auto vxlan60-int
iface vxlan60-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 70 (Prod) =====
auto vxlan70-pve01
iface vxlan70-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=70

auto vxlan70-pve02
iface vxlan70-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=70

auto vxlan70-int
iface vxlan70-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VXLAN 80 (DMZ) =====
auto vxlan80-pve01
iface vxlan80-pve01 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.100 options:
        key=80

auto vxlan80-pve02
iface vxlan80-pve02 inet manual
    ovs_type OVSPort
    ovs_bridge vmbr0
    ovs_options type=vxlan options:remote_ip=192.168.10.213 options:
        key=80

auto vxlan80-int
iface vxlan80-int inet static
    ovs_type OVSPort
    ovs_bridge vmbr0

# ===== VM Bridges =====
```

```
auto vmbr60
iface vmbr60 inet manual
    bridge_ports vxlan60-int
    bridge_stp off
    bridge_fd 0

auto vmbr70
iface vmbr70 inet manual
    bridge_ports vxlan70-int
    bridge_stp off
    bridge_fd 0

auto vmbr80
iface vmbr80 inet manual
    bridge_ports vxlan80-int
    bridge_stp off
    bridge_fd 0

source /etc/network/interfaces.d/*
```

Figure B.3 – Fichier /etc/network/interfaces complet – PVE-03 (192.168.10.84)

Annexe C

Résultat d'Exécution Rundeck

Cette annexe présente le résultat d'une exécution complète d'un job Rundeck déclenché depuis l'interface web. Le job exécuté est **Provision VM**, qui invoque le playbook Ansible `provision_vm.yml` pour créer une machine virtuelle à partir du template Debian stocké dans le pool Ceph.

[CAPTURE À INSÉRER]

FIGURE C.1 – Résultat d'exécution d'un job Rundeck avec le journal Ansible complet.
[CAPTURE : dans Rundeck → Activity, ouvrir le détail d'une exécution récente, capturer la page affichant le statut **Succeeded** et les tâches Ansible déroulées avec leurs états (ok, changed, skipped)]

La Figure C.1 présente la page de détail d'une exécution Rundeck. On y observe le statut global **Succeeded**, le nom du job déclenché, la durée d'exécution et le journal Ansible complet listant chaque tâche avec son état (**ok**, **changed** ou **skipped**). Le bouton "Re-run" permet de relancer le même job avec les mêmes paramètres sans repasser par le formulaire de création.